



République Tunisienne

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

École Supérieure Privée d'ingénierie et de technologie

TEK-UP

ASQII

HEALTHTECH STARTUP

RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : Développement Mobile, Web et Multimédia

Réalisé par

Mohamed Achraf Ben Fredj

Développement d'un dashboard organisationnel multiniveau

Encadrant professionnel : **Monsieur Ahmed Messouad**

Directeur Technique

Encadrant académique : **Madame Malek Ben Youssef**

Maître assistant

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel , **Monsieur Ahmed Messouad**

Signature et cachet

Asqii
Imm , Byzance Bloc A
Les Berges du Lac - 1053 Tunis
MF : 1729622 / E / A / M / 000
Tél : 25 304 756 / 71 731 501

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrante académique, **Madame Malek Ben Youssef**

Signature

Dédicaces

Tout d'abord, Je tiens à remercier Dieu Tout-Puissant pour m'avoir donné la force et l'effort nécessaires pour accomplir ce projet de fin d'études de cycle ingénieur.

À mes chers parents, Moufid Ben Fredj et Awatef Haj Salah, Votre amour inépuisable, votre dévouement sans faille, et vos innombrables sacrifices ont été les fondations sur lesquelles j'ai construit mon parcours. Vous êtes ma source d'inspiration et ma force, et je vous exprime toute ma reconnaissance pour tout ce que vous avez fait pour moi.

À mes frères, Abdelatif Ben Fredj et Mohamed Amine Moufid Ben Fredj, et à mes sœurs, Bochra Ben Fredj, Meryam Ben Fredj, et Salma Aicha Ben Fredj, Votre soutien constant, votre complicité, et vos encouragements ont été essentiels dans mon cheminement. Je suis profondément honoré d'être entouré de frères et sœurs aussi formidables que vous.

À ma grand-mère, Rachida Saadallah, et à mon oncle, Sami Haj Salah, Votre sagesse et votre amour ont été une lumière guidante dans ma vie. Vos conseils et votre affection m'ont profondément marqué, et je vous en suis infiniment reconnaissant.

À mon oncle Mohamed Najib Ben Fredj, Merci pour ton soutien constant, tes encouragements, et ta présence précieuse tout au long de ce parcours. Ta générosité et ton soutien ont été des atouts inestimables.

À ma future femme, Safa Chaabane, Ton amour, ton soutien, et ta compréhension m'ont donné la force de persévérer. Tu es mon roc, et je suis immensément chanceux de t'avoir à mes côtés pour partager ce voyage.

À mon amie, Khouloud Chaabane, Ta bienveillance et ton amitié ont été des sources précieuses de réconfort. Merci pour ton soutien indéfectible et pour toujours croire en moi.

Enfin, Je tiens à exprimer ma profonde gratitude à toute ma grande famille et à mes amis. Votre présence, vos encouragements, et votre amour m'ont aidé à surmonter chaque défi. Que cette dédicace soit un témoignage de mon affection et de ma reconnaissance pour vous tous.

Mohamed Achraf Ben Fredj

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué de manière inestimable à la réalisation de ce projet.

Mes remerciements particuliers vont à M. Adel Hamdi, Directeur Général de ASQII et mon encadrant, pour son encadrement, sa disponibilité, et ses précieux conseils, ainsi qu'à Mme Malek Ben Youssef, mon encadrante à TEKUP, pour son enseignement de qualité et son soutien tout au long du projet.

Je remercie également toute l'équipe de ASQII pour leur soutien moral et technique, ainsi que les enseignants de TEKUP pour leur contribution à mon développement scientifique.

Enfin, je remercie chaleureusement les membres du jury pour l'honneur d'évaluer ce travail, ainsi que toutes les personnes qui, de près ou de loin, ont participé à la réalisation de ce projet.

Table des matières

Introduction générale	1
1 Présentation générale du projet	3
1.1 Introduction	4
1.2 Présentation de l'organisme d'accueil	4
1.2.1 ASQII	4
1.2.2 Domaines d'expertise	4
1.3 Client cible	5
1.4 Étude de l'existant	5
1.4.1 Solutions existantes	5
1.4.2 Problématique	6
1.4.3 Solution proposée	7
1.4.4 Méthodologie Agile	7
1.4.5 Méthodologie SCRUM	7
1.4.6 Méthodologie Adoptée	8
1.5 Conclusion	10
2 Spécification des besoins et conception	11
2.1 Introduction	12
2.2 Analyse des besoins	12
2.2.1 Identification des acteurs	12
2.2.2 Analyse des besoins fonctionnels	12
2.2.3 Analyse des besoins non fonctionnels	14
2.3 Diagramme de cas d'utilisation global	15
2.4 Diagramme de classes global	17
2.5 Conception graphique	18
2.5.1 Logo	19
2.5.2 Schéma de Navigation	19
2.5.3 Scénario et Maquette (Storyboard)	20
2.6 Pilotage du sujet avec Scrum	20
2.6.1 Backlog Produit	20
2.6.2 Planification des sprints	23

2.6.3	Diagramme de gantt	23
2.7	Conception architecturale	24
2.7.1	Architecture Logicielle :	24
2.8	Environnement de travail	27
2.8.1	Environnement de développement matériel	27
2.8.2	Environnement de développement logiciel	27
2.8.3	Frameworks et Technologies	28
2.9	Conclusion	33
3	Release 1 : Sprint 1/2 : Authentification et gestion des organisations, de contenu et des rapports Power BI	34
3.1	Introduction	35
3.2	Sprint 1 : Authentification et gestion des organisations	35
3.2.1	Objectifs du Sprint 1	35
3.2.2	Backlog du sprint 1	35
3.2.3	Analyse fonctionnelle du sprint 1	36
3.2.4	Diagramme de classes	40
3.2.5	Diagrammes de séquence	40
3.3	Réalisation Sprint 1	42
3.4	Sprint 2 : Gestion des dossiers et téléverser des pièces jointes et intégrer des rapports Power BI	45
3.4.1	Objectifs du Sprint 2	45
3.4.2	Backlog du sprint 2	45
3.4.3	Analyse fonctionnelle du sprint 2	46
3.4.4	Diagramme de classes	48
3.4.5	Diagrammes de séquence	49
3.5	Réalisation Sprint 2	50
3.6	Conclusion	52
4	Release 2 : Sprint 3/4 : gestion des réclamations, planification des événements et messagerie instantanée	53
4.1	Introduction	54
4.2	Sprint 3 : Gestion des réclamations et Planification des événements	54
4.2.1	Objectifs du Sprint 3	54
4.2.2	Backlog du sprint 3	54

4.2.3	Analyse fonctionnelle du sprint 3	55
4.2.4	Diagramme de classes	58
4.2.5	Diagrammes de séquence	58
4.3	Réalisation Sprint 3	60
4.4	Sprint 4 : Gestion des Discussions en Temps Réel	62
4.4.1	Objectifs du Sprint 4	62
4.4.2	Backlog du sprint 4	62
4.4.3	Analyse fonctionnelle du sprint 4	63
4.4.4	Diagramme de classes	65
4.4.5	Diagrammes de séquence	66
4.5	Réalisation Sprint 4	67
4.6	Conclusion	68
5	Intégration continue/Livraison continue, Déploiement continu	69
5.1	Introduction	70
5.2	Approche intégration, livraison, déploiement continus	71
5.2.1	Intégration continue (CI)	71
5.2.2	La livraison continue/déploiement continu (CD)	71
5.3	Définition de DevOps	71
5.4	Approche virtualisation/conteneurisation	71
5.4.1	La virtualisation	71
5.4.2	La Conteneurisation	72
5.5	Réalisation	72
5.5.1	Mise en Place de l'infrastructure CI/CD	72
5.5.2	Pipeline CI/CD Frontend	78
5.5.3	Pipeline CI/CD Backend	86
5.5.4	Pipeline de Déploiement	93
5.5.5	Monitoring	99
5.6	Conclusion	100
	Conclusion générale et perspectives	101
	Netographie	103

Table des figures

1.1	Logo ASQII	4
1.2	Le processus SCRUM adopté dans notre projet	9
2.1	Diagramme de cas d'utilisation global	16
2.2	Diagramme de classes global	17
2.3	Logo de notre application	19
2.4	Logo monochrome de notre application	19
2.5	Diagramme de navigation	20
2.6	Planification des sprints	23
2.7	Diagramme de gantt	24
2.8	Architecture logicielle	24
2.9	Architecture physique	26
2.10	Logo de React	29
2.11	Logo de TypeScript	29
2.12	Logo de Redux	29
2.13	Logo de React Router DOM	29
2.14	Logo de Axios	30
2.15	Logo de Node.js	30
2.16	Logo de Express.js	30
2.17	Logo de Sequelize	30
2.18	Logo de SQL	31
2.19	Logo de Nodemailer	31
2.20	Logo de Socket.io	31
2.21	Logo de Yarn	31
2.22	Logo de NPM	32
2.23	Logo de Vite	32
2.24	Logo de Git	32
2.25	Logo de Docker	33
2.26	Logo de Jest	33
3.1	Diagramme de cas d'utilisation pour l'authentification et la gestion des organisations	36
3.2	Diagramme de classes de l'authentification et de la gestion des organisations	40

3.3	Diagramme de séquence du scénario « Inscription de l'utilisateur »	40
3.4	Diagramme de séquence du scénario « Authentification »	41
3.5	Diagramme de séquence du scénario « Création d'une organisation »	41
3.6	Interface d'authentification	42
3.7	Interface d'accueil pour le super administrateur	42
3.8	Interface d'ajout à l'organisation lors de la première étape	43
3.9	Interface d'ajout à l'organisation lors de la deuxième étape	43
3.10	Interface d'ajout à l'organisation lors de la troisième étape	43
3.11	Interface de formulaire d'inscription	44
3.12	Interface de formulaire d'inscription	44
3.13	Interface de formulaire d'inscription	44
3.14	Diagramme de cas d'utilisation pour la gestion des dossiers, le téléversement des pièces jointes et l'intégration des rapports Power BI	46
3.15	Diagramme de classes du sprint 2	49
3.16	Diagramme de séquence du scénario « Création des dossiers »	49
3.17	Diagramme de séquence du scénario « Téléversement des pièces jointes »	50
3.18	Interface des listes de dossiers et pièces jointes en format grid	50
3.19	Interface des listes de dossiers et pièces jointes en format tableau	51
3.20	Interface de liste des rapports BI	51
3.21	Interface de téléversement d'une pièce jointe	51
3.22	Interface de suppression d'un dossier	52
3.23	Interface d'ajout d'un dossier	52
4.1	Diagramme de cas d'utilisation pour la gestion des réclamations et la planification des événements	55
4.2	Diagramme de classes de la gestion des réclamations et la planification des événements	58
4.3	Diagramme de séquence du scénario « Création des réclamations »	58
4.4	Diagramme de séquence du scénario « Répondre aux réclamations »	59
4.5	Diagramme de séquence du scénario « Planifier des événements »	59
4.6	Interface de la liste des réclamations	60
4.7	Interface de création de réclamation	60
4.8	Interface de réponse à une réclamation	61
4.9	Interface de la liste des événements	61
4.10	Interface de planification d'événements	61

4.11	Diagramme de cas d'utilisation pour la gestion des Discussions en Temps Réel	63
4.12	Diagramme de classes du sprint 4	66
4.13	Diagramme de séquence du scénario « Envoyer un message »	66
4.14	Diagramme de séquence du scénario « Création de groupe de discussion »	67
4.15	Interface des listes de discussions	67
4.16	Interface de création de groupe	68
4.17	Interface d'un exemple de discussion de groupe	68
5.1	Diagramme de l'architecture CI/CD	70
5.2	Capture des Trois Pipelines Récemment Créés dans Jenkins	72
5.3	Capture des Deux Projets SonarQube Récemment Créés	74
5.4	Configuration "sonar-project.properties" pour le Frontend	74
5.5	Configuration "sonar-project.properties" pour le Backend	74
5.6	Capture du Dépôt Docker « communic-flow » dans Nexus Repository	75
5.7	Capture de la Configuration des Cibles de Surveillance dans Prometheus	76
5.8	Capture des tableaux de bord Grafana pour la surveillance des applications	77
5.9	Capture d'écran de la pipeline Jenkins pour le frontend.	78
5.10	test.sh	79
5.11	ToolTip.test.tsx	79
5.12	Rapport détaillé de SonarQube montrant les bugs, vulnérabilités et code smells	80
5.13	Résultats de SonarQube pour le projet frontend « communicFlow »	81
5.14	Trend des Dépendances OWASP dans Jenkins	82
5.15	Résultats de OWASP Dependency-Check dans Jenkins	82
5.16	Capture d'écran montrant le dernier artefact archivé dans Jenkins	84
5.17	Code du Jenkinsfile utilisé pour le pipeline CI/CD frontend.	85
5.18	Capture d'écran de la pipeline Jenkins pour le backend.	86
5.19	Script de test Jest pour le backend.	87
5.20	Exemple de test unitaire avec Jest pour le backend.	87
5.21	Rapport détaillé de SonarQube montrant les bugs, vulnérabilités et code smells	88
5.22	Résultats de SonarQube pour le projet backend « communicFlow-back »	89
5.23	Résultats de OWASP Dependency-Check dans Jenkins	89
5.24	Rapport détaillé de OWASP Dependency-Check montrant les vulnérabilités identifiées	90
5.25	Capture d'écran montrant les artefacts archivés dans Jenkins pour le backend	91
5.26	Code du Jenkinsfile utilisé pour le pipeline CI/CD backend.	92

5.27 Pipeline de déploiement	93
5.28 Dockerfile pour le backend	93
5.29 Dockerfile pour le frontend	94
5.30 Configuration NGINX pour le frontend	94
5.31 Images Docker du frontend dans Nexus	95
5.32 Images Docker du backend dans Nexus	95
5.33 Fichier Docker Compose pour le VPS	96
5.34 Images Docker sur le VPS	96
5.35 Conteneurs Docker en cours d'exécution sur le VPS	96
5.36 Première partie du Jenkinsfile pour le pipeline de déploiement	97
5.37 Deuxième partie du Jenkinsfile pour le pipeline de déploiement	98
5.38 Tableau de bord Grafana pour le monitoring des performances et de la santé de Jenkins	99

Liste des tableaux

2.1	Backlog produit	23
2.2	Environnement de développement matériel	27
3.1	Backlog du sprint 1	35
3.2	Description textuelle du cas d'utilisation «Création d'une organisation»	37
3.3	Description textuelle du cas d'utilisation «Authentification»	38
3.4	Description textuelle du cas d'utilisation «Inscription de l'utilisateur»	39
3.5	Backlog du sprint 2	45
3.6	Description textuelle du cas d'utilisation «Création des dossiers»	47
3.7	Description textuelle du cas d'utilisation «Téléversement des pièces jointes»	48
4.1	Backlog du sprint 3	55
4.2	Description textuelle du cas d'utilisation «Création des réclamations»	56
4.3	Description textuelle du cas d'utilisation «Répondre aux réclamations»	57
4.4	Backlog du sprint 4	62
4.5	Description textuelle du cas d'utilisation «Envoyer message»	64
4.6	Description textuelle du cas d'utilisation «Créer groupe de discussion»	65

Liste des abréviations

—	API	=	A pplication P rogramming I nterface
—	CD	=	C ontinuous D elivery/ D eployment
—	CI	=	C ontinuous I ntegration
—	CNIP	=	C hambre N ationale I ndustrie P harmaceutique
—	CRM	=	C ustomer R elationship M anagement
—	DAO	=	D ata A ccess O bject
—	DevOps	=	D evelopment O perations
—	HTML	=	H yper T ext M arkup L anguage
—	HTTP	=	H yper T ext T ransfer P rotocol
—	IHM	=	I nterface H omme M achine
—	JVM	=	J ava V irtual M achine
—	ORM	=	O bject R elational M apping
—	OWASP	=	O pen W eb A pplication S ecurity P roject
—	SSH	=	S ecure S hell
—	SSL	=	S ecure S ockets L ayer
—	TLS	=	T ransport L ayer S ecurity
—	TS	=	T ype S cript
—	URL	=	U niform R esource L ocator
—	VPS	=	V irtual P rivate S erver

Introduction générale

Dans un monde où l'information est devenue le pilier de toute activité, la nécessité d'une communication fluide et d'une gestion efficace des données est impérative pour assurer la pérennité et le succès des organisations. Les avancées technologiques ont révolutionné la manière dont nous interagissons et partageons l'information, créant ainsi de nouvelles attentes en matière de connectivité et d'accessibilité. Dans ce contexte, notre projet de fin d'études s'inscrit dans une démarche visant à répondre à ces exigences croissantes en proposant une solution innovante et performante : le développement d'un dashboard organisationnel multiniveau.

Notre application aspire à transcender les barrières organisationnelles en établissant un lien entre divers acteurs institutionnels et leurs affiliés. Parmi ces acteurs figurent les ministères, les syndicats et les organisations professionnelles, tandis que leurs affiliés englobent les facultés, les entreprises et autres entités similaires. L'essence même de notre application réside dans sa capacité à relier ces affiliés à leurs utilisateurs, qu'ils soient des administrateurs, des membres de l'organisation ou d'autres parties prenantes, tout en leur octroyant des privilèges spécifiques conformément à leurs rôles et responsabilités.

Les objectifs principaux de notre projet sont multiples et comprennent la centralisation de l'information pour simplifier l'accès aux données, l'optimisation de la communication pour rationaliser les processus internes et renforcer la collaboration, la gestion précise des privilèges pour garantir la sécurité des données, et l'intégration de fonctionnalités avancées comme le partage de rapports analytiques via Power BI pour améliorer l'expérience utilisateur et favoriser une prise de décision éclairée. De plus, en incluant une messagerie instantanée, notre application offre un moyen efficace pour les utilisateurs de communiquer et de collaborer en temps réel, améliorant ainsi l'efficacité et la productivité des organisations.

En somme, notre application de tableau de bord représente une réponse proactive aux besoins de communication et de gestion de l'information au sein d'organisations diversifiées et complexes. La Chambre Nationale de l'Industrie Pharmaceutique (CNIP), étant notre premier client, a sollicité cette solution pour répondre à ses besoins spécifiques. Cette première application pratique du projet nous a permis de tester et de valider notre approche, démontrant ainsi l'efficacité et la pertinence de notre solution.

Le présent rapport sera organisé comme suit :

- **Cadre du projet** : Ce chapitre présente la société et ses domaines d'expertise, analyse la situation existante à travers des projets similaires, et décrit la méthodologie de gestion de projet ainsi que l'approche de conception adoptée.

- **Analyse des besoins** : Ce chapitre se concentre sur l'analyse et la spécification des exigences fonctionnelles et non fonctionnelles. Il inclut le diagramme de cas d'utilisation global, le diagramme de classes de l'application, la charte graphique, la planification des sprints et l'architecture de l'application.
- **Release 1 : Sprints 1 et 2** : Ce chapitre couvre la mise en place de l'infrastructure, le développement du module d'authentification, la création des organisations, le partage de contenu, et la gestion des rapports Power BI.
- **Release 2 : Sprints 3 et 4** : Ce chapitre traite de la gestion des réclamations, de la planification des événements, et du développement des fonctionnalités de messagerie instantanée.
- **DevOps et déploiement** : Ce chapitre explique les principes fondamentaux de DevOps, son rôle dans le développement et le déploiement d'applications, ainsi que les outils DevOps adaptés pour la mise en place de pipelines.

Enfin, nous clôturons notre rapport par une conclusion générale et des perspectives.

PRÉSENTATION GÉNÉRALE DU PROJET

Plan

1.1	Introduction	4
1.2	Présentation de l'organisme d'accueil	4
1.3	Client cible	5
1.4	Étude de l'existant	5
1.5	Conclusion	10

1.1 Introduction

Dans ce premier chapitre, nous commençons par présenter l'organisme qui nous accueille pour notre stage. Ensuite, nous examinons l'existant avec une analyse critique, tout en proposant des solutions pour surmonter les défis identifiés. Enfin, nous abordons en détail la méthodologie que nous sélectionnons pour guider notre démarche tout au long de ce projet.

1.2 Présentation de l'organisme d'accueil

Dans cette partie, nous présentons la société d'accueil en abordant son historique et ses domaines d'expertise.

1.2.1 ASQII

ASQII, fondée en 2021 par Adel Hamdi, est une startup tunisienne spécialisée dans l'amélioration des soins contre le cancer grâce à la digitalisation, l'analyse de données et l'automatisation des processus. Son objectif principal était de fournir aux patients une expérience de santé optimale en utilisant des technologies de pointe.[1] Le logo de l'entreprise est présenté dans la figure 1.1.



FIGURE 1.1 : Logo ASQII

1.2.2 Domaines d'expertise

Les solutions de ASQII peuvent être réparties dans cinq catégories.

1.2.2.1 Digitalisation des soins de santé

ASQII utilise la technologie pour améliorer les soins contre le cancer en rendant les informations médicales accessibles et exploitables.

1.2.2.2 Analyse de données

Leur expertise consiste à analyser les données médicales pour fournir des informations précieuses sur les tendances et les traitements, facilitant ainsi une prise de décision éclairée.

1.2.2.3 Automatisation des processus

ASQII développe des solutions innovantes pour automatiser les tâches répétitives dans l'oncologie, permettant aux professionnels de se concentrer sur les soins aux patients.

1.2.2.4 Amélioration de l'expérience patient

L'entreprise s'engage à améliorer l'expérience des patients atteints de cancer en simplifiant les procédures et en offrant un soutien personnalisé tout au long de leur parcours de traitement.

1.2.2.5 Recherche et développement

Leur laboratoire de prototypage explore continuellement de nouvelles idées pour le traitement du cancer, leur permettant d'apporter constamment des améliorations à leur offre.

1.3 Client cible

Le CNIP, notre premier client, réunit des entrepreneurs et des dirigeants d'entreprises du secteur de l'industrie pharmaceutique. Ils partagent le désir de mutualiser leurs énergies, d'échanger leurs compétences et ont l'ambition de contribuer activement au développement de l'économie tunisienne.

1.4 Étude de l'existant

L'étude de l'existant permet de mieux comprendre l'idée de notre projet. Elle nous aide à s'inspirer de l'existant pour dégager les principaux points dans le but d'enrichir notre application.

1.4.1 Solutions existantes

Nous avons examiné plusieurs applications qui proposaient des fonctionnalités similaires à celles que nous envisagions pour notre projet. Voici un résumé des principales plateformes étudiées, avec leurs points forts et leurs limites :

- **Google Drive** : Un service de stockage en ligne proposé par Google, permettant aux utilisateurs de stocker des fichiers dans le cloud, de synchroniser des fichiers sur plusieurs appareils et de partager des fichiers avec d'autres[2].
- **Zendesk** : C'est une plateforme de gestion des relations avec les clients qui inclut un système de gestion des réclamations, de suivi des tickets, et de gestion des SLA[3].
- **Google Calendar** : C'est un service de gestion de calendrier développé par Google, qui permet de gérer les événements, les rendez-vous, et de les partager avec d'autres utilisateurs[4].

- **Slack Enterprise Grid** : Il s'agit d'une plateforme de messagerie d'entreprise qui facilite la communication et la collaboration en temps réel entre les équipes. Elle offre une gestion avancée des canaux, une intégration avec d'autres outils de productivité et facilite la connexion entre les acteurs[5].

En étudiant ces applications, nous avons pu mieux comprendre leurs forces et leurs faiblesses, ce qui nous a permis de concevoir une solution se démarquant sur le marché en offrant une expérience utilisateur complète et adaptée aux besoins des utilisateurs, notamment ceux du CNIP et de ses affiliés, les laboratoires pharmaceutiques.

1.4.2 Problématique

Toute entreprise ou organisation professionnelle, comme le CNIP, a besoin de communiquer efficacement. La réputation de notre organisation dépend de la qualité de la communication interne et externe, que ce soit avec les affiliés (les laboratoires pharmaceutiques) ou les membres de ces affiliés. Le problème de la communication dans les organisations reste un grand défi malgré toutes les mesures prises pour faire passer l'information. Pour adopter une méthodologie de travail complète, il est crucial d'améliorer la communication interne.

Pour le CNIP, il est crucial de transmettre les informations les plus pertinentes de l'organisation à ses affiliés, les laboratoires pharmaceutiques, ainsi que ces affiliés à leurs membres et équipes respectifs, afin d'assurer une communication efficace. Une communication inefficace peut compromettre la performance de l'entreprise et nuire à l'atteinte de ses objectifs.

La confiance se forme naturellement lorsque l'information est transmise en temps réel. En améliorant la communication au quotidien, on améliore considérablement la productivité de l'entreprise. Une relation de confiance peut se développer, les équipes étant immédiatement informées et capables de s'adapter aux nouvelles réformes ou informations importantes.

Cette circulation de l'information entre les différentes équipes s'inscrit dans une stratégie de communication efficace à travers un tableau de bord qui organise la relation et la communication entre les organisations et leurs affiliés, ainsi qu'avec les membres de différents affiliés. De plus, le partage de documents et de rapports Power BI est intégré pour faciliter l'accès à l'information. Enfin, une messagerie instantanée est disponible pour permettre une communication en temps réel en cas d'informations importantes.

Associé à une bonne communication et au partage d'informations, cela ne peut qu'avoir un impact positif sur les résultats de l'organisation, en particulier pour le CNIP.

1.4.3 Solution proposée

La communication efficace entre le CNIP, ses affiliés et les membres de ces affiliés est cruciale pour assurer la coordination des équipes, la transparence des informations, la prise de décisions rapides et la productivité globale. Notre solution propose un tableau de bord centralisé qui facilite cette communication en offrant la possibilité d'attribuer des privilèges à chaque utilisateur, de partager des informations, des documents et des rapports Power BI avec tous les membres concernés. De plus, la fonctionnalité de messagerie instantanée permet l'envoi de messages, d'alertes et de mises à jour en temps réel aux employés, facilitant ainsi la diffusion d'informations importantes et favorisant une collaboration efficace.

Après une analyse approfondie des solutions existantes, nous avons constaté qu'il existe plusieurs façons de communiquer avec les employés via des applications existantes, mais aucune ne regroupe toutes ces fonctionnalités de manière intégrée. Chaque application est spécialisée dans une fonctionnalité spécifique, ce qui limite la capacité des organisations à centraliser leur communication et leur collaboration. Afin de surmonter ces limites, nous avons décidé de créer une application qui répond à tous nos objectifs en offrant une application centralisée et complète pour la gestion de la communication et des informations au sein de l'organisation et avec ses affiliés, en particulier pour le CNIP.

1.4.4 Méthodologie Agile

La méthodologie Agile est une approche de gestion de projet qui découpe le projet en une série d'objectifs réalisables et itératifs. Cette approche a été développée en réponse aux méthodes de gestion de projet traditionnelles, jugées trop lourdes, lentes et rigides. Agile privilégie l'adaptabilité et la collaboration à travers des cycles courts appelés itérations ou sprints, qui durent généralement de 1 à 4 semaines. Cette méthode permet de s'adapter rapidement aux changements et de livrer des produits de qualité.[6]

1.4.5 Méthodologie SCRUM

SCRUM est une méthode Agile spécifique qui facilite la collaboration au sein des équipes et les aide à accomplir des tâches importantes. Elle repose sur un cadre structuré composé de valeurs, de rôles et de règles qui permettent de se concentrer sur chaque étape du projet et de s'améliorer progressivement. SCRUM est caractérisée par des cycles de travail appelés "sprints", des réunions quotidiennes de coordination "Daily Scrum" et des réunions de rétrospective à la fin de chaque sprint.[7]

1.4.6 Méthodologie Adoptée

1.4.6.1 Cadre du Projet

Dans le cadre de notre projet, nous avons appliqué la méthodologie SCRUM afin de répondre aux exigences spécifiques des utilisateurs tout en respectant les délais et les coûts. Cette approche nous a permis de décomposer le projet en itérations courtes et itératives, chacune se concentre sur la livraison de fonctionnalités incrémentielles.

1.4.6.2 Processus SCRUM Mis en Place

- **Initialisation du Projet :**

- Le projet a débuté avec le Sprint 0, une phase de préparation et de mise en œuvre. Cette phase comprenait la conception et l'architecture du projet, la configuration de l'environnement de développement, ainsi que l'intégration des outils de suivi et de gestion [7].

- **Gestion des Fonctionnalités :**

- Les fonctionnalités du projet ont été répertoriées et décrites sous forme de "User Stories" dans le Product Backlog. Chaque sprint commençait par une réunion de planification "Sprint Planning" où l'équipe décidait quels user stories seraient développées durant le sprint. Ces user stories constituaient alors le Sprint Backlog [8].

- **Exécution des Sprints :**

- Chaque sprint, d'une durée de deux semaines, était marqué par des réunions quotidiennes appelées "Daily Scrum". Ces réunions visaient à synchroniser les activités de l'équipe et à assurer une communication fluide. À la fin de chaque sprint, une revue de sprint "Sprint Review" était organisée pour présenter l'avancement du projet et obtenir des retours des parties prenantes [8].

- **Rétrospectives et Améliorations :**

- Après chaque sprint, une rétrospective "Sprint Retrospective" était tenue pour évaluer les processus et identifier les points d'amélioration. Cette démarche nous a permis de mettre en place des actions correctives et d'optimiser notre méthodologie pour les sprints suivants [8].

Comme l'illustre la figure 1.2, le processus SCRUM adopté dans notre projet.

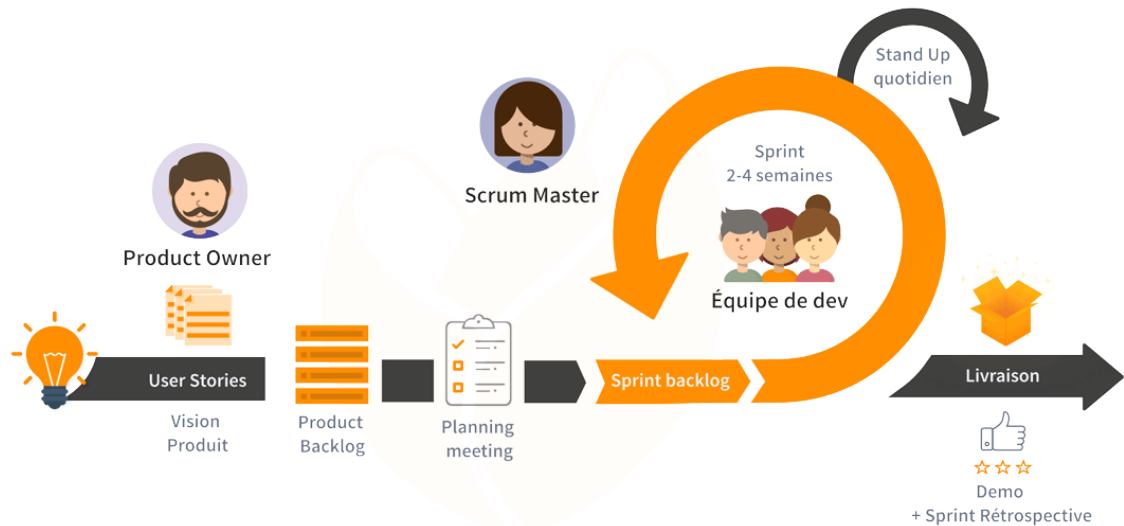


FIGURE 1.2 : Le processus SCRUM adopté dans notre projet

1.4.6.3 Rôles et Responsabilités

- **Product Owner :**

- Le Product Owner était responsable de définir les exigences du projet et de prioriser les éléments du Product Backlog. Il travaillait en étroite collaboration avec les parties prenantes pour s'assurer que les besoins des utilisateurs étaient bien compris et pris en compte [7].

- **Scrum Master :**

- Le Scrum Master jouait un rôle crucial en facilitant les cérémonies SCRUM et en s'assurant que l'équipe respectait les principes et les pratiques de la méthodologie SCRUM. Il aidait également à résoudre les obstacles qui pouvaient entraver le progrès de l'équipe [7].

- **Équipe de Développement :**

- L'équipe de développement était auto-organisée et interdisciplinaire, composée de développeurs, testeurs et autres professionnels nécessaires à la réalisation des objectifs du sprint. Chaque membre connaissait clairement son rôle et ses responsabilités, contribuant ainsi à l'efficacité de l'équipe [9].

1.4.6.4 Cycle de Développement SCRUM

Le cycle de développement SCRUM mis en place dans notre projet suivait une structure rigoureuse. À la fin de chaque sprint, nous présentions les fonctionnalités développées aux utilisateurs pour obtenir leurs retours. Ces retours étaient ensuite intégrés dans le cycle suivant, garantissant ainsi une amélioration continue et une adéquation optimale aux besoins des utilisateurs [8].

L'adoption de la méthodologie SCRUM dans notre projet a permis de structurer notre travail de manière efficace et d'assurer une livraison régulière et qualitative des fonctionnalités. Les pratiques de SCRUM ont favorisé une meilleure collaboration au sein de l'équipe et ont aidé à répondre de manière agile aux défis rencontrés tout au long du projet.

1.5 Conclusion

La définition des repères du projet constitue une étape essentielle du premier chapitre. Après avoir présenté l'organisme d'accueil, nous avons déterminé le cadre du projet, les limites du travail existant, ainsi que la méthodologie à suivre pendant le stage. Le chapitre suivant abordera l'analyse et la spécification des besoins, les diagrammes de cas d'utilisation et de classes, la charte graphique, ainsi que la gestion du projet avec Scrum, la conception architecturale et l'environnement de travail.

SPÉCIFICATION DES BESOINS ET CONCEPTION

Plan

2.1	Introduction	12
2.2	Analyse des besoins	12
2.3	Diagramme de cas d'utilisation global	15
2.4	Diagramme de classes global	17
2.5	Conception graphique	18
2.6	Pilotage du sujet avec Scrum	20
2.7	Conception architecturale	24
2.8	Environnement de travail	27
2.9	Conclusion	33

2.1 Introduction

Ce chapitre représente notre sprint initial, où nous définissons précisément les besoins des utilisateurs et réalisons une analyse approfondie des exigences fonctionnelles et non fonctionnelles. Ensuite, nous présentons les diagrammes de cas d'utilisation et de classes, ainsi que la charte graphique. Nous exposons également le backlog du produit et la planification de nos sprints. Enfin, nous détaillons l'architecture proposée et l'environnement de travail associé.

2.2 Analyse des besoins

Dans cette section, nous commençons par identifier les parties prenantes de l'application, puis nous exposons les exigences fonctionnelles et non fonctionnelles.

2.2.1 Identification des acteurs

Pendant notre travail, nous identifions trois catégories d'acteurs.

- **Super Administrateur :**

- **Rôle :** Responsable de l'organisation centrale.
- **Responsabilités :** Créer et gérer les organisations affiliées, gérer les utilisateurs, créer des groupes de discussion, créer des dossiers, partager des fichiers et des rapports Power BI, gérer les réclamations et organiser des événements.

- **Administrateur de l'organisation affiliée : « Administrateur »**

- **Rôle :** Responsable d'une organisation affiliée.
- **Responsabilités :** Inviter et inscrire des membres, créer des groupes de discussion pour son organisation, créer des dossiers, partager des fichiers et des rapports Power BI, organiser des événements, envoyer des réclamations au Super Administrateur, gérer les membres de son organisation.

- **Membre de l'organisation affiliée : « Membre »**

- **Rôle :** Utilisateur faisant partie d'une organisation affiliée.
- **Responsabilités :** Discuter avec les autres membres, partager des fichiers et des rapports Power BI (si autorisé), consulter et potentiellement créer des événements (si autorisé).

2.2.2 Analyse des besoins fonctionnels

Voici ce que le système doit faire en réponse aux demandes de l'utilisateur. Les exigences fonctionnelles sont les suivantes :

(i) Création et gestion des organisations affiliées :

- Le Super Administrateur peut créer des organisations affiliées pour structurer l'organisation centrale.
- Le Super Administrateur peut inviter des utilisateurs à devenir administrateurs d'organisations affiliées via des invitations par email.

(ii) Authentification et inscription :

- Les utilisateurs peuvent s'authentifier sur la plateforme via un système sécurisé.
- Les administrateurs reçoivent une invitation par email contenant une URL pour compléter leur inscription via un formulaire en ligne faisant partie de l'application.
- Les administrateurs peuvent inviter d'autres utilisateurs à devenir membres de leur organisation.
- Les membres reçoivent une invitation pour rejoindre une organisation affiliée et compléter leur inscription en ligne.

(iii) Groupes de discussion :

- Le Super Administrateur peut créer des groupes de discussion pour faciliter la communication interne.
- Un groupe de discussion spécifique est automatiquement généré lors de la création d'une organisation affiliée pour assurer une communication fluide.
- Les administrateurs peuvent créer des groupes de discussion réservés aux membres de leur organisation pour une communication ciblée.

(iv) Partage de fichiers et rapports Power BI :

- Un dossier de fichiers partagés est automatiquement créé pour chaque organisation affiliée.
- Un dossier de rapports Power BI est automatiquement créé pour chaque organisation affiliée.
- Le Super Administrateur peut créer des dossiers supplémentaires dans les sections de fichiers partagés et de rapports Power BI.
- Les administrateurs peuvent créer des dossiers dans les sections de fichiers partagés et de rapports Power BI de leur organisation.
- Les membres peuvent ajouter des fichiers ou des rapports Power BI si l'administrateur leur en donne le privilège.

(v) Gestion des réclamations :

- Le Super Administrateur peut répondre aux réclamations envoyées uniquement par les administrateurs des organisations affiliées pour gérer efficacement les plaintes.

- Les administrateurs peuvent envoyer des réclamations au Super Administrateur pour résoudre les problèmes rapidement.

(vi) Organisation d'événements :

- Le Super Administrateur peut créer des événements dans le calendrier de la plateforme pour organiser les activités centrales.
- Les administrateurs peuvent créer des événements visibles pour les membres de leur organisation et le Super Administrateur.
- Les membres peuvent créer des événements si l'administrateur leur en donne le privilège.

(vii) Gestion des utilisateurs :

- Le Super Administrateur peut modifier ou supprimer les administrateurs et les membres des organisations affiliées pour maintenir une gestion à jour des utilisateurs.
- Les administrateurs peuvent ajouter ou supprimer des membres de leur organisation.

2.2.3 Analyse des besoins non fonctionnels

Après avoir défini les besoins fonctionnels, il est essentiel de présenter les exigences non fonctionnelles de notre application. Pour garantir le bon fonctionnement de notre application, il est crucial de respecter ces exigences :

Performance

- La plateforme doit pouvoir gérer un grand nombre d'utilisateurs simultanément sans subir de dégradation significative des performances.
- Les temps de réponse pour les actions courantes (connexion, création de groupes, partage de fichiers) doivent être inférieurs à 2 secondes.

Sécurité

- Toutes les communications doivent être chiffrées à l'aide du protocole SSL/TLS.
- Les données sensibles doivent être stockées de manière sécurisée et chiffrée.
- Un système de gestion des accès basé sur les rôles doit être mis en place pour garantir que les utilisateurs n'accèdent qu'aux fonctionnalités et données autorisées.

Disponibilité

- La plateforme doit être disponible 24/7 avec un temps de disponibilité minimum de 99,9
- Un système de sauvegarde et de récupération des données doit être en place pour assurer la continuité des services en cas de panne.

Utilisabilité

- L'interface utilisateur doit être intuitive et facile à utiliser pour tous les types d'utilisateurs (Super Administrateur, Administrateurs, Membres).
- Des guides d'utilisation et des tutoriels doivent être fournis pour aider les utilisateurs à comprendre comment utiliser la plateforme.

Scalabilité

- La plateforme doit être conçue pour être évolutive et pouvoir s'adapter à une augmentation du nombre d'utilisateurs et d'organisations affiliées sans nécessiter de modifications majeures de l'infrastructure.

Maintenance

- La plateforme doit être conçue de manière à faciliter les mises à jour et les corrections de bugs sans interruption prolongée des services.
- Un système de journalisation doit être mis en place pour suivre les actions des utilisateurs et aider à diagnostiquer et résoudre les problèmes rapidement.

2.3 Diagramme de cas d'utilisation global

Pour présenter les fonctionnalités de notre système de manière structurée, nous avons recours au diagramme de cas d'utilisation issu du langage de modélisation UML.

Les diagrammes de cas d'utilisation, qui font partie des diagrammes UML, fournissent une vue d'ensemble du comportement fonctionnel d'un système logiciel. Ils sont particulièrement efficaces pour les présentations destinées à la direction ou aux parties prenantes d'un projet. Toutefois, lors du développement, les cas d'utilisation spécifiques sont plus adéquats. Un cas d'utilisation désigne une interaction distincte entre un utilisateur (humain ou machine) et le système[10].

La figure 2.1 présente le diagramme général des cas d'utilisation de notre application. Ce schéma montre les différents acteurs ainsi que les cas d'utilisation associés à chacun d'eux.

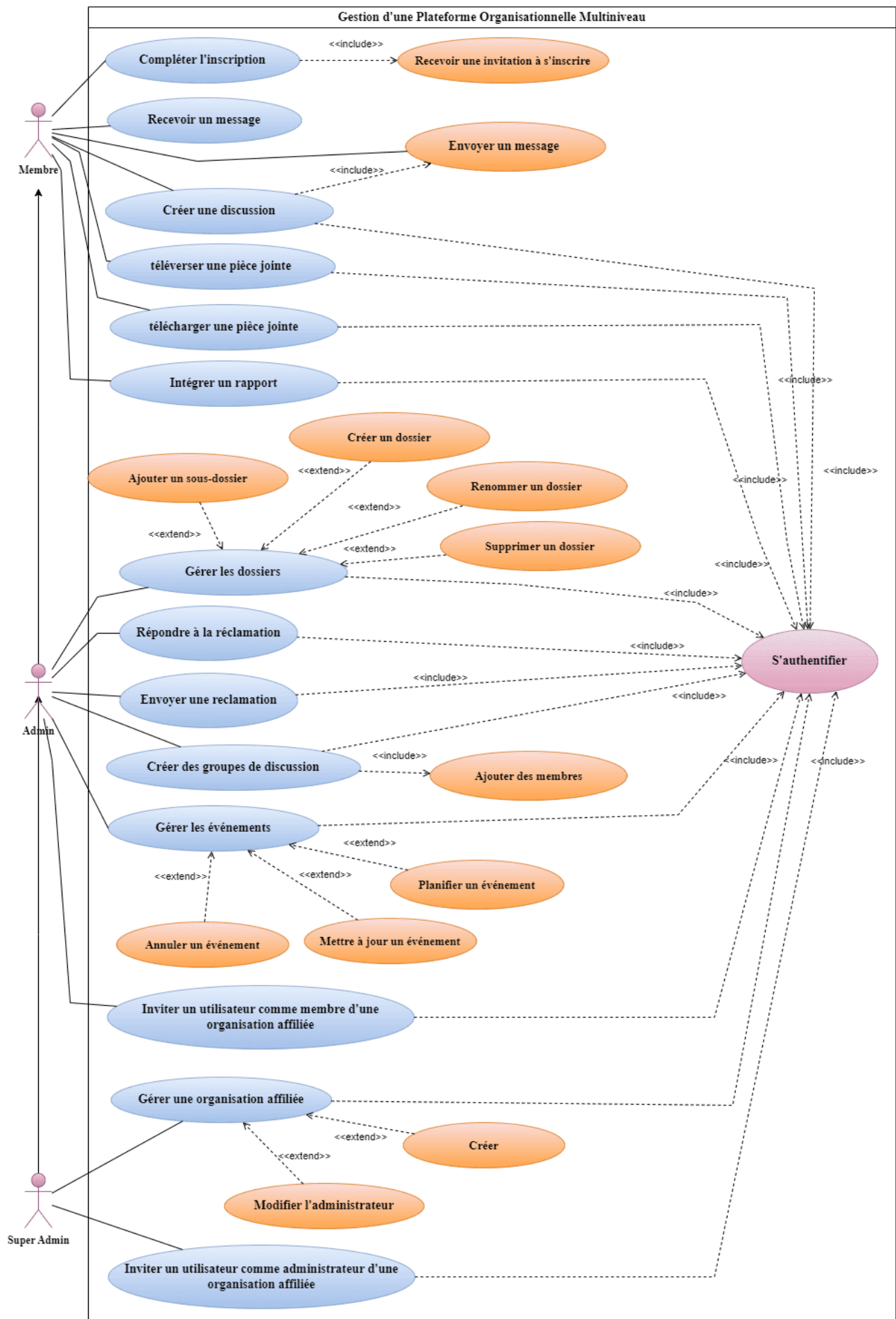


FIGURE 2.1 : Diagramme de cas d'utilisation global

2.4 Diagramme de classes global

Le diagramme de classes fournit une représentation abstraite des objets du système et de leurs interactions, ce qui facilite l'implémentation des cas d'utilisation. La Figure 2.2 illustre en détail les différentes classes du diagramme de classes global.

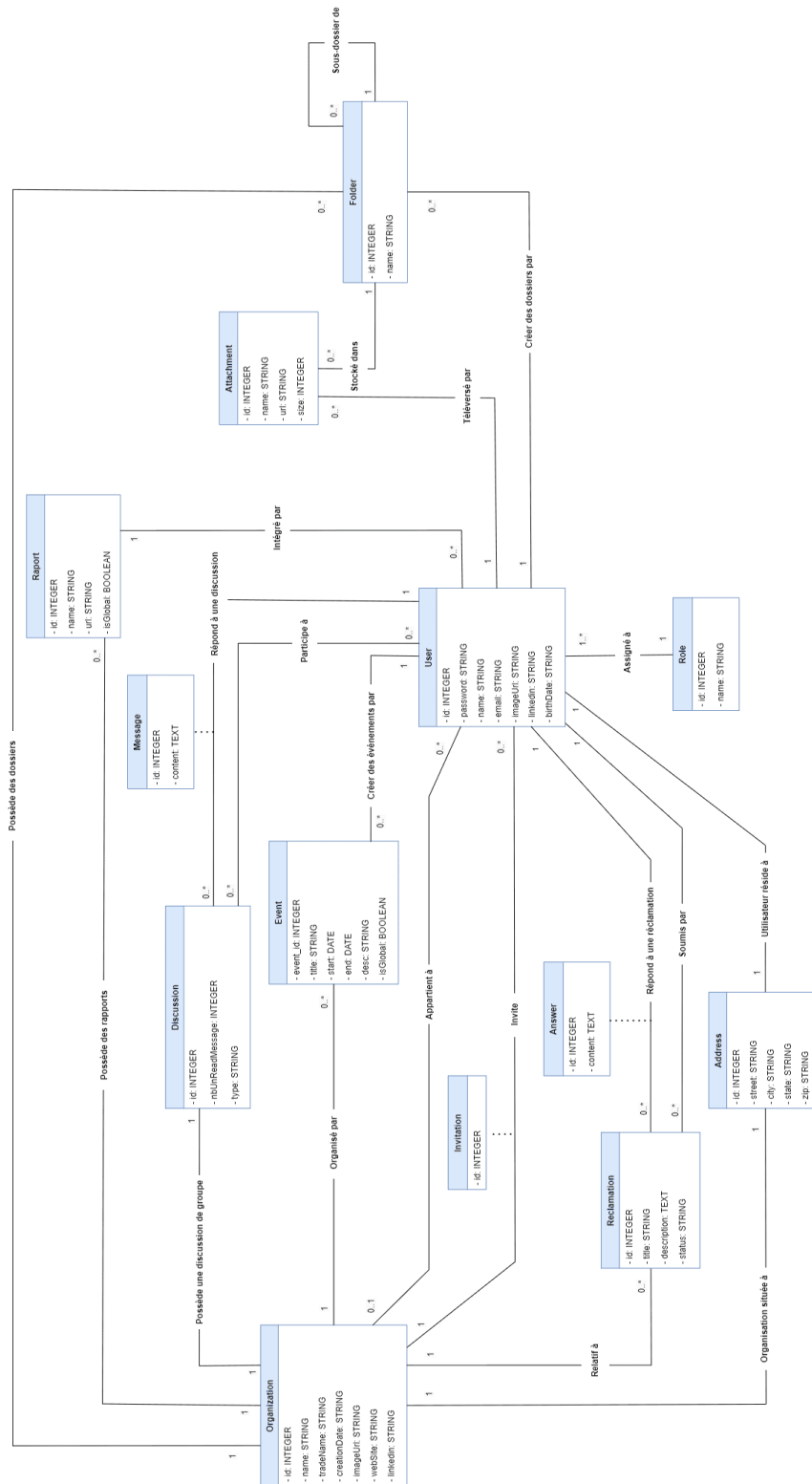


FIGURE 2.2 : Diagramme de classes global

Nous allons maintenant procéder à la description des classes :

- **Organization** : représente les organisations avec leurs informations telles que le nom, le nom commercial, la date de création, etc.
- **User** : identifie les utilisateurs avec leurs informations personnelles comme le nom, l'email, le mot de passe, etc.
- **Discussion** : désigne les discussions, avec des informations telles que le dernier message et le nombre de messages non lus.
- **Message** : contient le contenu des messages échangés dans les discussions.
- **Event** : décrit les événements organisés par l'organisation, avec des détails comme le titre, la date de début et de fin, la description, etc.
- **Reclamation** : gère les réclamations faites par les utilisateurs avec un titre, une description et un statut.
- **Answer** : englobe les réponses apportées aux réclamations et aux événements.
- **Raport** : représente les rapports générés, incluant des informations comme le nom, l'URL et le propriétaire.
- **Attachment** : contient les pièces jointes avec des détails comme le nom, l'URL et la taille.
- **Folder** : organise les dossiers qui peuvent contenir des pièces jointes et d'autres dossiers.
- **Address** : stocke les informations d'adresse associées à une organisation.
- **Role** : définit les rôles des utilisateurs dans le système.

2.5 Conception graphique

La conception des maquettes et du logo est essentielle pour garantir une identité visuelle cohérente et professionnelle à travers toute la plateforme.

Cette étape consiste à préparer quelques interfaces graphiques de l'application à l'aide de l'outil de conception Figma. Elle permet d'évaluer la compréhension du projet par le développeur et de recueillir le feedback du client. Les échanges qui en découlent entre l'utilisateur final et le développeur permettent d'ajuster précisément les besoins et de garantir une conception exacte. Les interfaces graphiques favorisent une interaction plus fluide et précise avec l'utilisateur final, l'aidant ainsi à exprimer clairement ses attentes.

2.5.1 Logo

Le logo de notre application est présenté dans la Figure 2.3 dans sa version normale et dans la Figure 2.4 dans sa version monochrome.

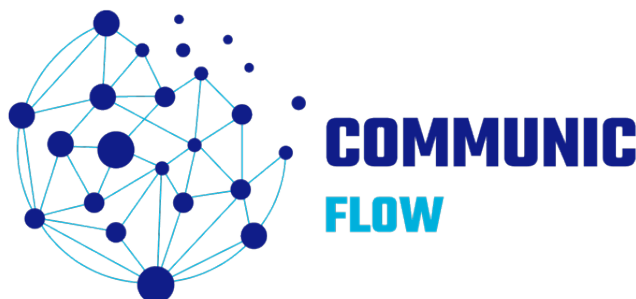


FIGURE 2.3 : Logo de notre application

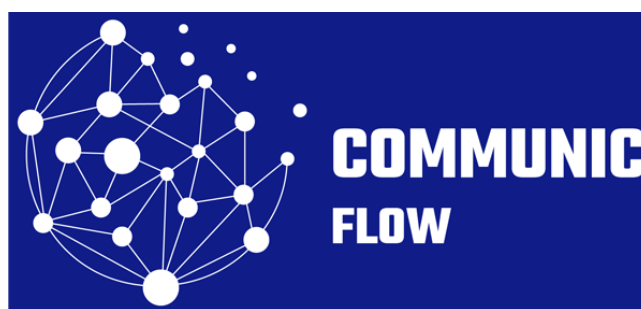


FIGURE 2.4 : Logo monochrome de notre application

2.5.2 Schéma de Navigation

Le schéma de navigation de l'application, illustré dans la Figure 2.5, représente la structure organisationnelle des différentes pages et fonctionnalités. Il a été soigneusement conçu pour garantir une expérience utilisateur intuitive et facile à utiliser. Chaque chemin de navigation a été pensé afin d'assurer une progression fluide à travers l'application, permettant aux utilisateurs de trouver rapidement ce dont ils ont besoin. Ce schéma constitue un guide visuel essentiel pour comprendre la manière dont les différentes parties de l'application sont interconnectées et naviguées.

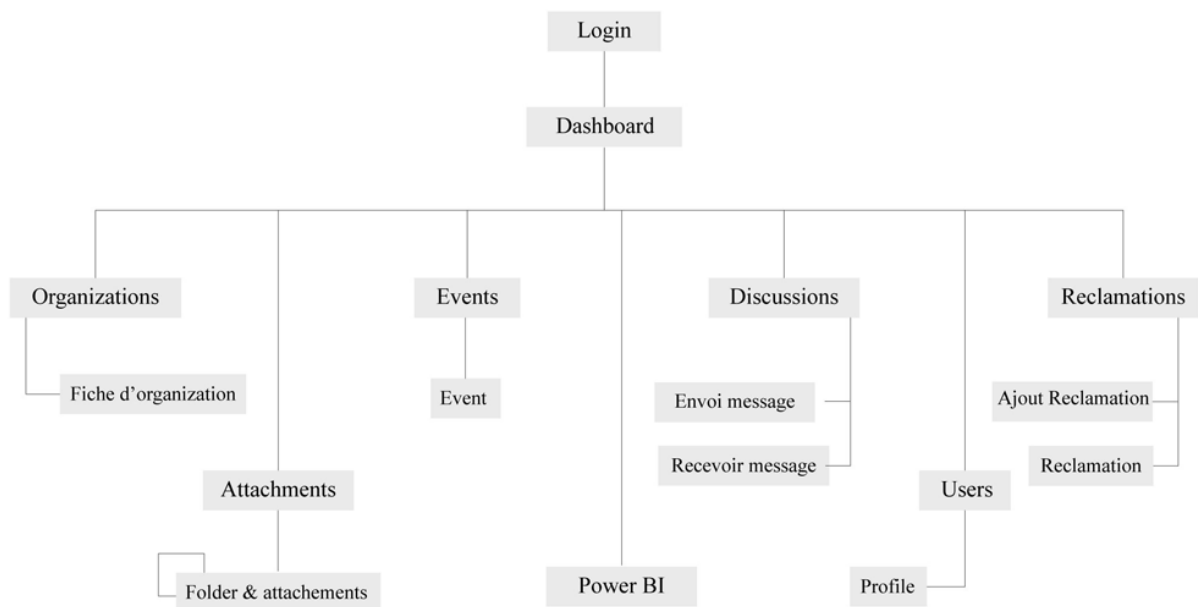


FIGURE 2.5 : Diagramme de navigation

2.5.3 Scénario et Maquette (Storyboard)

Le storyboard présente de manière interactive les divers scénarios d'utilisation de l'application. Chaque étape du parcours utilisateur est représentée, permettant ainsi de visualiser comment les utilisateurs interagissent avec l'interface dans différentes situations. Cela garantit que l'expérience utilisateur est fluide et cohérente, tout en permettant d'identifier les points forts et les éventuelles améliorations de l'interface.

2.6 Pilotage du sujet avec Scrum

2.6.1 Backlog Produit

Dans cette section, nous explorons en détail le Backlog Produit de notre projet. Le backlog produit sert à recueillir l'ensemble des besoins du client que l'équipe projet doit mettre en œuvre. Il comprend une liste des fonctionnalités attendues pour le produit, classées par ordre de priorité.

Ces fonctionnalités sont formulées sous forme d'histoires utilisateurs (user stories) pour définir les différents aspects du projet. Le tableau 2.1 présente un résumé du backlog produit, offrant ainsi un aperçu des fonctionnalités à développer dans notre projet.

Fonctionnalité	Description	Priorité
Authentification	- En tant qu'utilisateur, je peux m'authentifier sur la plateforme via un système sécurisé.	1
	- En tant qu'utilisateur, je peux recevoir une invitation par email pour rejoindre une organisation affiliée et compléter mon inscription comme administrateur.	1
	- En tant qu'utilisateur, je dois recevoir une invitation pour rejoindre une organisation affiliée et compléter mon inscription comme membre.	1
Gestion des Organisations	- En tant que Super Administrateur, je peux créer des organisations affiliées pour structurer l'organisation centrale.	1
	- En tant que Super Administrateur, je peux inviter des utilisateurs à devenir administrateurs d'organisations affiliées via l'email.	2
	- En tant qu'Administrateur, je peux inviter d'autres utilisateurs à devenir membres de mon organisation affiliée.	3
Création de dossiers, Partage de pièces jointes et Rapports Power BI	- En tant que Super Administrateur, je peux créer des dossiers ou ajouter des sous-dossiers supplémentaires sous le dossier principal.	1
	- En tant que Super Administrateur, je peux téléverser ou télécharger des pièces jointes dans le dossier principal ou dans les sous-dossiers de ce dossier.	1
	- En tant que Super Administrateur, je peux renommer un dossier ou sous-dossier.	3
	- En tant que Super Administrateur, je peux supprimer un dossier ou sous-dossier.	2
	- En tant que Super Administrateur, je peux renommer un fichier.	3
	- En tant que Super Administrateur, je peux supprimer un fichier.	2
	- En tant qu'Administrateur, je peux créer des dossiers supplémentaires ou des sous-dossiers dans le dossier de mon organisation affiliée.	1
	- En tant qu'Administrateur, je peux télécharger des dossiers de mon organisation affiliée.	1
	- En tant qu'Administrateur, je peux téléverser ou télécharger des fichiers dans le dossier de mon organisation ou dans les sous-dossiers de ce dossier.	1

	- En tant que membre, je peux téléverser ou télécharger des fichiers dans mon organisation affiliée.	1
	- En tant que Super Administrateur, je peux intégrer des rapports Power BI.	1
	- En tant qu'Administrateur, je peux intégrer des rapports Power BI.	1
	- En tant que membre, je peux intégrer des rapports Power BI.	1
Gestion des Réclamations	- En tant que Super Administrateur, je peux répondre aux réclamations envoyées.	1
	- En tant qu'Administrateur, je peux envoyer des réclamations au Super Administrateur.	1
	- En tant que membre, je peux consulter les réclamations de mon organisation affiliée.	1
Organisation d'Événements	- En tant que Super Administrateur, je peux planifier des événements globaux pour organiser les activités centrales.	1
	- En tant que Super Administrateur, je peux mettre à jour des événements globaux.	2
	- En tant que Super Administrateur, je peux annuler des événements globaux.	1
	- En tant qu'Administrateur, je peux planifier des événements pour mon organisation.	1
	- En tant qu'Administrateur, je peux mettre à jour des événements de mon organisation affiliée.	2
	- En tant qu'Administrateur, je peux annuler des événements de mon organisation affiliée.	2
	- En tant que membre, je peux consulter les événements globaux et ceux de mon organisation affiliée.	1
Gestion des Discussions en temps réel	- En tant que Super Administrateur, je peux discuter avec tous les utilisateurs.	1
	- En tant qu'Administrateur, je peux discuter avec tous les membres de mon organisation affiliée.	1
	- En tant qu'Administrateur, je peux discuter avec le Super Administrateur et les autres administrateurs des autres organisations affiliées.	1

	- En tant que membre, je peux discuter avec l'administrateur de mon organisation affiliée.	1
	- En tant que membre, je peux discuter avec tous les membres de mon organisation affiliée.	1
	- En tant que Super Administrateur, je peux créer des groupes de discussion.	1
	- En tant qu'Administrateur, je peux créer des groupes de discussion avec n'importe quel membre de mon organisation affiliée.	1
	- En tant que Super Administrateur, je peux retirer un membre des groupes de discussion.	1

TABLEAU 2.1 : Backlog produit

2.6.2 Planification des sprints

La planification du travail à réaliser durant le Sprint est effectuée lors de la séance de Planification de Sprint, impliquant toute l'Équipe Scrum. Ce processus collaboratif permet d'élaborer un plan détaillé pour le Sprint à venir. La figure 2.6 fournit une vue d'ensemble de la répartition des releases dans la planification du projet. Chaque release est associée à des sprints spécifiques, chacun se concentrant sur différents modules de l'application. Cette répartition structurée permet une approche de développement incrémentale, garantissant l'achèvement en temps opportun des fonctionnalités clés.



FIGURE 2.6 : Planification des sprints

2.6.3 Diagramme de gantt

La figure 2.7 résume visuellement l'état d'avancement des différentes tâches qui constituent notre projet.

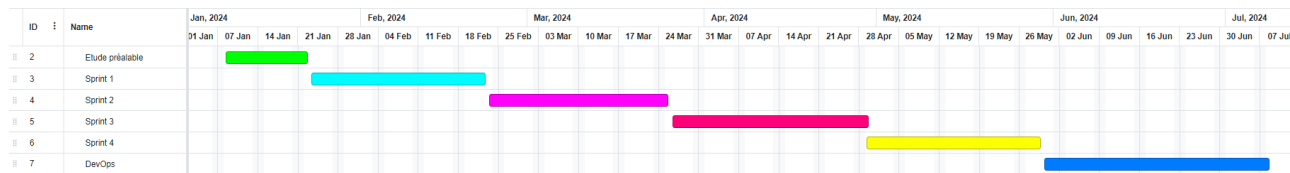


FIGURE 2.7 : Diagramme de gantt

2.7 Conception architecturale

2.7.1 Architecture Logicielle :

L'architecture logicielle de notre application est conçue pour assurer une séparation claire des responsabilités entre les différentes couches de l'application. Elle comprend des composants clients et serveurs qui interagissent de manière fluide pour offrir une expérience utilisateur efficace et réactive, comme illustré dans la figure 2.8.

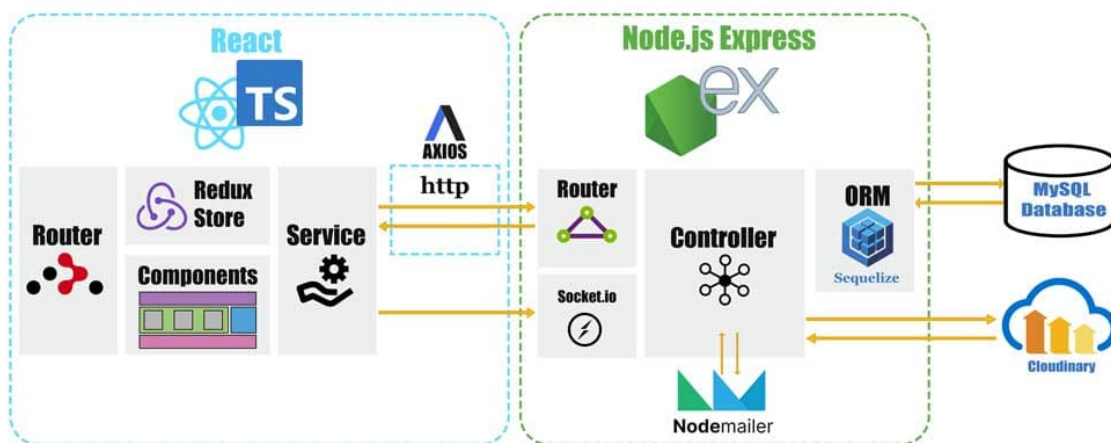


FIGURE 2.8 : Architecture logicielle

2.7.1.1 Client-Side (React)

- **Router** : Utilisé pour la navigation entre les différentes vues de l'application. Il permet de gérer les différentes routes et les composants associés.
- **Redux Store** [11] : Gère l'état global de l'application. Redux est utilisé pour centraliser l'état et permettre une gestion efficace des données partagées entre les composants.
- **Components** [12] : Réutilisables et modulaires, ces composants constituent l'interface utilisateur. Ils sont responsables de l'affichage des données et des interactions avec les utilisateurs.
- **Service** [13] : Contient les fonctions qui effectuent des appels API via Axios. Ces services permettent de communiquer avec le serveur pour récupérer ou envoyer des données.

- **Communication** [13] : Les appels HTTP sont effectués via Axios pour interagir avec le backend. Les données sont envoyées et reçues au format JSON, assurant ainsi une communication fluide entre le client et le serveur.

2.7.1.2 Server-Side (Node.js & Express)

- **Router** : Détermine la manière dont les requêtes HTTP sont dirigées vers les contrôleurs appropriés. Il définit les routes de l'application et associe chaque route à un contrôleur spécifique.
- **Controller** : Contient la logique métier et traite les requêtes HTTP. Les contrôleurs reçoivent les requêtes, interagissent avec les services et renvoient les réponses appropriées au client.
- **Socket.io** [14] : Permet une communication en temps réel entre le serveur et le client, facilitant les fonctionnalités telles que les notifications instantanées et les mises à jour en direct
- **ORM (Sequelize)** [15] : Interface entre l'application et la base de données MySQL. Sequelize permet de gérer les opérations de base de données de manière abstraite et efficace, facilitant ainsi les interactions avec la base de données.
- **Nodemailer** [16] : Utilisé pour l'envoi d'emails depuis le serveur. Il permet d'automatiser les notifications par email et autres communications.
- **Base de Données (MySQL)** : accessible via Sequelize ORM. La base de données stocke toutes les données essentielles de l'application, telles que les informations utilisateur, les transactions, et les médias.
- **Intégrations Externes (Cloudinary)** [17] : Utilisé pour la gestion et le stockage des fichiers médias. Cloudinary offre des services de gestion d'images et de vidéos, y compris le stockage, la transformation, et la livraison optimisée.

Cette architecture logicielle, en combinant des technologies modernes et performantes, permet de développer une application web robuste et scalable. La séparation des responsabilités entre le client et le serveur assure une gestion efficace des données et une expérience utilisateur fluide.

2.7.1.3 Architecture physique

L'architecture physique de notre système prend en compte le contexte d'exécution et divise le système en plusieurs composants. Nous avons identifié quatre niveaux : **le tier client**, **le tier présentation**, **le tier applicatif** et **le tier données**. Tous ces composants sont déployés sur un VPS avec Docker pour garantir une gestion efficace et une isolation des services. Cette architecture est illustrée par la figure 2.9.

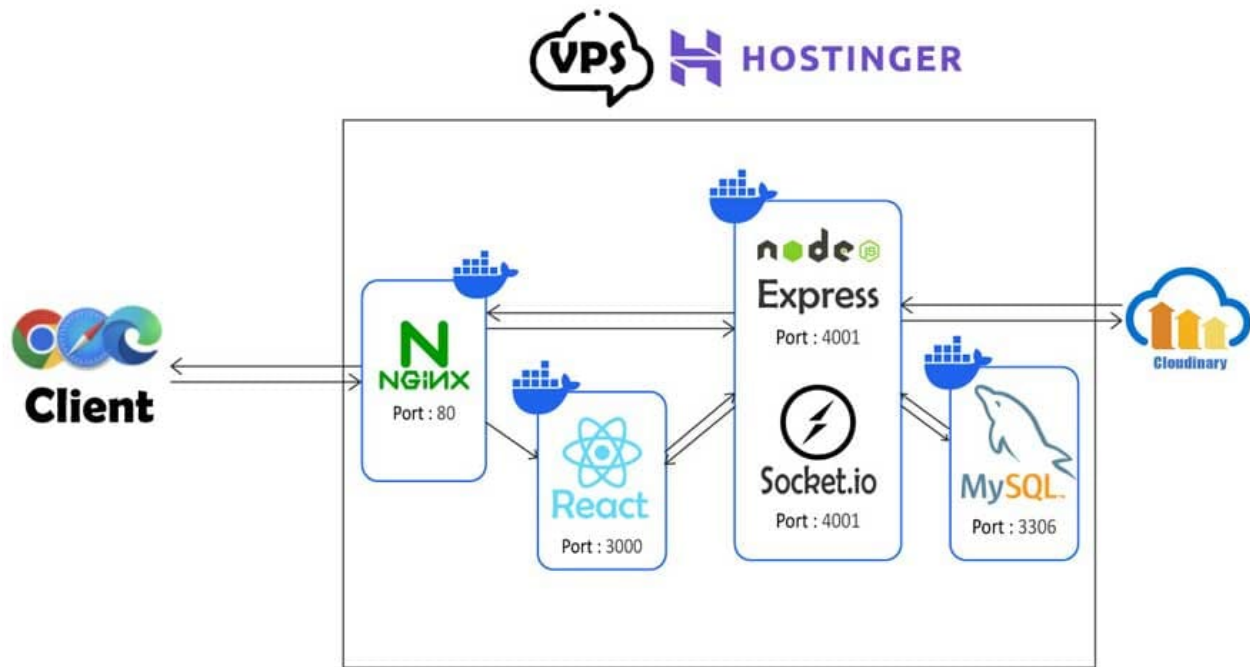


FIGURE 2.9 : Architecture physique

- **Tier client** : Nous avons à notre disposition plusieurs navigateurs web performants et sécurisés, notamment Google Chrome, Internet Explorer, Mozilla Firefox et Opera. Ces navigateurs offrent aux utilisateurs finaux la possibilité d'interagir avec l'application.
- **Tier présentation** : Ce niveau inclut un serveur Nginx opérant sur le port 80. Nginx fonctionne comme un serveur web, recevant les requêtes des clients et les redirigeant vers le serveur d'application pour traitement. Il gère également le contenu statique et les requêtes de chargement initial des applications React. Nginx est déployé dans un conteneur Docker, assurant une configuration et une gestion uniformes.
- **Tier applicatif** : Ce niveau comprend l'application React, fonctionnant sur le port 3000, et le serveur Node.js avec Express et Socket.io, fonctionnant sur le port 4001. React gère l'interface utilisateur et les interactions côté client, tandis qu'Express traite les requêtes HTTP et Socket.io facilite la communication en temps réel entre le client et le serveur. L'application React et le serveur Node.js/Express/Socket.io sont chacun déployés dans des conteneurs Docker distincts, assurant une isolation et une portabilité optimales des environnements.
- **Tier données (MySQL)** : Le serveur de base de données MySQL, fonctionnant sur le port 3306, permet l'accès aux données stockées. Performant et multi-threadé, MySQL supporte plusieurs utilisateurs simultanément. Il est déployé dans un conteneur Docker pour une gestion simplifiée et une isolation des données. Ce niveau inclut également l'intégration avec Cloudinary pour la gestion des médias.

2.8 Environnement de travail

Pour la mise en œuvre de notre projet, nous avons utilisé des équipements matériels et des logiciels spécifiques, détaillés dans les sections suivantes :

2.8.1 Environnement de développement matériel

Pour ce faire, nous avons utilisé un ordinateur portable et un vps (hostinger vps) avec les caractéristiques suivantes, comme le montre le tableau 2.2.

Périphérique	PC Portable Dell G3	VPS Hosting KVM 2
Système d'exploitation	windows 11	Ubuntu 22.04
Processeur	Intel Core I7-8750H	2 vCPU Core
RAM	8 Go	8 GO
Disque	1 To	100 GB

TABLEAU 2.2 : Environnement de développement matériel

2.8.2 Environnement de développement logiciel

Pour garantir un développement, une gestion et un déploiement efficaces de notre application, nous avons établi un environnement de développement logiciel intégrant divers outils et services. Ces outils sont essentiels pour les différentes phases du cycle de vie du développement logiciel, incluant l'écriture du code, les tests, la mise en production et la surveillance.

Voici les outils logiciels utilisés dans notre application :

- **Visual Studio Code (VS Code)** [18] : Utilisé comme principal éditeur de code, VS Code offre une interface conviviale, des extensions riches et des fonctionnalités avancées telles que le débogage, la gestion de version intégrée et la prise en charge des langages modernes.
- **Postman** [19] : Utilisé pour tester les API. Postman permet de concevoir, tester et documenter les API de manière interactive, assurant ainsi que les services backend fonctionnent comme prévu.
- **XAMPP** [20] : Un environnement de développement PHP local qui intègre Apache, MySQL, et d'autres outils utiles pour tester des applications web localement.
- **Cloudinary Cloud** [13] : Utilisé pour la gestion des médias, Cloudinary facilite le stockage, la transformation et la livraison optimisée d'images et de vidéos.
- **GitHub** [21] : Utilisé pour le contrôle de version et la collaboration, GitHub héberge le code source, gère les branches, les pull requests, et permet de suivre les issues et les tâches.

- **Jenkins** [22] : Outil d'intégration continue qui automatise le processus de construction, de test et de déploiement de l'application, assurant une livraison continue et des mises à jour régulières.
- **Prometheus** [23] : Utilisé pour la surveillance et l'alerte. Prometheus collecte et stocke les métriques et fournit des alertes en temps réel.
- **Grafana** [24] : Utilisé en conjonction avec Prometheus pour la visualisation des métriques, Grafana offre des tableaux de bord interactifs et des graphiques détaillés.
- **SonarQube** [25] : Utilisé pour l'analyse de la qualité du code, SonarQube identifie les bugs, les vulnérabilités et les problèmes de code duplicatif, assurant un code propre et maintenable.
- **OWASP** [26] : Utilisé pour vérifier la sécurité des applications, les outils et les lignes directrices OWASP aident à identifier et à corriger les vulnérabilités de sécurité dans le code.
- **Nexus** [27] : Utilisé pour gérer les dépôts de paquets, Nexus stocke et gère les dépendances, assurant une gestion efficace des bibliothèques et des composants tiers.
- **ngrok** [28] : Utilisé pour créer des tunnels sécurisés vers les serveurs locaux, ngrok permet de tester et de partager localement des applications web sur Internet de manière sécurisée.
- **Overleaf** [29] : Utilisé pour rédiger la documentation du projet en LaTeX, Overleaf facilite la collaboration et la création de documents professionnels.
- **Draw.io** [30] : Utilisé pour créer des diagrammes et des schémas, Draw.io aide à visualiser l'architecture et les flux de travail de l'application.
- **Figma** [31] : Utilisé pour la conception d'interfaces utilisateur, Figma permet de créer des maquettes interactives et de collaborer en temps réel sur la conception UI/UX.
- **Nginx** [32] : Utilisé comme serveur web et reverse proxy, Nginx gère la distribution des requêtes et améliore les performances et la sécurité de l'application.

2.8.3 Frameworks et Technologies

Dans cette section, nous détaillons les bibliothèques, les frameworks et les technologies que nous avons employés pour développer l'application, tout en expliquant les raisons de nos choix :

- **React** [12] : Une bibliothèque JavaScript pour la construction d'interfaces utilisateur. React facilite la création de composants réutilisables et la gestion de l'état de l'application. La figure 2.10 montre le logo de React.

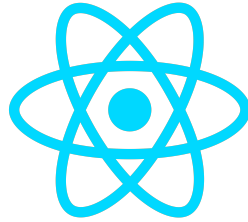


FIGURE 2.10 : Logo de React

- **TypeScript (TS)** [33] : Un sur-ensemble typé de JavaScript qui permet de détecter les erreurs à la compilation, améliorant ainsi la qualité du code et la productivité du développement. La figure 2.11 montre le logo de TypeScript.



FIGURE 2.11 : Logo de TypeScript

- **Redux** [34] : Une bibliothèque de gestion de l'état prédictive pour JavaScript. Redux centralise l'état de l'application et permet de gérer les états complexes de manière efficace. La figure 2.12 montre le logo de Redux.



FIGURE 2.12 : Logo de Redux

- **React Router DOM** [35] : Une bibliothèque pour la gestion des routes dans les applications React. Elle permet de définir des routes et de naviguer entre les vues de manière dynamique. La figure 2.13 montre le logo de React Router DOM.



FIGURE 2.13 : Logo de React Router DOM

- **Axios** [36] : Une bibliothèque pour effectuer des requêtes HTTP depuis le navigateur. Axios simplifie les appels API et la gestion des promesses. La figure 2.14 montre le logo de Axios.



FIGURE 2.14 : Logo de Axios

- **Node.js** [37] : Un environnement d'exécution JavaScript côté serveur qui permet de construire des applications réseau rapides et évolutives. La figure 2.15 montre le logo de Node.js.



FIGURE 2.15 : Logo de Node.js

- **Express.js** [38] : Un framework web minimaliste pour Node.js qui facilite la création de serveurs web et d'API. Express gère les requêtes HTTP et la logique de routage. La figure 2.16 montre le logo de Express.js.



FIGURE 2.16 : Logo de Express.js

- **Sequelize** [39] : Un ORM (Object-Relational Mapping) pour Node.js qui facilite l'interaction avec les bases de données SQL. Sequelize supporte les transactions, les migrations et les modèles. La figure 2.17 montre le logo de Sequelize.



FIGURE 2.17 : Logo de Sequelize

- **SQL** [40] : Un langage de programmation utilisé pour gérer et manipuler les bases de données relationnelles. SQL est utilisé en conjonction avec Sequelize pour accéder aux données stockées. La figure 2.18 montre le logo de SQL.



FIGURE 2.18 : Logo de SQL

- **Nodemailer** [41] : Un module pour Node.js permettant l'envoi d'emails. Nodemailer supporte les protocoles SMTP et assure une communication par email fiable et sécurisée. La figure 2.19 montre le logo de Nodemailer.



FIGURE 2.19 : Logo de Nodemailer

- **Socket.io** [42] : Une bibliothèque JavaScript qui permet la communication bidirectionnelle en temps réel entre les clients et le serveur. Socket.io est utilisé pour implémenter les fonctionnalités en temps réel comme les notifications et les chats. La figure 2.20 montre le logo de Socket.io.



FIGURE 2.20 : Logo de Socket.io

- **Yarn** [43] : Un gestionnaire de paquets rapide et sécurisé pour JavaScript. Yarn améliore la rapidité des installations de dépendances et assure la cohérence des versions. La figure 2.21 montre le logo de Yarn.



FIGURE 2.21 : Logo de Yarn

- **NPM** [44] : Le gestionnaire de paquets officiel pour Node.js. npm permet d'installer, de partager et de gérer les dépendances et les packages. La figure 2.22 montre le logo de NPM.



FIGURE 2.22 : Logo de NPM

- **Vite** [45] : Un outil de construction et de développement rapide pour les projets JavaScript modernes. Vite offre une expérience de développement optimisée avec un démarrage rapide du serveur de développement et une reconstruction rapide des modules. La figure 2.23 montre le logo de Vite.



FIGURE 2.23 : Logo de Vite

- **Git** [46] : Un système de contrôle de version distribué qui permet de suivre les modifications du code source et de collaborer efficacement avec les autres développeurs. La figure 2.24 montre le logo de Git.



FIGURE 2.24 : Logo de Git

- **Docker** [47] : Une plateforme qui permet de créer, déployer et exécuter des applications dans des conteneurs. Docker assure la portabilité et l'isolation des applications, facilitant ainsi le développement et le déploiement. La figure 2.25 montre le logo de Docker.

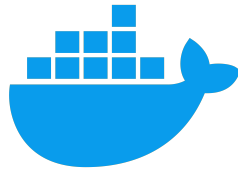


FIGURE 2.25 : Logo de Docker

- **Jest** [48] : Un framework de test JavaScript qui permet d’écrire des tests unitaires et d’intégration. Jest est utilisé pour assurer la qualité et la fiabilité du code en détectant les régressions et les erreurs. La figure 2.26 montre le logo de Jest.



FIGURE 2.26 : Logo de Jest

2.9 Conclusion

La phase de spécification des exigences est une étape cruciale du cycle de vie du projet, car elle permet de mieux comprendre le système et ses fonctionnalités clés. Nous avons ainsi introduit le diagramme de cas d’utilisation, le diagramme de classes et la charte graphique. Par la suite, nous nous sommes concentrés sur le backlog du produit, la planification des versions, la conception architecturale et les choix technologiques retenus. Le prochain chapitre sera consacré à notre premier release et à son développement.

RELEASE 1 : SPRINT 1/2 :

AUTHENTIFICATION ET GESTION DES ORGANISATIONS, DE CONTENU ET DES RAPPORTS POWER BI

Plan

3.1	Introduction	35
3.2	Sprint 1 : Authentification et gestion des organisations	35
3.3	Réalisation Sprint 1	42
3.4	Sprint 2 : Gestion des dossiers et téléverser des pièces jointes et intégrer des rapports Power BI	45
3.5	Réalisation Sprint 2	50
3.6	Conclusion	52

3.1 Introduction

Dans ce chapitre, nous détaillons la mise en œuvre des deux premiers sprints : « Authentification et gestion des organisations » et « Gestion des dossiers et téléverser des pièces jointes et intégrer des rapports Power BI ». Pour chaque sprint, nous présentons d’abord le backlog produit, puis nous examinons une partie dédiée à l’analyse détaillée avec l’exposition des diagrammes, suivie d’une autre partie consacrée à la conception du sprint. Enfin, nous abordons la phase de réalisation en présentant les interfaces web développées.

3.2 Sprint 1 : Authentification et gestion des organisations

Dans cette section, nous présentons les étapes de mise en œuvre du sprint « Authentification et gestion des organisations ».

3.2.1 Objectifs du Sprint 1

L’objectif du premier sprint est de développer l’authentification et la gestion des organisations, permettant ainsi aux super administrateurs de s’identifier et de gérer la création ainsi que la suppression des organisations. De plus, ce sprint inclut la fonctionnalité d’inviter les administrateurs des organisations à compléter leur inscription via l’e-mail. En outre, il offre la possibilité aux super administrateurs et aux administrateurs d’inviter des utilisateurs à rejoindre ces organisations.

3.2.2 Backlog du sprint 1

Le tableau 3.1 illustre le backlog du sprint « l’authentification et la gestion des organisations ».

Fonctionnalités	Priorité	Estimation (jours)
✓ Authentification sur la plateforme	1	7
✓ Créer une nouvelle organisation affiliée	1	8
✓ Inviter un utilisateur à devenir administrateur d’une organisation affiliée	1	5
✓ Inviter un utilisateur à devenir membre d’une organisation affiliée	1	4
✓ Recevoir une invitation et compléter l’inscription comme administrateur ou membre	1	6

TABLEAU 3.1 : Backlog du sprint 1

3.2.3 Analyse fonctionnelle du sprint 1

3.2.3.1 Diagramme des cas d'utilisation

La figure 3.1 représente le diagramme de cas d'utilisation du sprint « l'authentification et la gestion des organisations ».

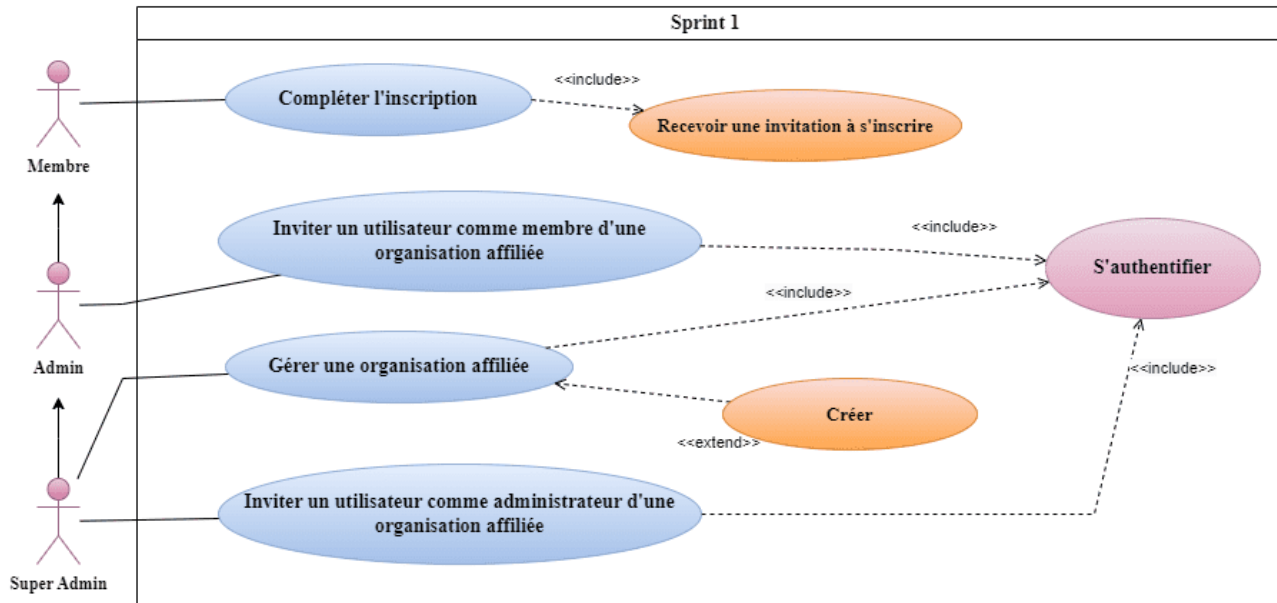


FIGURE 3.1 : Diagramme de cas d'utilisation pour l'authentification et la gestion des organisations

3.2.3.2 Description textuelle du cas d'utilisation pour le sprint 1

En fournissant une description textuelle d'un cas d'utilisation, nous clarifions le fonctionnement détaillé de la fonctionnalité et spécifions la séquence d'actions effectuées par l'acteur. Les tableaux 3.2 , 3.3 et 3.4 illustrent respectivement cette description des cas d'utilisation «Création des organisations» , «Authentification» et «Inscription de l'utilisateur».

Création des organisations :

Le système assure une vérification des autorisations avant de permettre l'accès au formulaire de création, garantissant ainsi que seuls les Super Administrateurs peuvent procéder. En cas de problème technique durant le processus, des messages d'erreur sont affichés avec des instructions pour corriger les éventuelles erreurs. Après la création réussie de l'organisation, un récapitulatif des informations saisies est présenté pour permettre au Super Administrateur de vérifier et confirmer les détails avant la finalisation.

Titre	Création d'une organisation
Acteur	Super Administrateur
Pré-condition	L'utilisateur doit être authentifié avec un rôle de Super Administrateur.
Scénario nominal	01 : Le Super Administrateur accède à l'interface des organisations. 02 : Le Super Administrateur remplit le formulaire pour créer une nouvelle organisation. 03 : Le système vérifie que les champs ne sont pas vides. 04 : Le Super Administrateur soumet la demande de vérification du nom de l'organisation. 05 : Le système vérifie que le nom de l'organisation n'existe pas déjà dans la base de données. 06 : Le système retourne le résultat de la vérification. 07 : Si le nom est valide, le Super Administrateur soumet la demande de création de l'organisation. 08 : Le système crée l'organisation et confirme sa création. 09 : Le système demande la création du groupe de discussion associé. 10 : Le système confirme la création du groupe de discussion. 11 : Le système demande la création du dossier partagé associé. 12 : Le système confirme la création du dossier partagé. 13 : Le système demande l'invitation de l'administrateur de la nouvelle organisation. 14 : Le système confirme l'envoi de l'invitation. 15 : Le système affiche l'interface de confirmation d'ajout réussi.
Post-condition	L'organisation est créée dans le système avec le groupe de discussion, le dossier partagé et l'administrateur invité.
Scénario alternatif	A1 : Si des champs obligatoires sont vides, le système affiche un message d'erreur demandant de remplir les champs. A2 : Si le nom de l'organisation existe déjà, le système affiche un message d'erreur. A3 : Le Super Administrateur remplit les champs manquants ou modifie le nom de l'organisation et soumet à nouveau le formulaire. A4 : Le scénario principal reprend au point 04.

TABLEAU 3.2 : Description textuelle du cas d'utilisation «Création d'une organisation»

Authentification :

Le système vérifie que l'utilisateur dispose d'un compte valide avec les informations nécessaires avant de permettre l'accès à la page d'authentification. Si les informations d'identification sont incorrectes ou manquantes, des messages d'erreur sont affichés pour corriger les erreurs et permettre une nouvelle tentative. Après une authentification réussie, l'utilisateur est dirigé vers la page d'accueil et a accès aux fonctionnalités de l'application en fonction de son rôle.

Titre	Authentification
Acteurs	Membre, Administrateur, Super Administrateur
Pré-condition	L'utilisateur doit avoir un compte existant avec un email et un mot de passe.
Scénario nominal	01 : L'utilisateur accède à la page d'authentification de l'application. 02 : L'utilisateur saisit son email et son mot de passe. 03 : Le système vérifie que les champs ne sont pas vides. 04 : L'utilisateur clique sur le bouton de connexion. 05 : Le système demande l'authentification en vérifiant les informations d'identification. 06 : Le système recherche l'utilisateur par email dans la base de données et retourne les informations de l'utilisateur. 07 : Si l'utilisateur est trouvé, le système vérifie le mot de passe. 08 : Si le mot de passe est correct, le système génère un token et retourne ce token. 09 : L'utilisateur est authentifié avec succès et redirigé vers la page d'accueil. 10 : L'utilisateur peut maintenant accéder aux fonctionnalités selon son rôle.
Post-condition	L'utilisateur est authentifié et a accès aux fonctionnalités de l'application.
Scénario alternatif	A1 : Si les champs sont vides, le système affiche une erreur demandant de remplir les champs. A2 : Si l'utilisateur n'est pas trouvé, le système affiche un message d'erreur indiquant que l'utilisateur n'existe pas. A3 : Si le mot de passe est incorrect, le système affiche un message d'erreur indiquant un mot de passe incorrect. A4 : L'utilisateur peut vérifier ses informations et tenter de se connecter à nouveau. Le scénario principal reprend au point 02.

TABLEAU 3.3 : Description textuelle du cas d'utilisation «Authentification»

Inscription de l'utilisateur :

Le système vérifie la présence de l'email d'invitation avant de permettre à l'utilisateur d'accéder à l'interface d'inscription. En cas d'erreurs dans le formulaire, des messages d'erreur sont affichés pour indiquer les champs manquants ou incorrects, et l'utilisateur est invité à corriger les informations. Une fois l'inscription réussie, l'utilisateur est redirigé vers la page d'accueil et reçoit un token pour une utilisation ultérieure.

Titre	Inscription de l'utilisateur
Acteur	Utilisateur
Pré-condition	L'utilisateur doit avoir reçu un email d'invitation.
Scénario nominal	01 : L'utilisateur reçoit l'email d'invitation. 02 : L'utilisateur accède à l'interface d'inscription. 03 : L'interface d'inscription est affichée. 04 : L'utilisateur remplit le formulaire d'inscription. 05 : Le système vérifie que les champs ne sont pas vides. 06 : Si les champs sont vides, une erreur est affichée. 07 : L'utilisateur complète le formulaire et soumet la demande d'inscription. 08 : Le système crée le nouvel utilisateur et le stocke dans la base de données. 09 : Le système génère un token pour l'utilisateur. 10 : Le système retourne le token à l'utilisateur. 11 : L'interface d'inscription redirige l'utilisateur vers la page d'accueil après une inscription réussie.
Post-condition	L'utilisateur est inscrit et redirigé vers la page d'accueil.
Scénario alternatif	A1 : Si les champs du formulaire sont vides, le système affiche un message d'erreur. 01 : L'utilisateur corrige les erreurs et tente de soumettre à nouveau le formulaire. 02 : Le scénario principal reprend au point 04.

TABLEAU 3.4 : Description textuelle du cas d'utilisation «Inscription de l'utilisateur»

3.2.4 Diagramme de classes

La figure 3.2 illustre le diagramme de classes du sprint 1 .

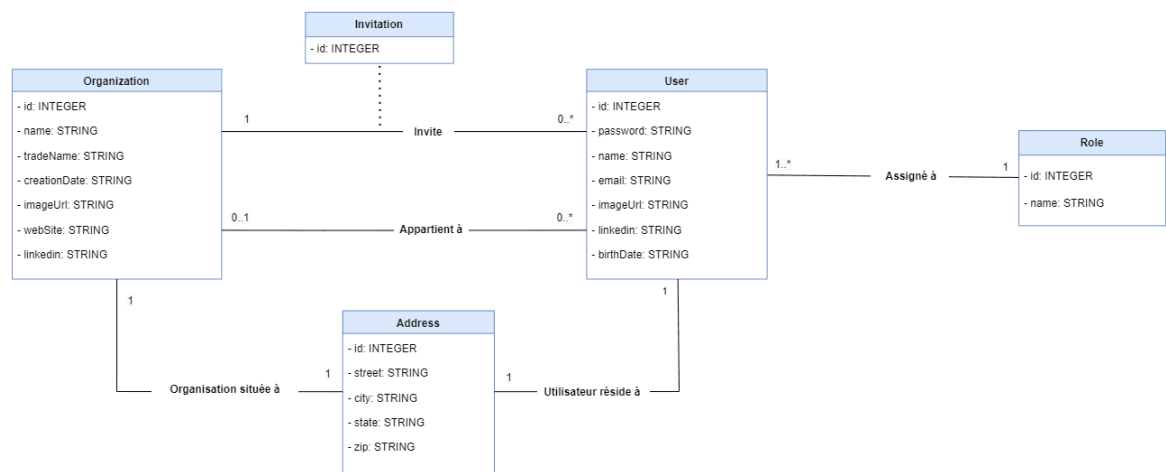


FIGURE 3.2 : Diagramme de classes de l’authentification et de la gestion des organisations

3.2.5 Diagrammes de séquence

Les diagrammes de séquence représentent graphiquement les interactions chronologiques entre les acteurs et le système. Cette section présente les diagrammes de séquence système pour les cas d’utilisation « Inscription », « Authentification » et « Création d’une organisation », illustrés respectivement par les figures 3.3, 3.4 et 3.5 .

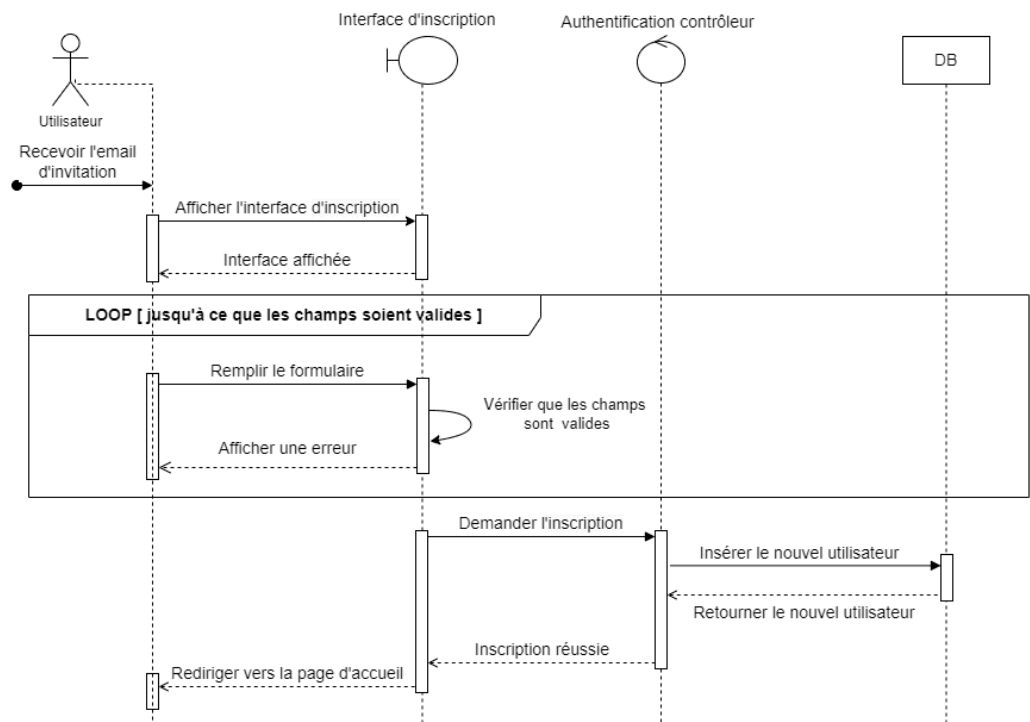


FIGURE 3.3 : Diagramme de séquence du scénario « Inscription de l’utilisateur »

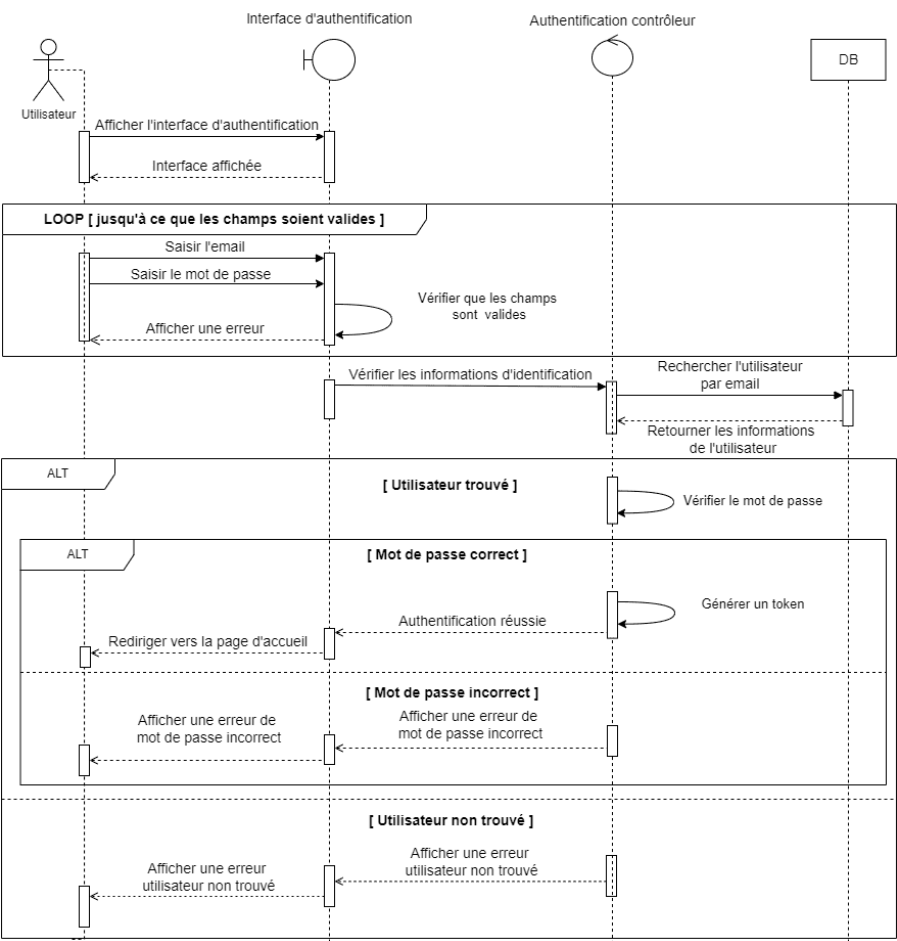


FIGURE 3.4 : Diagramme de séquence du scénario « Authentification »

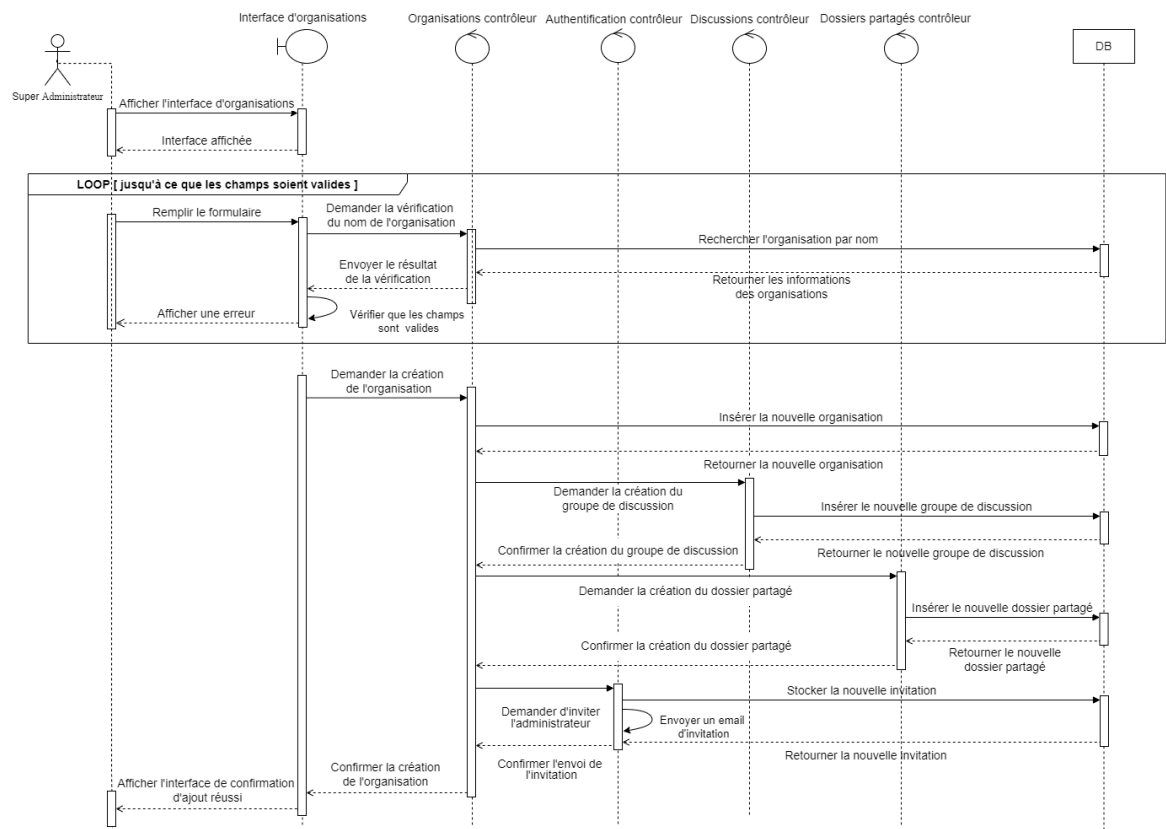


FIGURE 3.5 : Diagramme de séquence du scénario « Création d'une organisation »

3.3 Réalisation Sprint 1

Dans cette section, nous présentons les interfaces homme-machine illustrant les services proposés par notre tableau de bord. Nous avons réussi à structurer méthodiquement les différentes fonctionnalités pour fournir des interfaces simples, claires, conviviales, validées par le propriétaire du produit. Nous présentons ci-dessous des captures d'écran de quelques interfaces réalisées lors du sprint 1 de release 1.

La figure 3.6 représente l'interface d'authentification de notre tableau de bord.

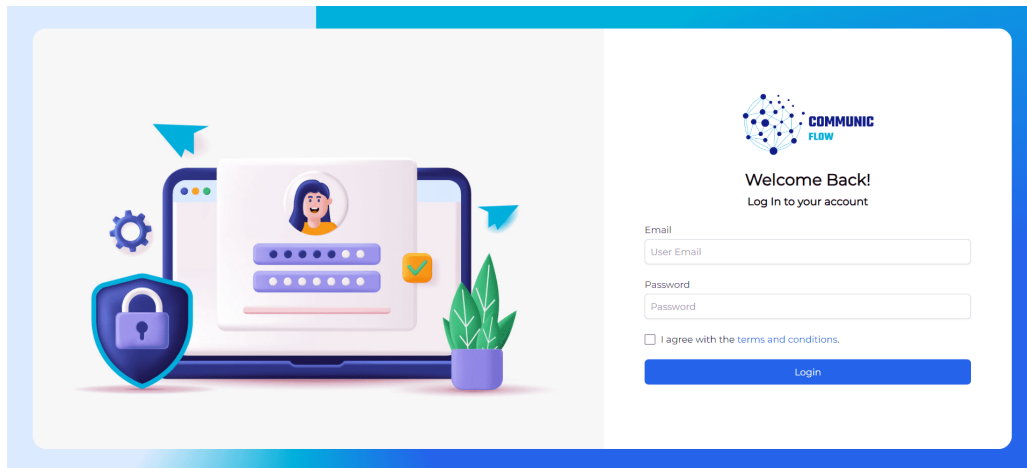


FIGURE 3.6 : Interface d'authentification

La figure 3.7 représente l'interface d'accueil de notre tableau de bord pour le super administrateur.

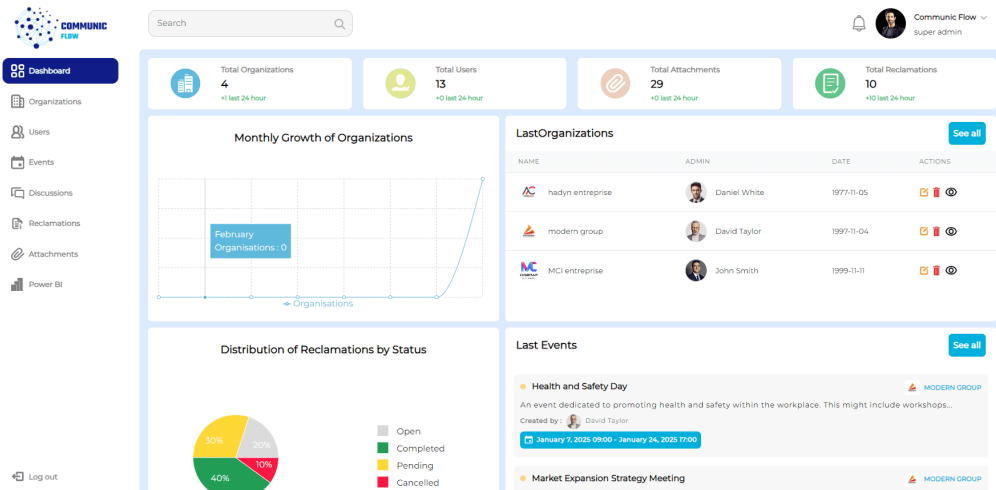


FIGURE 3.7 : Interface d'accueil pour le super administrateur

La figure 3.8 représente l'interface d'ajout à l'organisation lors de la première étape.

The screenshot shows the 'Add organization' form in the first step of the Communic Flow interface. The form is titled 'Add organization:' and features a progress bar with three steps: the first step is active (indicated by a blue circle), and the second and third steps are inactive (indicated by grey circles). The form contains the following fields:

- Organization name: example : xxx holding
- Trade name: example : xxx
- Date of creation: mm/dd/yyyy
- Address: example : Rue Habib borgiba
- State: Choisissez une État
- City: Choose a City
- Postal code: example : 5000
- Web site: example : www.asqil.tn
- LinkedIn account: example : /company/asqil/

There is an 'Upload' button for the organization logo and a 'Next' button at the bottom right.

FIGURE 3.8 : Interface d'ajout à l'organisation lors de la première étape

La figure 3.9 représente l'interface d'ajout à l'organisation lors de la deuxième étape.

The screenshot shows the 'Add organization' form in the second step of the Communic Flow interface. The progress bar now shows the first step as completed (green checkmark) and the second step as active (blue circle). The form contains the following fields:

- Email: exemple2@example.com
- Verify button
- Add button
- example@example.com admin

There is a 'Next' button at the bottom right.

FIGURE 3.9 : Interface d'ajout à l'organisation lors de la deuxième étape

La figure 3.10 représente l'interface d'ajout à l'organisation lors de la troisième étape.

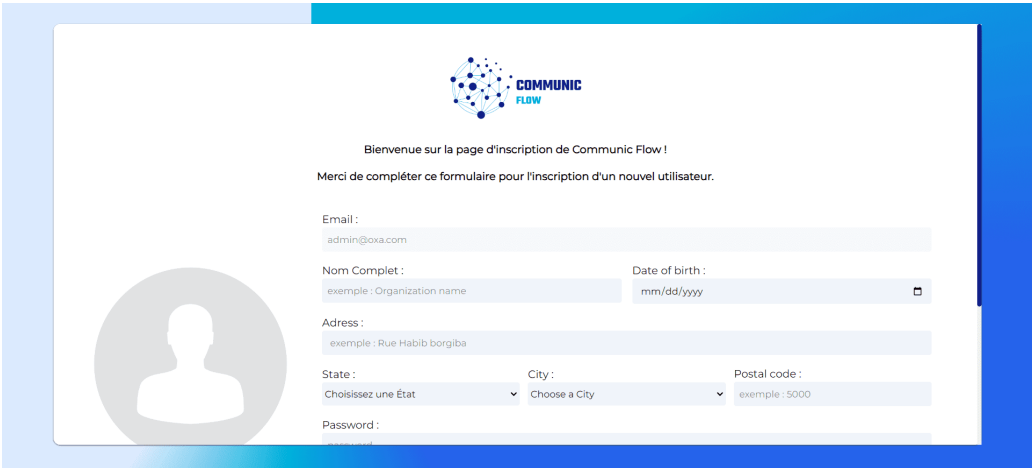
The screenshot shows the 'Add organization' form in the third step of the Communic Flow interface. The progress bar now shows the first two steps as completed (green checkmarks) and the third step as active (blue circle). The form displays the following information:

- Organization name : exemple of organization
- Creation date : 2024-08-04
- LinkedIn : https://www.linkedin.com/company/exemple/
- Web site : https://www.exemple.com
- adress : exemple of organization street
- State : Ariana
- City : Ariana
- postal code : 3000
- Admin's email invitation : exemple@example.com

There is a 'Confirm' button at the bottom right.

FIGURE 3.10 : Interface d'ajout à l'organisation lors de la troisième étape

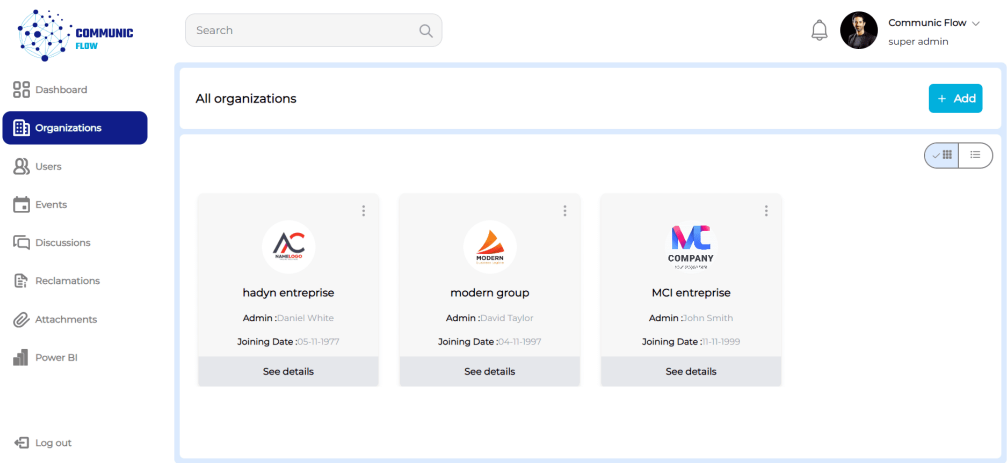
La figure 3.11 représente l'interface d'inscription envoyée par email.



The screenshot shows a registration form for 'Communic Flow'. At the top, there is a logo and a welcome message: 'Bienvenue sur la page d'inscription de Communic Flow !' followed by 'Merci de compléter ce formulaire pour l'inscription d'un nouvel utilisateur.' The form includes fields for Email (pre-filled with 'admin@oxa.com'), Nom Complet (with an example 'Organization name'), Date of birth (with a date picker), Address (with an example 'Rue Habib borgiba'), State (a dropdown menu), City (a dropdown menu), Postal code (with an example '5000'), and Password. There is a placeholder for a profile picture on the left.

FIGURE 3.11 : Interface de formulaire d'inscription

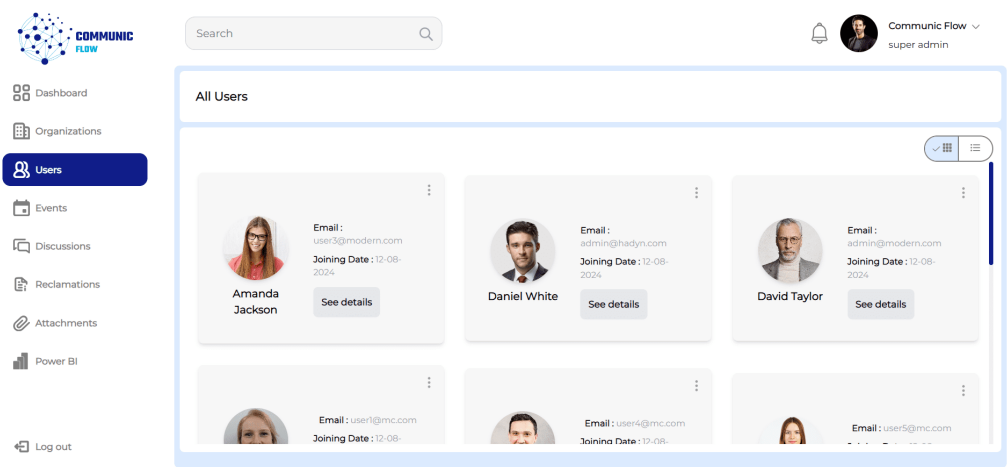
La figure 3.12 représente l'interface de la liste des organisations membres.



The screenshot shows the 'All organizations' page in the Communic Flow dashboard. The left sidebar contains navigation links: Dashboard, Organizations (selected), Users, Events, Discussions, Reclamations, Attachments, Power BI, and Log out. The main content area shows a list of three organizations: 'hadyn entreprise' (Admin: Daniel White, Joining Date: 05-11-1977), 'modern group' (Admin: David Taylor, Joining Date: 04-11-1997), and 'MCI entreprise' (Admin: John Smith, Joining Date: 01-11-1999). Each organization card has a 'See details' button. The top right of the dashboard shows a search bar, a notification bell, and the user profile 'Communic Flow super admin'.

FIGURE 3.12 : Interface de formulaire d'inscription

La figure 3.13 représente l'interface de la liste des utilisateurs.



The screenshot shows the 'All Users' page in the Communic Flow dashboard. The left sidebar is the same as in Figure 3.12, with 'Users' selected. The main content area shows a list of five users: 'Amanda Jackson' (Email: user3@modern.com, Joining Date: 12-08-2024), 'Daniel White' (Email: admin@hadyn.com, Joining Date: 12-08-2024), 'David Taylor' (Email: admin@modern.com, Joining Date: 12-08-2024), 'user1@mcc.com' (Email: user1@mcc.com, Joining Date: 12-08-2024), and 'user4@mcc.com' (Email: user4@mcc.com, Joining Date: 12-08-2024). Each user card has a 'See details' button. The top right of the dashboard is the same as in Figure 3.12.

FIGURE 3.13 : Interface de formulaire d'inscription

3.4 Sprint 2 : Gestion des dossiers et téléverser des pièces jointes et intégrer des rapports Power BI

Dans cette section, nous détaillons les étapes de mise en œuvre du sprint «Gestion des dossiers et téléversement des pièces jointes et intégration des rapports Power BI».

3.4.1 Objectifs du Sprint 2

L’objectif du deuxième sprint est de développer des fonctionnalités essentielles pour la gestion des dossiers, le téléversement des pièces jointes, et l’intégration des rapports Power BI. Ces fonctionnalités permettent aux super administrateurs de s’authentifier et de gérer efficacement la création, la modification, et la suppression de dossiers ainsi que de sous-dossiers, tout en facilitant le téléversement et la gestion des pièces jointes associées. L’intégration de Power BI dans le système permet également d’afficher des rapports dynamiques et interactifs, renforçant ainsi la capacité des utilisateurs à analyser et interpréter les données de manière visuelle. En somme, ce sprint vise à améliorer l’efficacité et la convivialité du système en offrant des outils robustes pour la gestion des informations et l’analyse des données.

3.4.2 Backlog du sprint 2

Le tableau 3.5 présente le contenu du backlog du sprint « Gestion des dossiers et téléverser des pièces jointes et intégrer des rapports Power BI ».

Fonctionnalités	Priorité	Estimation (jours)
✓ Créer des dossiers ou ajouter des sous-dossiers supplémentaires	1	6
✓ Téléverser ou télécharger des pièces jointes dans le dossier principal ou les sous-dossiers	1	6
✓ Renommer un dossier ou sous-dossier	3	2
✓ Supprimer un dossier ou sous-dossier	2	3
✓ Renommer une pièce jointe	3	3
✓ Supprimer une pièce jointe	2	3
✓ Intégrer des rapports Power BI	1	5

TABLEAU 3.5 : Backlog du sprint 2

3.4.3 Analyse fonctionnelle du sprint 2

3.4.3.1 Diagramme des cas d'utilisation

La figure 3.14 représente le diagramme de cas d'utilisation du sprint « Gestion des dossiers et téléverser des pièces jointes et intégrer des rapports Power BI ».

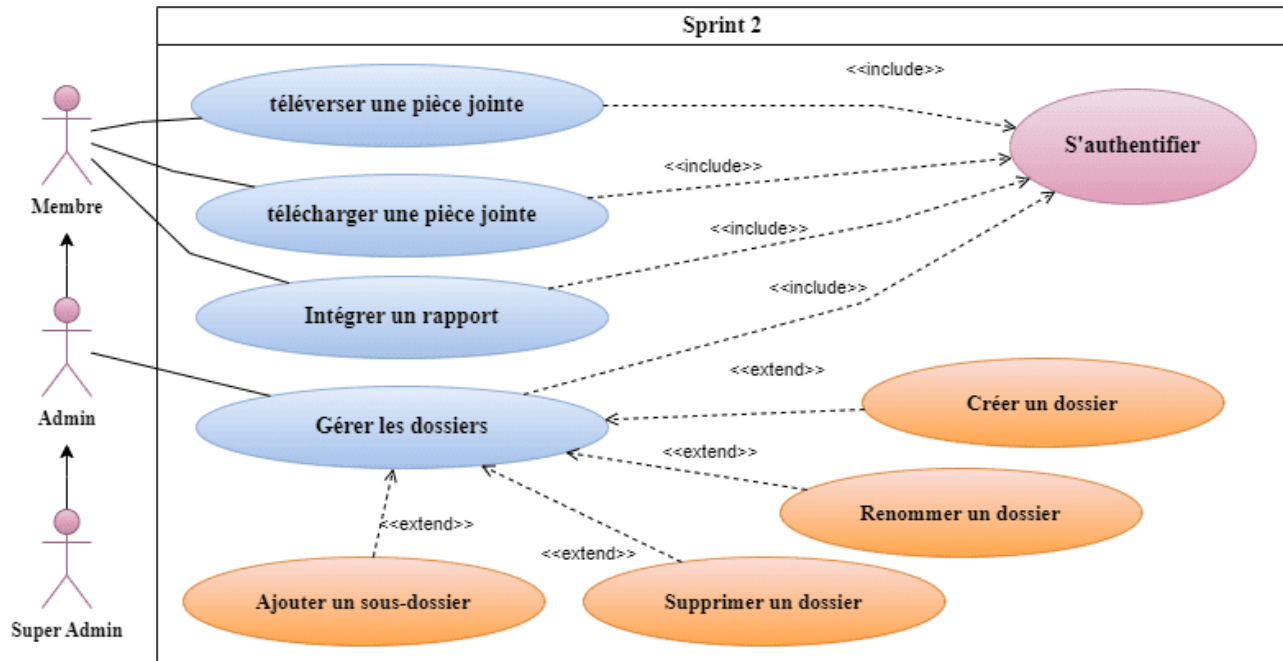


FIGURE 3.14 : Diagramme de cas d'utilisation pour la gestion des dossiers, le téléversement des pièces jointes et l'intégration des rapports Power BI

3.4.3.2 Description textuelle du cas d'utilisation pour la gestion des modèles

En fournissant une description textuelle d'un cas d'utilisation, nous clarifions le fonctionnement détaillé de la fonctionnalité et spécifions la séquence d'actions effectuées par l'acteur. Les tableaux 3.6 et 3.7 illustrent respectivement cette description des cas d'utilisation «Création des dossiers» et «Téléversement des pièces jointes».

Création des dossiers :

Le système vérifie que l'utilisateur est authentifié avant de lui permettre d'accéder à l'interface de création des dossiers. Si des champs obligatoires sont vides lors de la soumission du formulaire, des messages d'erreur sont affichés pour guider l'utilisateur dans la correction des informations. Après la création réussie du dossier, celui-ci est ajouté à la base de données et affiché à l'utilisateur pour confirmation.

Titre	Création des dossiers
Acteur	Utilisateur
Pré-condition	L'utilisateur doit être authentifié.
Scénario nominal	01 : L'utilisateur accède à l'interface de création de dossiers. 02 : L'utilisateur remplit le formulaire de création de dossier avec les informations requises. 03 : L'utilisateur soumet le formulaire. 04 : Le système vérifie que les champs ne sont pas vides. 05 : Le système crée le nouveau dossier et le stocke dans la base de données. 06 : Le système affiche le nouveau dossier à l'utilisateur.
Post-condition	Le nouveau dossier est créé et visible par l'utilisateur.
Scénario alternatif	A1 : Si des champs obligatoires sont vides, le système affiche un message d'erreur et encadre les champs en rouge. 01 : L'utilisateur remplit les champs manquants et soumet à nouveau le formulaire. 02 : Le scénario principal reprend au point 04.

TABLEAU 3.6 : Description textuelle du cas d'utilisation «Création des dossiers»

Téléversement des pièces jointes :

Le système permet à l'utilisateur authentifié de téléverser des pièces jointes en remplissant un formulaire dédié. Après vérification que tous les champs requis sont complets, la pièce jointe est téléversée vers le cloud. Le lien généré est ensuite stocké dans la base de données, et l'utilisateur voit la pièce jointe affichée à l'écran. Si des champs obligatoires sont manquants, des messages d'erreur apparaissent, invitant l'utilisateur à les compléter avant de soumettre à nouveau le formulaire.

Titre	Téléversement des pièces jointes
Acteur	Utilisateur
Pré-condition	L'utilisateur doit être authentifié.
Scénario nominal	01 : L'utilisateur accède à l'interface de téléversement d'une pièce jointe. 02 : L'utilisateur remplit le formulaire de téléversement avec les informations requises. 03 : L'utilisateur soumet le formulaire. 04 : Le système vérifie que les champs ne sont pas vides. 05 : Le système téléverse la pièce jointe vers le cloud. 06 : Le cloud renvoie l'url de la pièce jointe. 07 : Le système crée la nouvelle pièce jointe et la stocke dans la base de données. 08 : Le système affiche la nouvelle pièce jointe à l'utilisateur.
Post-condition	La nouvelle pièce jointe est téléversée et visible par l'utilisateur.
Scénario alternatif	A1 : Si des champs obligatoires sont vides, le système affiche un message d'erreur et encadre les champs en rouge. 01 : L'utilisateur remplit les champs manquants et soumet à nouveau le formulaire. 02 : Le scénario principal reprend au point 04.

TABLEAU 3.7 : Description textuelle du cas d'utilisation «Téléversement des pièces jointes»

3.4.4 Diagramme de classes

La figure 3.15 illustre le diagramme de classes du sprint 2.

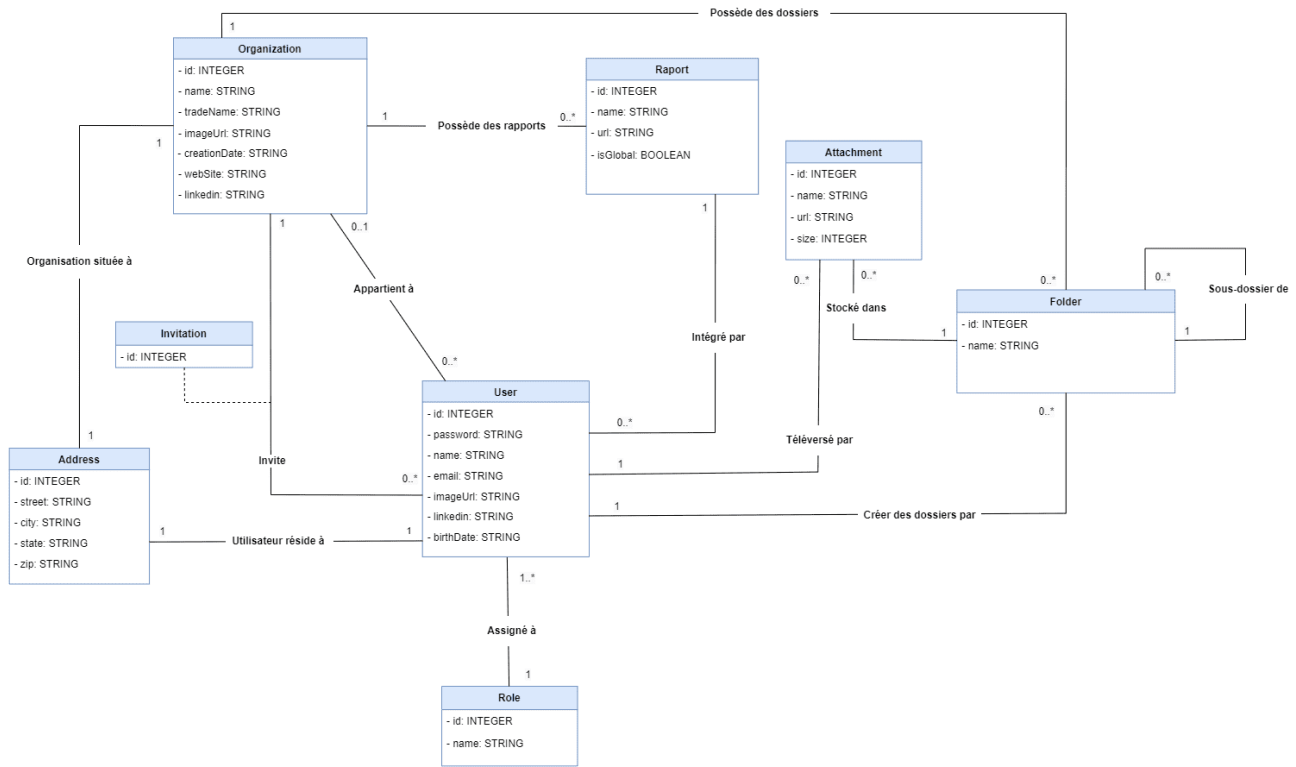


FIGURE 3.15 : Diagramme de classes du sprint 2

3.4.5 Diagrammes de séquence

Dans cette partie, nous présentons les diagrammes de séquence système des cas d'utilisation « Création des dossiers » et « Téléversement des pièces jointes », illustrés respectivement par les figures 3.16 et 3.17.

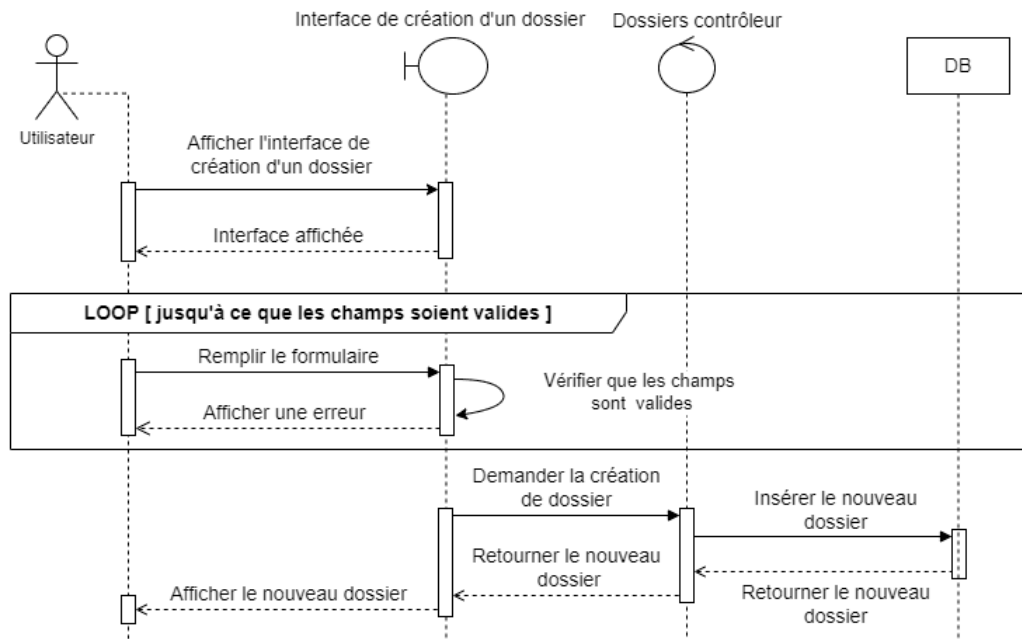


FIGURE 3.16 : Diagramme de séquence du scénario « Création des dossiers »

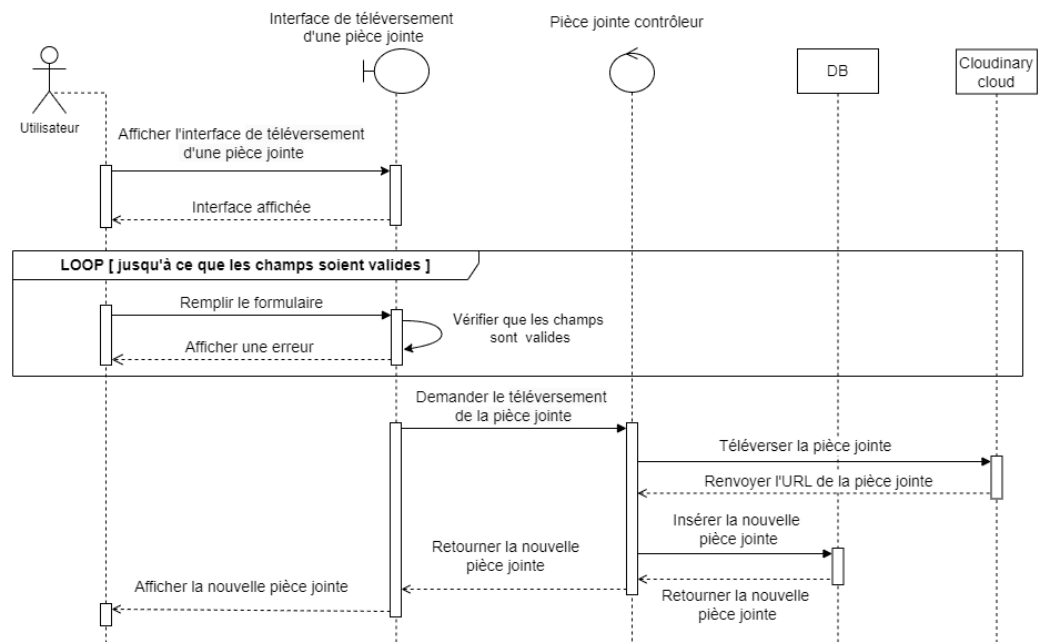


FIGURE 3.17 : Diagramme de séquence du scénario « Téléversement des pièces jointes »

3.5 Réalisation Sprint 2

Nous présentons ci-dessous des captures d'écran de quelques interfaces réalisées lors du sprint 2 de la release 1.

La figure 3.18 représente l'interface des listes de dossiers et pièces jointes pour tous les utilisateurs en format grid.

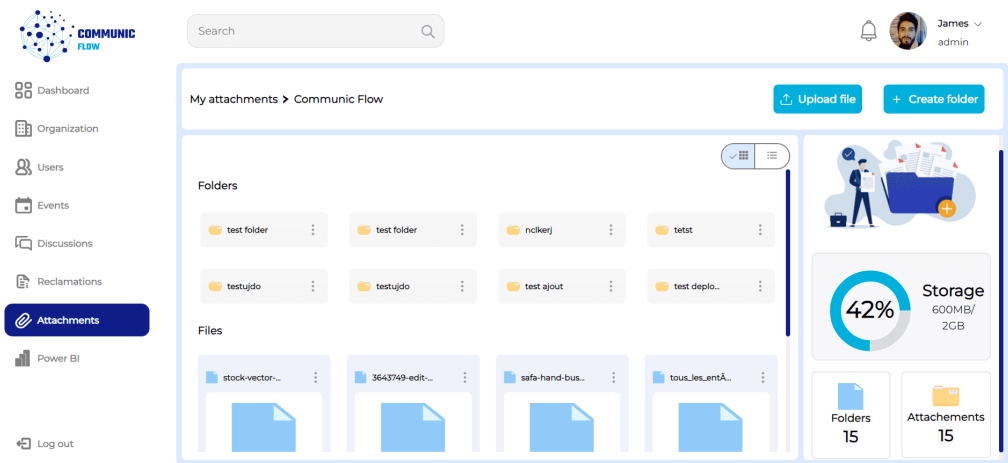


FIGURE 3.18 : Interface des listes de dossiers et pièces jointes en format grid

La figure 3.19 représente l'interface des listes de dossiers et pièces jointes pour tous les utilisateurs en format tableau.

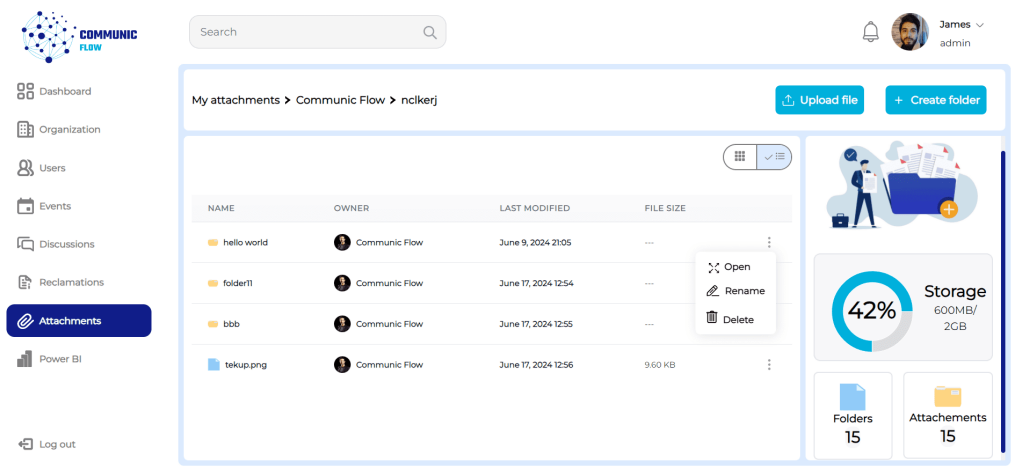


FIGURE 3.19 : Interface des listes de dossiers et pièces jointes en format tableau

La figure 3.20 représente l'interface de liste des rapports BI.

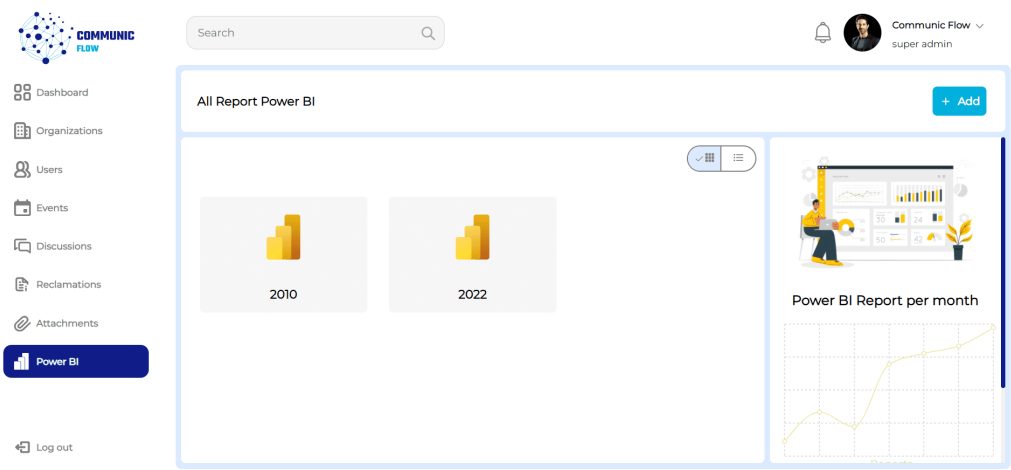


FIGURE 3.20 : Interface de liste des rapports BI

La figure 3.21 représente l'interface de téléversement d'une pièce jointe.

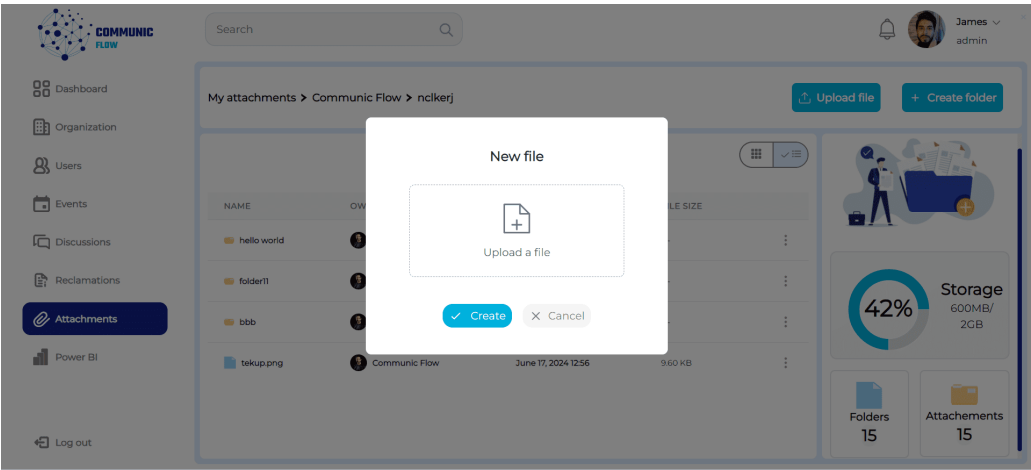


FIGURE 3.21 : Interface de téléversement d'une pièce jointe

La figure 3.22 représente l'interface de suppression d'un dossier.

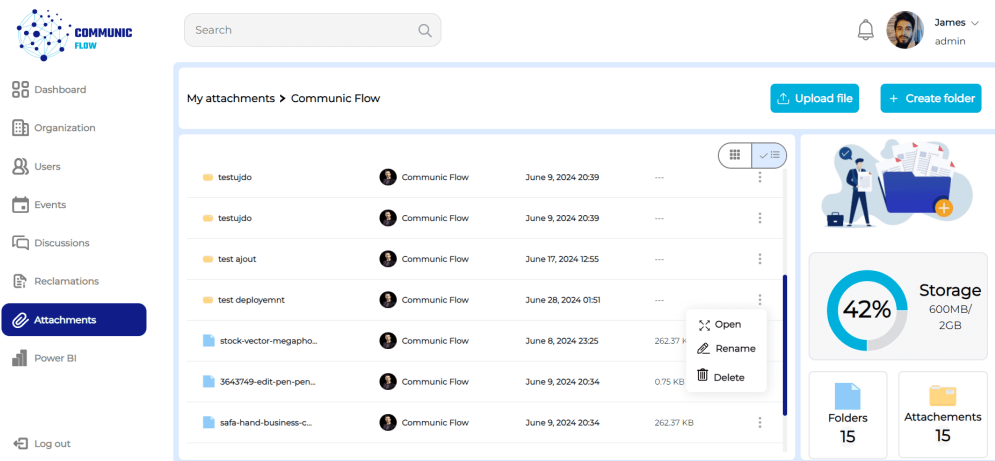


FIGURE 3.22 : Interface de suppression d'un dossier

La figure 3.23 représente l'interface d'ajout d'un dossier.

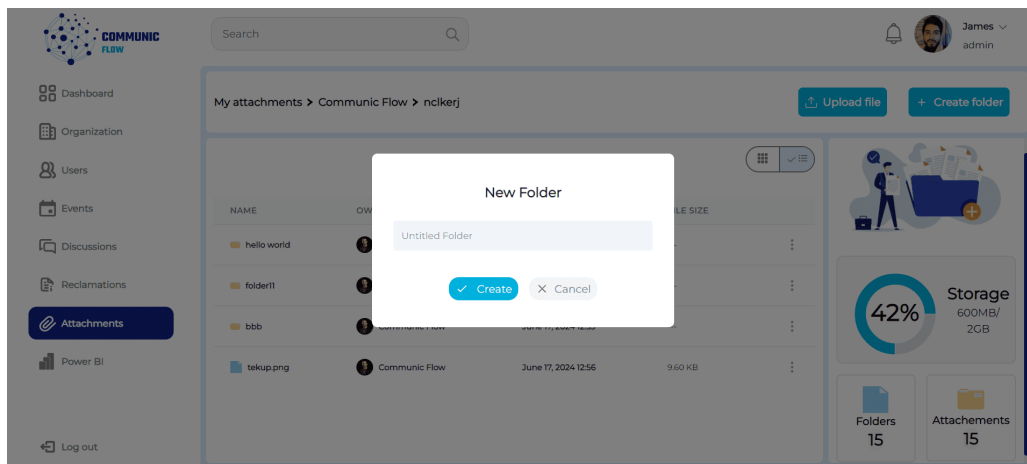


FIGURE 3.23 : Interface d'ajout d'un dossier

3.6 Conclusion

Dans ce chapitre, nous avons examiné en détail et expliqué la première release du projet. Tout d'abord, nous avons présenté la phase d'analyse en utilisant des diagrammes de cas d'utilisation et des descriptions textuelles pour illustrer le fonctionnement du système. Ensuite, nous avons abordé la phase de conception en fournissant le diagramme de classes de conception, qui décrit la structure et les relations entre les différentes entités du système, ainsi que les diagrammes de séquence de quelques cas d'utilisation. Enfin, nous avons conclu ce sprint en mettant en évidence les interfaces graphiques qui représenteront l'apparence visuelle du système final. À ce stade, nous sommes prêts à passer au chapitre suivant, où nous expliquerons en détail la deuxième release du projet.

RELEASE 2 : SPRINT 3/4 : GESTION DES RÉCLAMATIONS, PLANIFICATION DES ÉVÉNEMENTS ET MESSAGERIE INSTANTANÉE

Plan

4.1	Introduction	54
4.2	Sprint 3 :Gestion des réclamations et Planification des événements	54
4.3	Réalisation Sprint 3	60
4.4	Sprint 4 : Gestion des Discussions en Temps Réel	62
4.5	Réalisation Sprint 4	67
4.6	Conclusion	68

4.1 Introduction

Au cours de ce chapitre, nous abordons la deuxième release : « Gestion des réclamations et Planification des événements » ainsi que « Messagerie instantanée ». Tout d'abord, nous expliquons la planification de ce sprint. Ensuite, nous consacrons la première partie à l'analyse, en exposant les diagrammes de cas d'utilisation et les descriptions textuelles, suivie d'une autre partie dédiée à la conception du sprint. Enfin, nous passons à la mise en œuvre de ce sprint.

4.2 Sprint 3 : Gestion des réclamations et Planification des événements

Dans cette section, nous décrivons en détail les étapes de mise en œuvre du sprint « Gestion des réclamations et planification des événements ».

4.2.1 Objectifs du Sprint 3

L'objectif du troisième sprint est de développer la gestion des réclamations et la planification des événements, permettant ainsi aux administrateurs de créer et de supprimer des réclamations, tandis que les super administrateurs peuvent répondre à ces réclamations. De plus, ce sprint inclut la fonctionnalité de planifier des événements pour les super administrateurs et les administrateurs.

4.2.2 Backlog du sprint 3

Dans cette section, nous présentons le backlog du sprint, une liste priorisée des tâches et des fonctionnalités à développer. Ce backlog nous guide en définissant les objectifs et les priorités pour assurer une livraison efficace et alignée sur les attentes des utilisateurs. Le tableau 4.1 présente le contenu du backlog du sprint 3.

Fonctionnalités	Priorité	Estimation (jours)
✓ Envoyer des réclamations	1	4
✓ Répondre aux réclamations envoyées	1	5
✓ Consulter les réclamations d’une organisation affiliée	1	3
✓ Planifier des événements	1	5
✓ Mettre à jour un événement	2	4
✓ Annuler un événement	1	3
✓ Consulter les événements globaux	1	3
✓ Consulter les événements des organisations affiliées	1	3

TABLEAU 4.1 : Backlog du sprint 3

4.2.3 Analyse fonctionnelle du sprint 3

4.2.3.1 Diagramme des cas d'utilisation

La figure 4.1 représente le diagramme de cas d'utilisation du sprint « la gestion des réclamations et la planification des événements ».

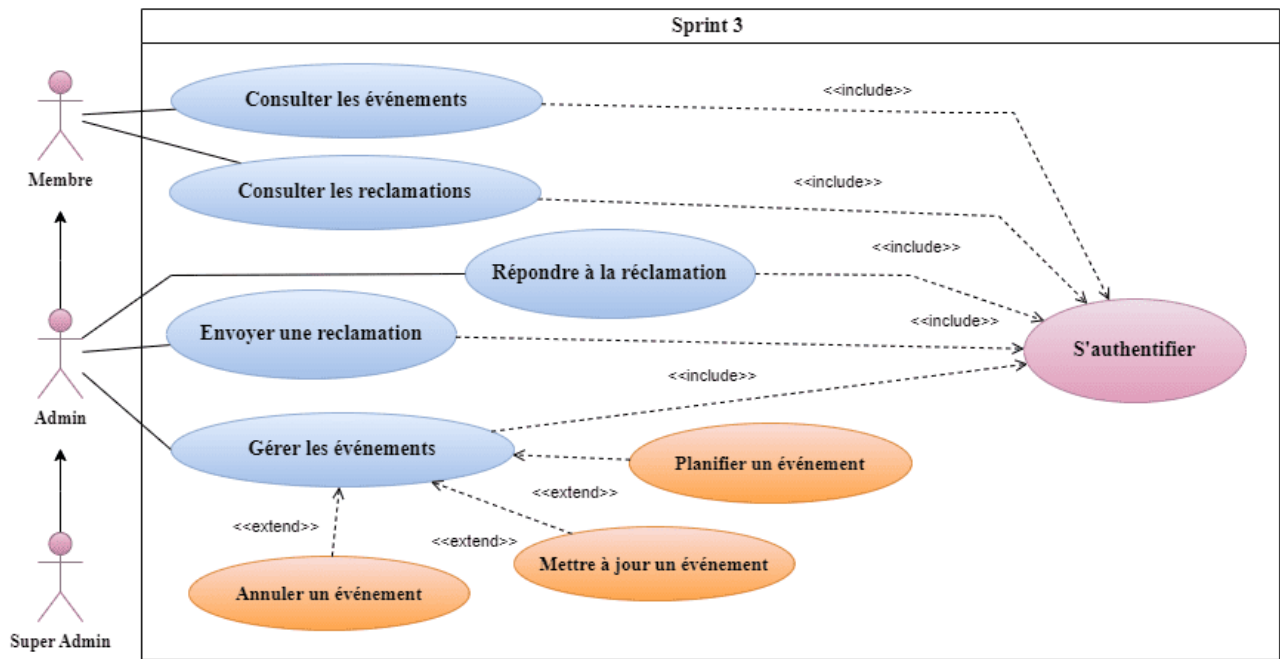


FIGURE 4.1 : Diagramme de cas d'utilisation pour la gestion des réclamations et la planification des événements

4.2.3.2 Description textuelle du cas d'utilisation pour le sprint 3

En fournissant une description textuelle d'un cas d'utilisation, nous clarifions le fonctionnement détaillé de la fonctionnalité et spécifions la séquence d'actions effectuées par l'acteur. Les tableaux 4.2 et 4.3 illustrent respectivement la description des cas d'utilisation « Création des réclamations » et « Répondre aux réclamations ».

Création des réclamations :

Le système permet à un administrateur authentifié de créer et soumettre des réclamations via un formulaire dédié. Après validation des champs, la réclamation est ajoutée à la base de données et devient visible pour le super administrateur. En cas de champs manquants, un message d'erreur s'affiche, demandant de compléter les informations.

Titre	Création des réclamations
Acteur	Administrateur, Super Administrateur
Pré-condition	L'administrateur doit être authentifié.
Scénario nominal	01 : L'administrateur accède à l'interface d'envoi de réclamation. 02 : Le système affiche l'interface. 03 : L'administrateur remplit le formulaire de réclamation. 04 : L'administrateur soumet le formulaire. 05 : Le système vérifie que les champs ne sont pas vides. 06 : Le système crée la nouvelle réclamation. 07 : Le système insère la nouvelle réclamation dans la base de données. 08 : Le système retourne la nouvelle réclamation. 09 : L'interface redirige l'administrateur vers la page d'affichage des réclamations. 10 : Le système affiche la nouvelle réclamation au super administrateur.
Post-condition	La nouvelle réclamation est créée et visible par le super administrateur.
Scénario alternatif	A1 : Si les champs obligatoires sont vides, le système affiche un message d'erreur. 01 : L'administrateur remplit les champs manquants et soumet à nouveau le formulaire. 02 : Le scénario principal reprend au point 05.

TABLEAU 4.2 : Description textuelle du cas d'utilisation « Création des réclamations »

Répondre aux réclamations :

Le système permet aux administrateurs et super administrateurs authentifiés de répondre aux réclamations via une interface dédiée. Le super administrateur peut créer une réponse initiale, qui est alors visible par l'administrateur. Celui-ci peut choisir de répondre à son tour ou de fermer la réclamation. Si une réponse est créée, elle est enregistrée et affichée. Si l'administrateur décide de fermer la réclamation, le système met à jour son état en conséquence.

Titre	Répondre aux réclamations
Acteur	Administrateur, Super Administrateur
Pré-condition	Les acteurs doivent être authentifiés.
Scénario nominal	01 : L'administrateur accède à l'interface d'affichage des réclamations. 02 : Le Super Administrateur accède à l'interface d'affichage des réclamations. 03 : Le système affiche l'interface. 04 : Le super administrateur crée une réponse à la réclamation. 05 : Le système enregistre et retourne la nouvelle réponse. 06 : L'administrateur voit la nouvelle réponse. 07 : L'administrateur décide soit de répondre soit de fermer la réclamation. 08 : Pour répondre, l'administrateur crée et envoie une nouvelle réponse. 09 : Le système enregistre et retourne la nouvelle réponse. 10 : L'interface affiche la nouvelle réponse. 11 : Pour fermer la réclamation, l'administrateur demande la clôture. 12 : Le système ferme la réclamation et met à jour son état. 13 : L'interface affiche la clôture de la réclamation.
Post-condition	La réclamation est mise à jour avec une nouvelle réponse ou est fermée.
Scénario alternatif	A1 : Si l'administrateur décide de fermer directement la réclamation, il demande la clôture. 01 : Le scénario principal reprend au point 12.

TABLEAU 4.3 : Description textuelle du cas d'utilisation « Répondre aux réclamations »

4.2.4 Diagramme de classes

La figure 4.2 illustre le diagramme de classes du sprint 3.

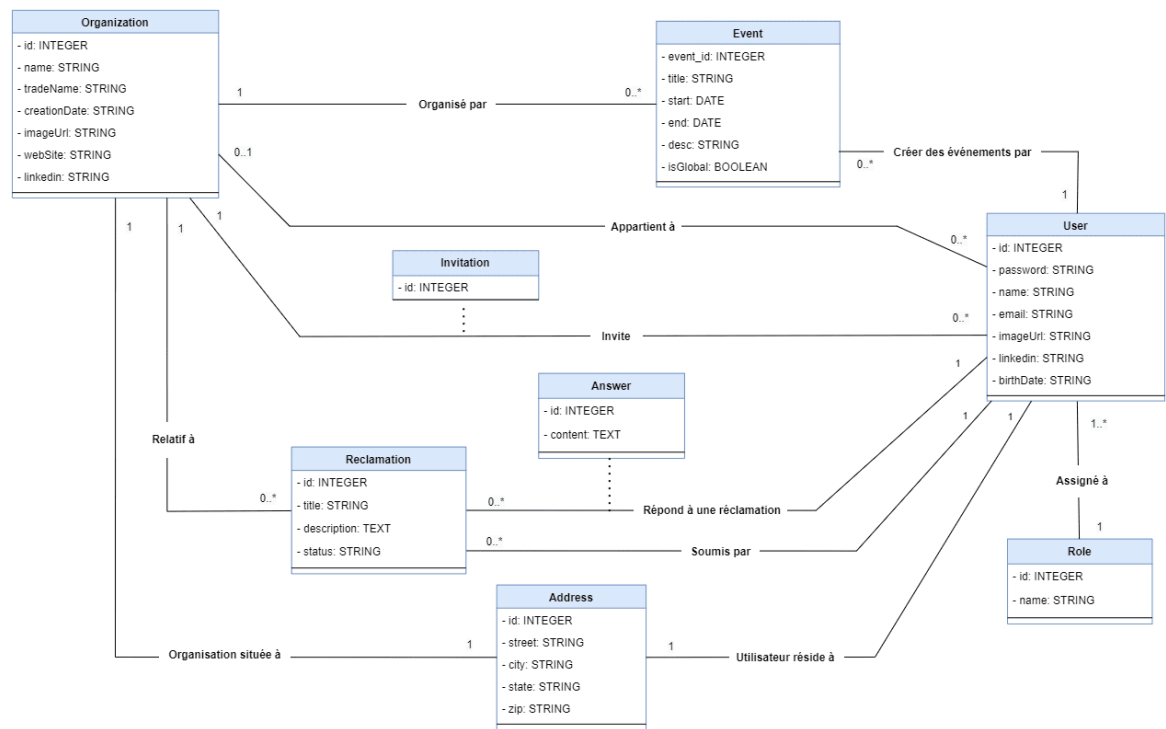


FIGURE 4.2 : Diagramme de classes de la gestion des réclamations et la planification des événements

4.2.5 Diagrammes de séquence

Dans cette partie, nous présentons les diagrammes de séquence système des cas d'utilisation «Création des réclamations», «Répondre aux réclamations» et «Planifier des événements», illustrés respectivement par les figures 4.3 , 4.4 et 4.5.

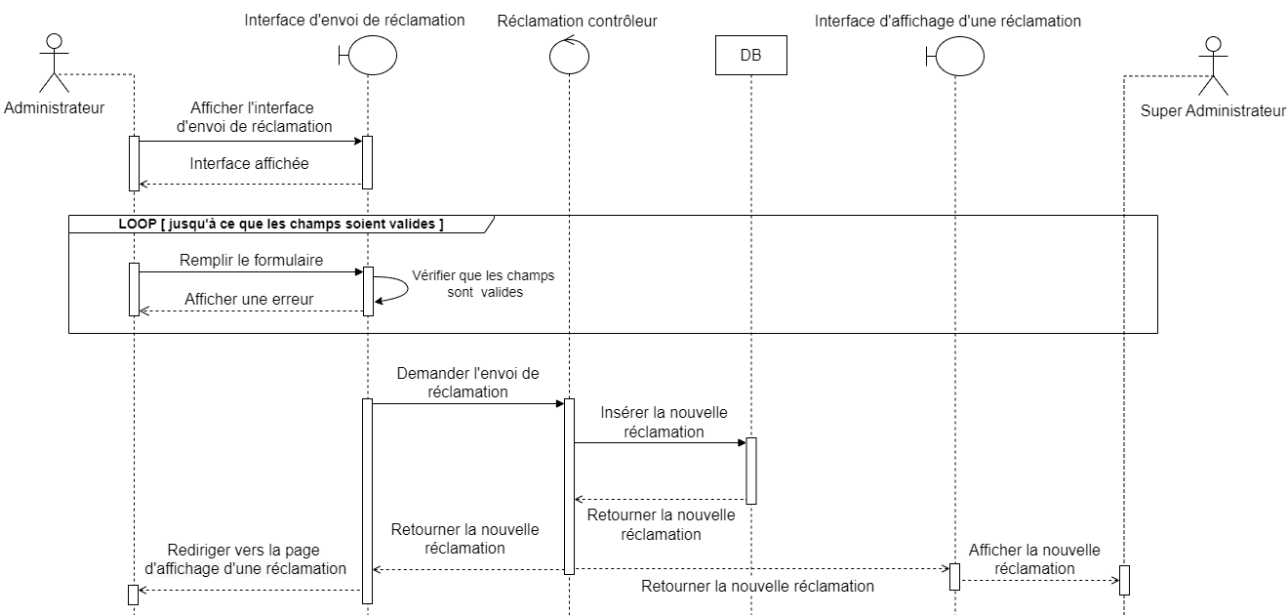


FIGURE 4.3 : Diagramme de séquence du scénario « Création des réclamations »

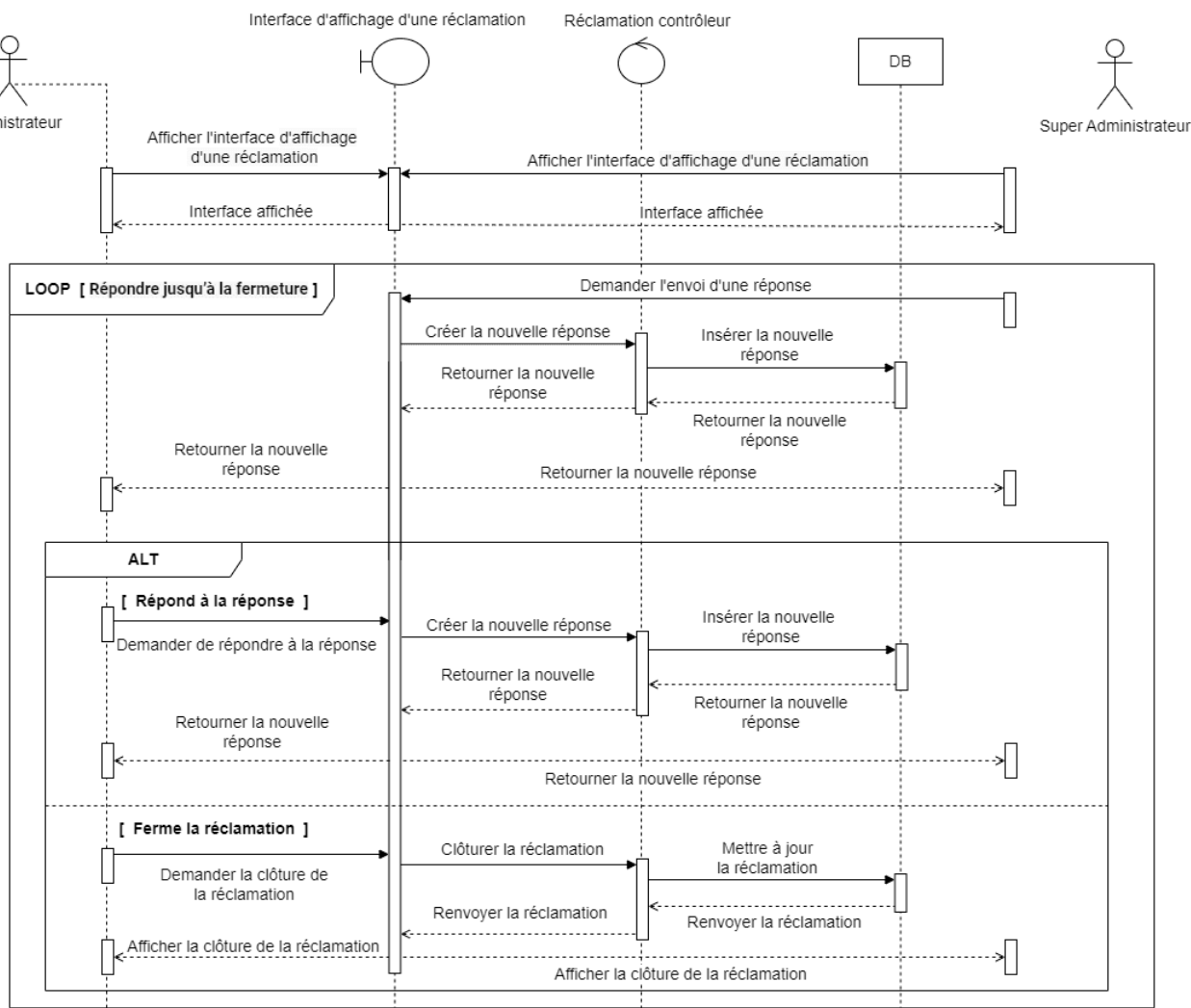


FIGURE 4.4 : Diagramme de séquence du scénario « Répondre aux réclamations »

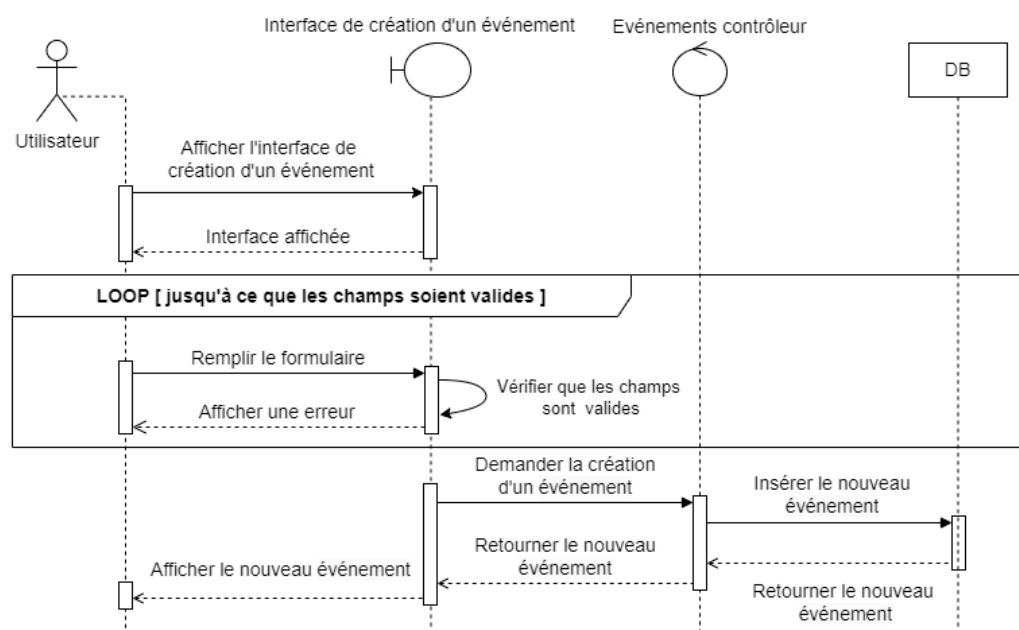


FIGURE 4.5 : Diagramme de séquence du scénario « Planifier des événements »

4.3 Réalisation Sprint 3

Dans cette section, nous présentons les interfaces utilisateur qui illustrent les services développés lors du sprint 3. Nous avons structuré méthodiquement les différentes fonctionnalités pour offrir des interfaces simples, claires et conviviales, validées par le propriétaire du produit. Les captures d'écran des interfaces réalisées au cours du sprint 3 de la release 2 sont affichées ci-dessous.

La figure 4.6 représente l'interface de la liste des réclamations pour tous les utilisateurs.

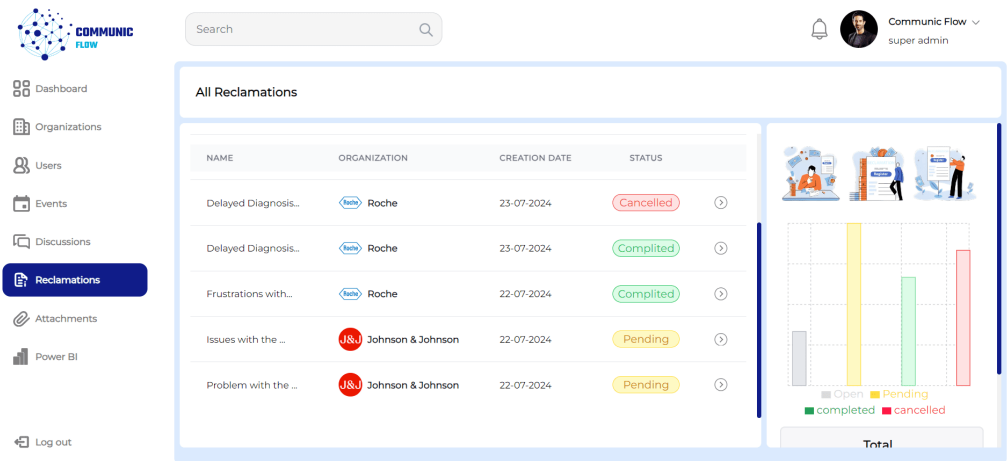


FIGURE 4.6 : Interface de la liste des réclamations

La figure 4.7 représente l'interface de création de réclamation pour l'administrateur.

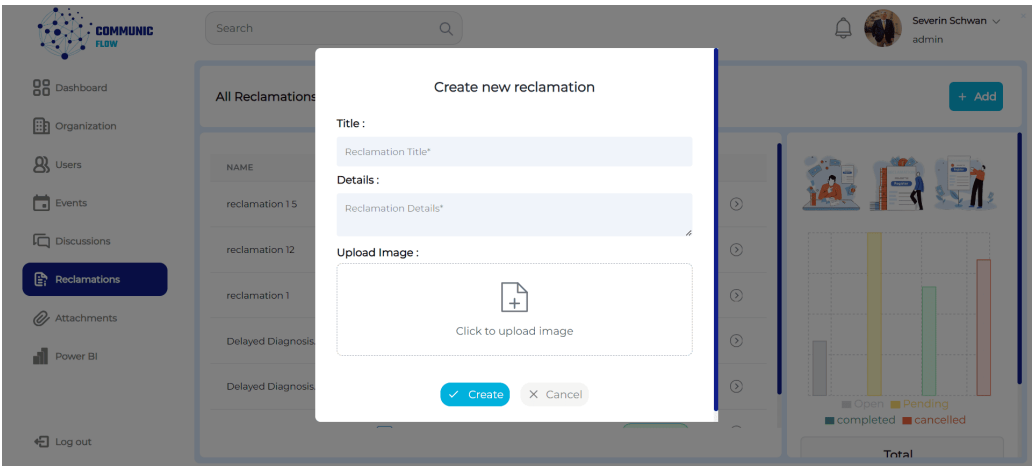


FIGURE 4.7 : Interface de création de réclamation

La figure 4.8 représente l'interface de réponse à une réclamation pour le super administrateur.

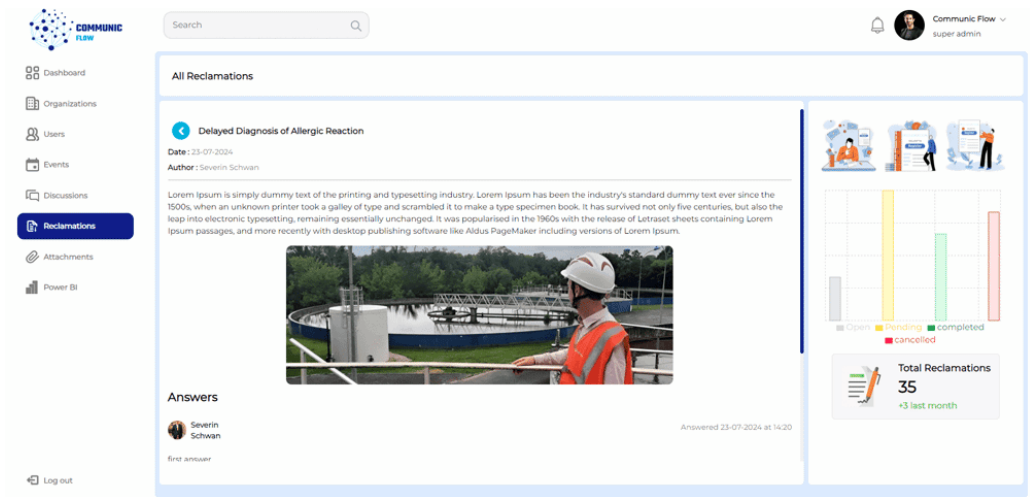


FIGURE 4.8 : Interface de réponse à une réclamation

La figure 4.9 représente l'interface de la liste des événements pour tous les utilisateurs.

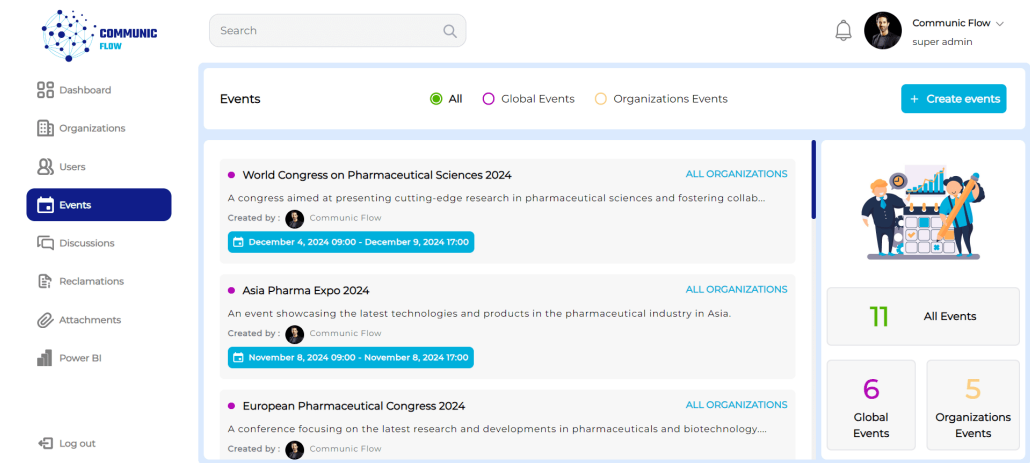


FIGURE 4.9 : Interface de la liste des événements

La figure 4.10 représente l'interface de planification d'événements pour les administrateurs et les super administrateurs.

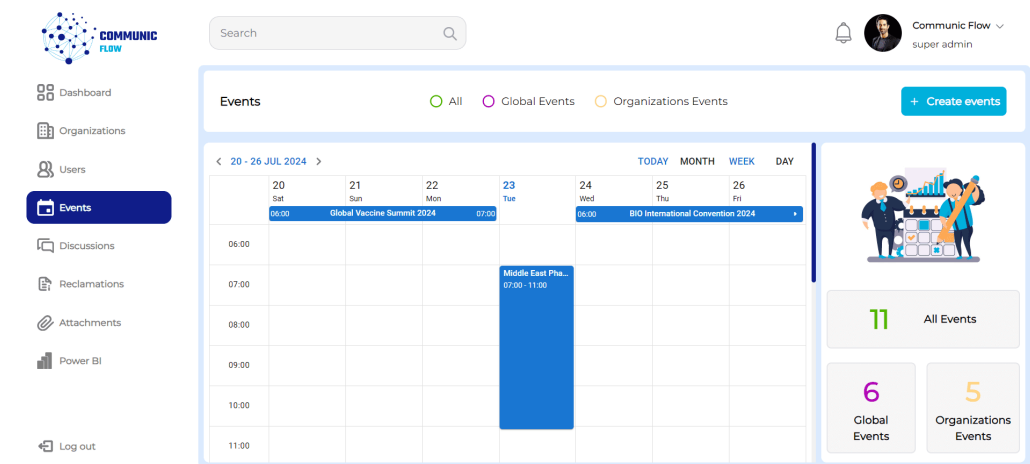


FIGURE 4.10 : Interface de planification d'événements

4.4 Sprint 4 : Gestion des Discussions en Temps Réel

Dans cette section, nous détaillons les étapes de mise en œuvre du sprint « Gestion des Discussions en Temps Réel ».

4.4.1 Objectifs du Sprint 4

L'objectif du quatrième sprint est de développer la gestion des discussions en temps réel. Il permet aux super administrateurs de discuter avec tous les utilisateurs, de créer des groupes de discussion et de supprimer des membres de ces groupes. Les administrateurs peuvent dialoguer avec tous les membres de leur organisation affiliée, créer des groupes de discussion au sein de celle-ci et retirer des membres de ces groupes. Enfin, les membres peuvent échanger avec tous les autres membres de leur organisation affiliée.

4.4.2 Backlog du sprint 4

Le tableau 4.4 présente le contenu du backlog du sprint « Gestion des Discussions en Temps Réel ».

Fonctionnalités	Priorité	Estimation (jours)
✓ Discuter avec tous les utilisateurs (Super Administrateur)	1	7
✓ Discuter avec tous les membres de son organisation affiliée (Administrateur)	1	4
✓ Discuter avec le Super Administrateur et les autres administrateurs des autres organisations affiliées (Administrateur)	1	4
✓ Discuter avec l'administrateur de son organisation affiliée (Membre)	1	3
✓ Créer des groupes de discussion (Super Administrateur)	1	5
✓ Créer des groupes de discussion avec n'importe quel membre de son organisation affiliée (Administrateur)	1	4
✓ Retirer un membre des groupes de discussion (Super Administrateur ou Administrateur)	1	3

TABLEAU 4.4 : Backlog du sprint 4

4.4.3 Analyse fonctionnelle du sprint 4

4.4.3.1 Diagramme des cas d'utilisation

La figure 4.11 représente le diagramme de cas d'utilisation du sprint « Gestion des Discussions en Temps Réel ».

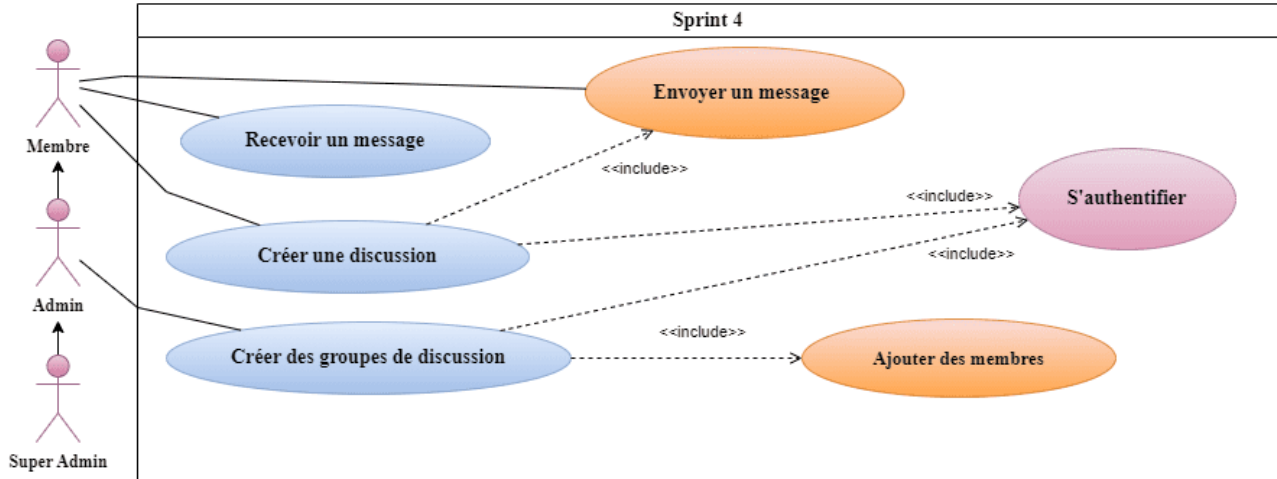


FIGURE 4.11 : Diagramme de cas d'utilisation pour la gestion des Discussions en Temps Réel

4.4.3.2 Description textuelle du cas d'utilisation pour la gestion des Discussions en Temps Réel

En fournissant une description textuelle d'un cas d'utilisation, nous clarifions le fonctionnement détaillé de la fonctionnalité et spécifions la séquence d'actions effectuées par l'acteur. Les tableaux 4.5 et 4.6 illustrent respectivement la description des cas d'utilisation « Envoyer un message » et « Création de groupe de discussion ».

Envoyer un message :

Le système permet aux participants authentifiés et connectés d'envoyer des messages via une interface de discussion. Lorsqu'un participant A envoie un message, le système crée ce message, l'enregistre dans la base de données, et l'affiche immédiatement dans l'interface de discussion pour le participant A. Si le participant B est également connecté, le message est affiché pour lui également en temps réel. Si le participant B n'est pas connecté au moment de l'envoi, le message sera disponible pour lui lors de sa prochaine connexion.

Titre	Envoyer message
Acteur	Participant A, Participant B
Pré-condition	Les participants doivent être authentifiés et connectés à l'interface de discussion.
Scénario nominal	01 : Le participant A accède à l'interface de discussion. 02 : Le système affiche l'interface de discussion. 03 : Le participant A demande l'envoi d'un message. 04 : Le système crée un nouveau message. 05 : Le système insère le nouveau message dans la base de données. 06 : Le système retourne le nouveau message. 07 : Le système affiche le nouveau message à l'interface de discussion. 08 : Si le participant B est connecté, le système affiche également le nouveau message pour le participant B.
Post-condition	Le nouveau message est créé, stocké dans la base de données, et affiché aux participants connectés.
Scénario alternatif	A1 : Si le participant B n'est pas connecté, le message sera affiché lors de la prochaine connexion du participant B.

TABLEAU 4.5 : Description textuelle du cas d'utilisation « Envoyer message »

Créer groupe de discussion :

Le système permet à un utilisateur authentifié de créer un groupe de discussion. L'utilisateur accède à l'interface dédiée, où il remplit un formulaire pour la création du groupe. Après vérification des champs du formulaire, le système crée le groupe, l'enregistre dans la base de données, et l'affiche dans l'interface de groupe de discussion. En cas de champs manquants ou vides, un message d'erreur est affiché pour guider l'utilisateur, qui peut alors corriger les informations et soumettre à nouveau le formulaire.

Titre	Créer groupe de discussion
Acteur	Utilisateur
Pré-condition	L'utilisateur doit être authentifié.
Scénario nominal	01 : L'utilisateur accède à l'interface de groupe de discussion. 02 : Le système affiche l'interface de groupe de discussion. 03 : L'utilisateur remplit le formulaire de création de groupe. 04 : Le système vérifie que les champs ne sont pas vides. 05 : L'utilisateur demande la création d'un groupe. 06 : Le système crée le nouveau groupe. 07 : Le système insère le nouveau groupe dans la base de données. 08 : Le système retourne le nouveau groupe. 09 : L'interface de groupe de discussion affiche le nouveau groupe.
Post-condition	Le nouveau groupe est créé, stocké dans la base de données, et affiché dans l'interface de groupe de discussion.
Scénario alternatif	A1 : Si les champs obligatoires sont vides, le système affiche un message d'erreur. 01 : L'utilisateur remplit les champs manquants et soumet à nouveau le formulaire. 02 : Le scénario principal reprend au point 04.

TABLEAU 4.6 : Description textuelle du cas d'utilisation « Créer groupe de discussion »

4.4.4 Diagramme de classes

La figure 4.12 illustre le diagramme de classes du sprint 4.

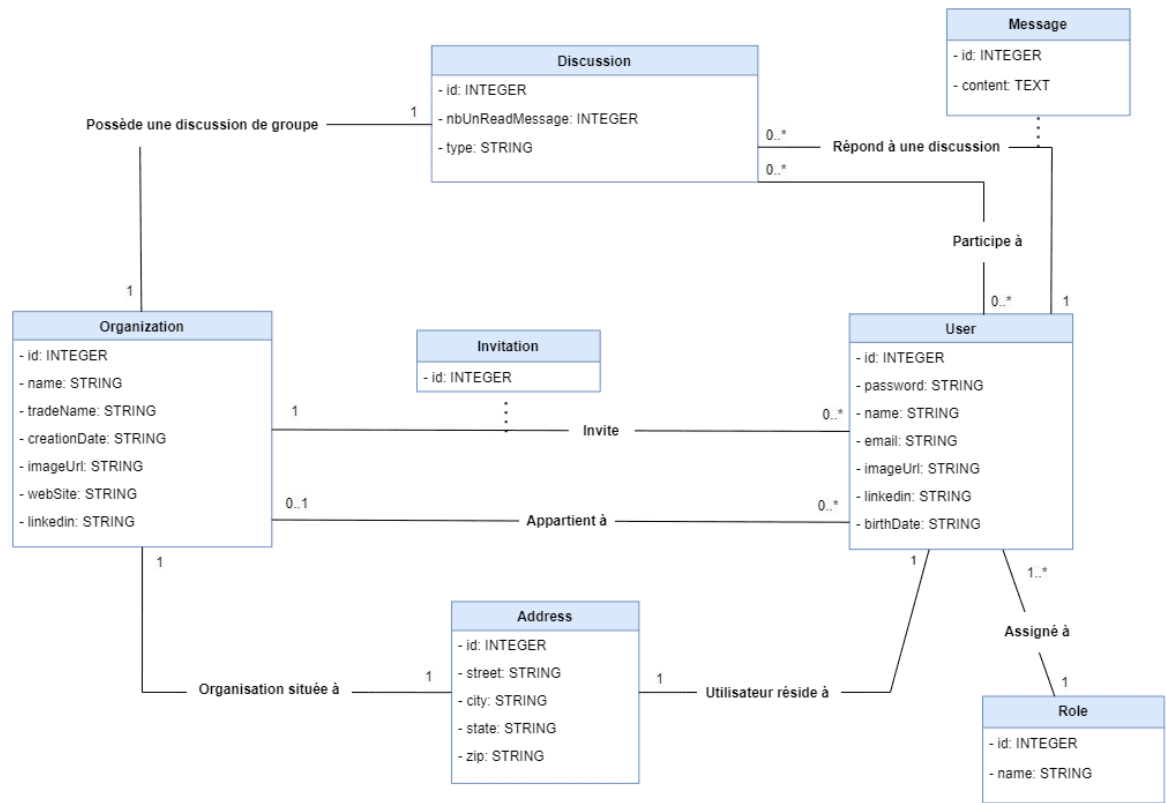


FIGURE 4.12 : Diagramme de classes du sprint 4

4.4.5 Diagrammes de séquence

Dans cette partie, nous présentons les diagrammes de séquence système des cas d'utilisation « Envoyer un message » et « Création de groupe de discussion », illustrés respectivement par les figures 4.13 et 4.14.

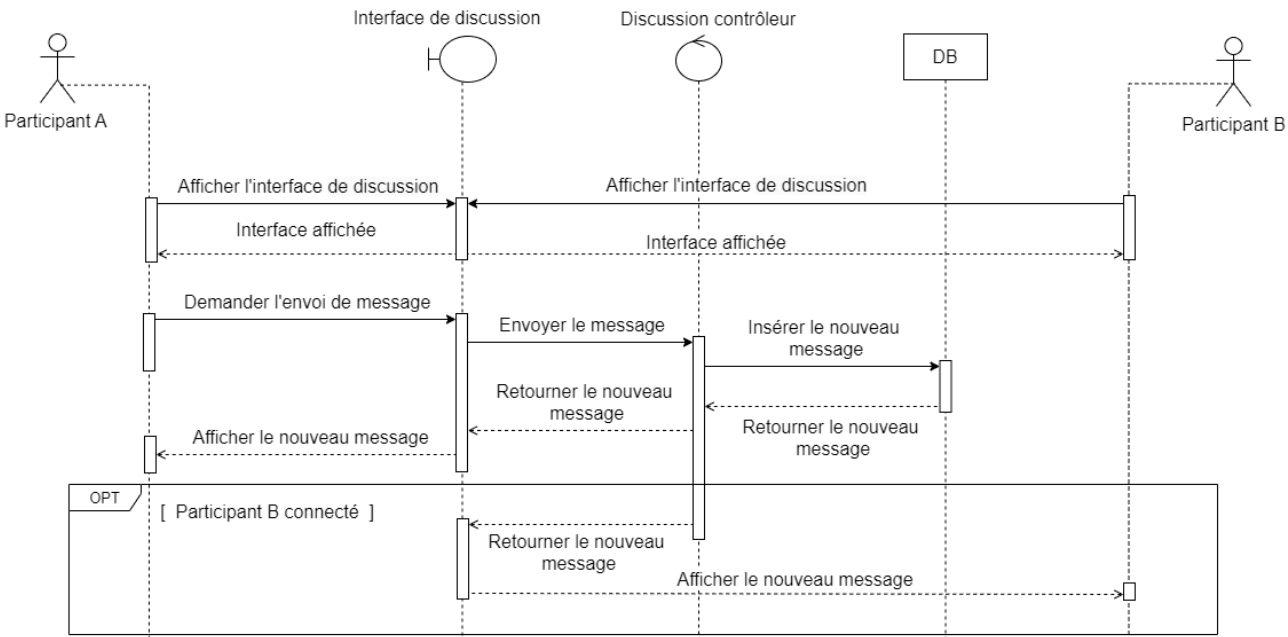


FIGURE 4.13 : Diagramme de séquence du scénario « Envoyer un message »

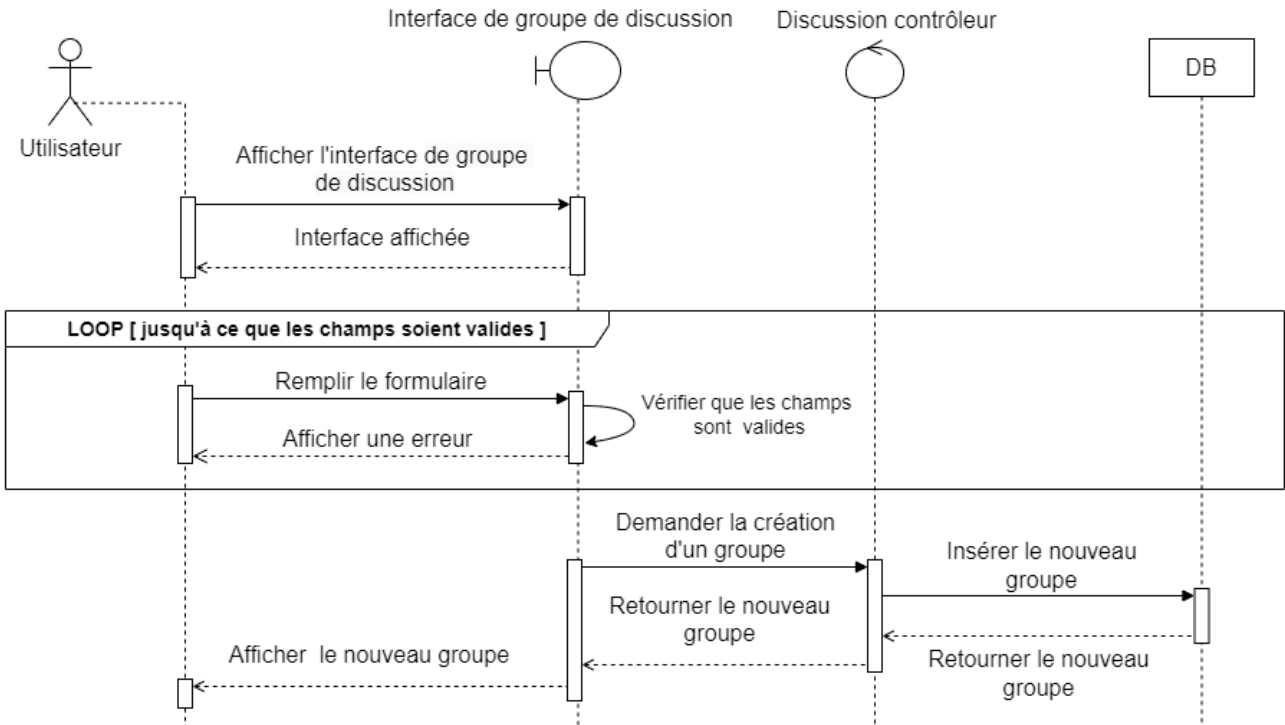


FIGURE 4.14 : Diagramme de séquence du scénario « Création de groupe de discussion »

4.5 Réalisation Sprint 4

Nous présentons ci-dessous des captures d’écran de quelques interfaces réalisées lors du sprint 4 de la release 2.

La figure 4.15 représente l’interface de liste de discussions pour tous les utilisateurs.

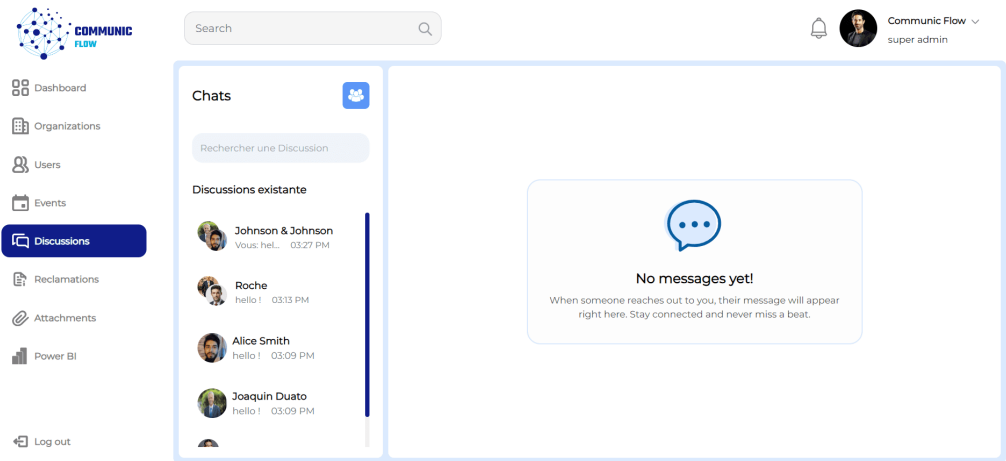


FIGURE 4.15 : Interface des listes de discussions

La figure 4.16 représente l'interface de création de groupe.

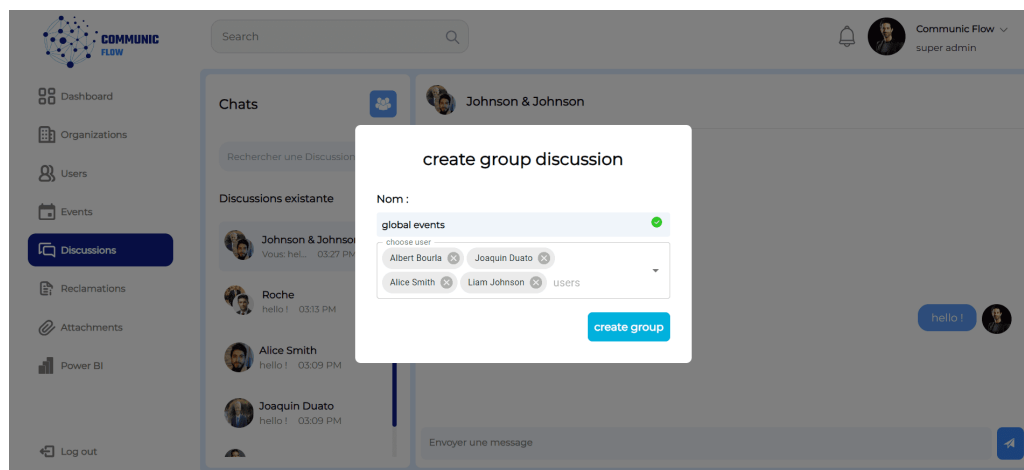


FIGURE 4.16 : Interface de création de groupe

La figure 4.17 représente l'interface d'un exemple de discussion de groupe.

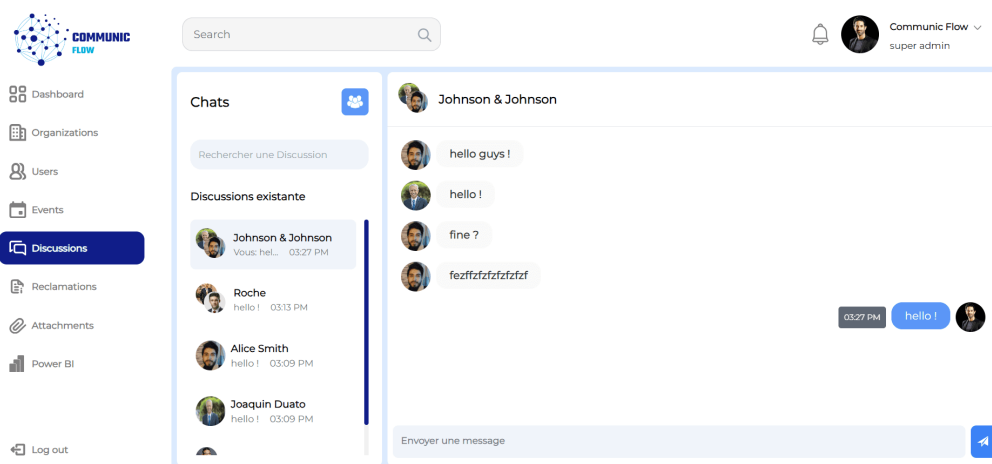


FIGURE 4.17 : Interface d'un exemple de discussion de groupe

4.6 Conclusion

En conclusion, ce chapitre a détaillé la mise en œuvre des fonctionnalités clés de la deuxième release de notre système. Les Sprints 3 et 4 ont marqué des progrès significatifs dans la gestion des réclamations, la planification des événements et l'intégration d'une messagerie instantanée. Chaque sprint avait été spécifiquement conçu pour répondre aux besoins variés des utilisateurs, des administrateurs aux super administrateurs, en passant par les membres réguliers.

Dans le chapitre suivant, nous avons abordé le domaine crucial du DevOps et du déploiement, en intégrant ces concepts pour optimiser le cycle de vie des applications et garantir une livraison continue et fiable des services informatiques.

INTÉGRATION CONTINUE/LIVRAISON CONTINUE, DÉPLOIEMENT CONTINU

Plan

5.1	Introduction	70
5.2	Approche intégration, livraison, déploiement continus	71
5.3	Définition de DevOps	71
5.4	Approche virtualisation/conteneurisation	71
5.5	Réalisation	72
5.6	Conclusion	100

5.1 Introduction

Dans ce chapitre, nous avons exploré les concepts d'intégration continue (CI), de livraison continue (CD) et de déploiement continu. Ces pratiques ont été essentielles pour garantir la qualité et la rapidité de mise en production de notre application. Elles ont permis d'automatiser les processus de test, de build et de déploiement, assurant ainsi une livraison fiable et rapide. La figure 5.1 illustre le diagramme de notre architecture CI/CD.

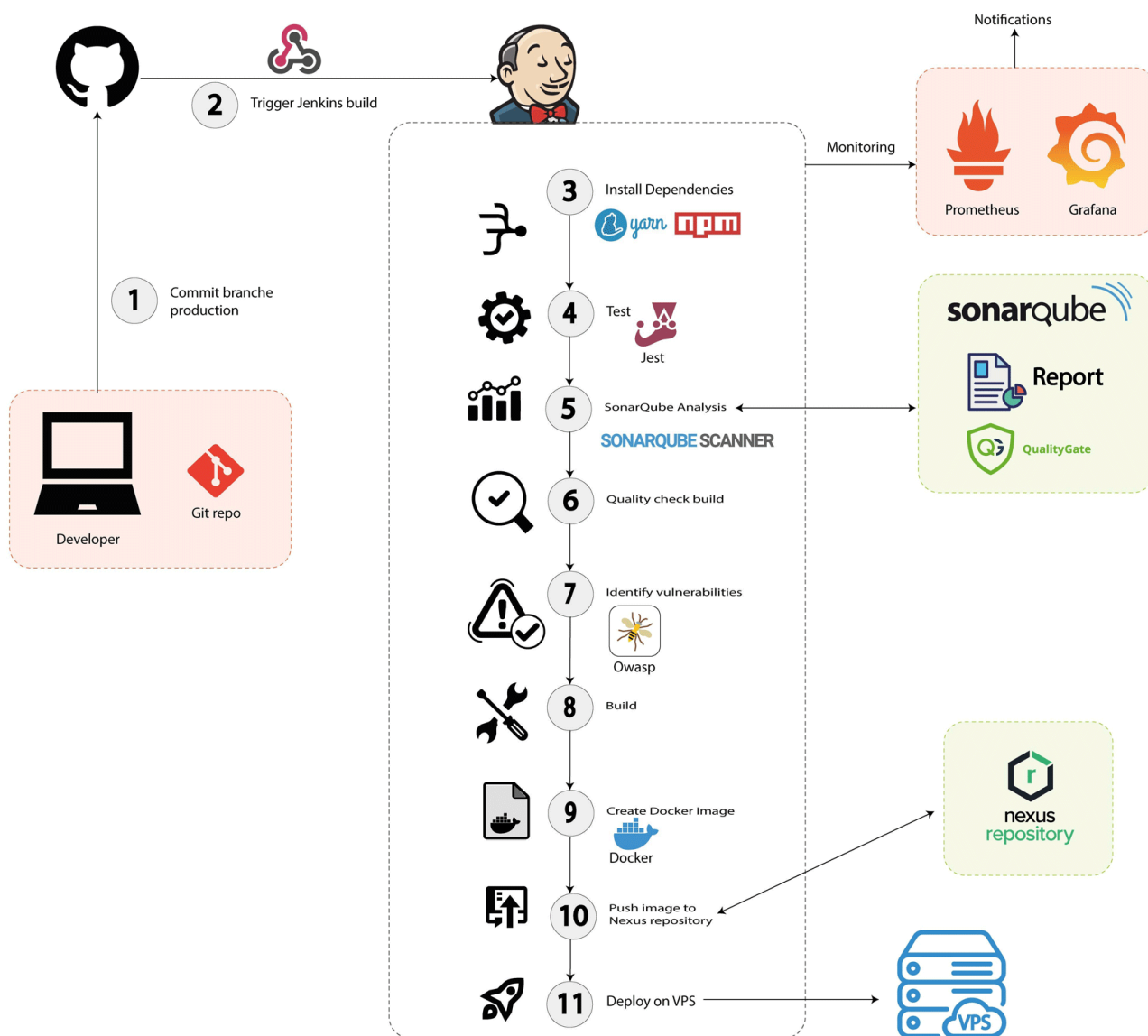


FIGURE 5.1 : Diagramme de l'architecture CI/CD

5.2 Approche intégration, livraison, déploiement continu

5.2.1 Intégration continue (CI)

L'intégration continue est une pratique où les développeurs intègrent fréquemment leur code dans un dépôt central, chaque intégration étant validée par une build automatique. Cela permet de détecter et corriger les erreurs rapidement, avant qu'elles ne deviennent des problèmes plus graves. Les tests automatisés jouent un rôle crucial dans cette étape, garantissant que chaque modification n'affecte pas la stabilité du système.[49]

5.2.2 La livraison continue/déploiement continu (CD)

La livraison continue garantit que le code est toujours dans un état déployable, tandis que le déploiement continu automatise le déploiement des modifications en production. Ces pratiques réduisent le temps de mise sur le marché et augmentent la fréquence de livraison des nouvelles fonctionnalités. Elles permettent également de minimiser les risques en déployant les changements par petites incrémentations.[50]

5.3 Définition de DevOps

DevOps est un ensemble de pratiques qui combine le développement et les opérations pour améliorer la collaboration et automatiser les processus de livraison logicielle. Cette approche vise à créer une culture de collaboration entre les équipes de développement et d'exploitation. L'objectif est d'améliorer l'efficacité, d'accélérer les cycles de livraison, et de garantir une qualité constante.[51]

5.4 Approche virtualisation/conteneurisation

5.4.1 La virtualisation

La virtualisation crée plusieurs environnements virtuels à partir d'un seul matériel physique, chaque environnement fonctionnant comme une machine distincte. Cela permet d'optimiser l'utilisation des ressources et de simplifier la gestion des environnements de développement et de production. La virtualisation offre également une meilleure isolation entre les applications, réduisant ainsi les conflits potentiels.[52]

5.4.2 La Conteneurisation

La conteneurisation empaquette les applications et leurs dépendances dans des conteneurs, assurant un fonctionnement uniforme sur différents environnements. Les conteneurs sont légers et démarrent rapidement, ce qui les rend idéaux pour les environnements de développement agile. Cette approche garantit également que les applications fonctionnent de manière cohérente, indépendamment de l'environnement d'exécution.[53]

5.5 Réalisation

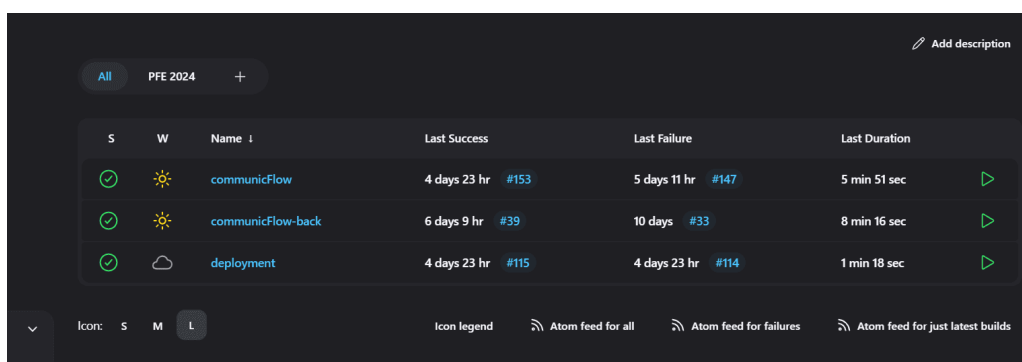
5.5.1 Mise en Place de l'infrastructure CI/CD

Pour automatiser et améliorer l'efficacité de notre cycle de développement, nous avons mis en place une infrastructure CI/CD (Intégration Continue et Déploiement Continu) robuste. Cette infrastructure repose sur plusieurs outils essentiels, notamment Jenkins pour l'automatisation des builds et des déploiements, SonarQube pour l'analyse de la qualité du code, Nexus Repository pour la gestion des artefacts, ainsi que Prometheus et Grafana pour la surveillance des performances. Voici les étapes détaillées de cette mise en place :

5.5.1.1 Jenkins

Jenkins est un serveur d'intégration continue open source qui permet d'automatiser les tâches de build, de test et de déploiement. Il est au cœur de notre infrastructure CI/CD, facilitant la gestion et l'exécution des pipelines pour les différents composants de notre projet.

Pour répondre aux besoins spécifiques de notre projet, nous avons configuré plusieurs éléments, ce qui a conduit à la création des 3 pipelines illustrés dans la figure 5.2.



S	W	Name	Last Success	Last Failure	Last Duration
✓	☀️	communicFlow	4 days 23 hr #153	5 days 11 hr #147	5 min 51 sec
✓	☀️	communicFlow-back	6 days 9 hr #39	10 days #33	8 min 16 sec
✓	☁️	deployment	4 days 23 hr #115	4 days 23 hr #114	1 min 18 sec

FIGURE 5.2 : Capture des Trois Pipelines Récemment Créés dans Jenkins

Installation et configuration des plugins nécessaires :

- GitHub Integration Plugin.
- NodeJS Plugin.
- Docker Pipeline.
- Docker plugin.
- SSH Agent Plugin.
- SonarQube Scanner for Jenkins.
- Prometheus metrics plugin.
- OWASP Dependency-Check Plugin.
- Copy Artifact Plugin.
- Credentials Binding Plugin.

Création des pipelines CI/CD :

- Pipeline Frontend « `communicFlow` ».
- Pipeline Backend « `communicFlow-back` ».
- Pipeline de Déploiement « `deployment` ».

Configuration des Credentials :

- Credentials GitHub : Pour l'accès aux dépôts GitHub.
- Credentials SonarQube : Pour l'authentification auprès de SonarQube.
- Credentials Docker Nexus : Pour la connexion au registre Docker Nexus.
- Credentials SSH VPS : Pour l'accès SSH au VPS pour le déploiement.

5.5.1.2 Webhooks GitHub

Pour automatiser le déclenchement des builds Jenkins à chaque commit, nous avons configuré des webhooks dans GitHub pour nos dépôts frontend et backend. En ajoutant l'URL de Jenkins dans les paramètres des dépôts GitHub, nous avons assuré que chaque push vers les branches principales déclenche automatiquement un build Jenkins. Dans nos pipelines Jenkins, nous avons activé le déclencheur "GitHub hook trigger for GITScm polling", ce qui garantit une intégration continue et un déploiement rapide des modifications de code.

5.5.1.3 SonarQube

Pour intégrer l'analyse de qualité de code dans notre pipeline CI/CD, nous avons configuré SonarQube avec nos projets frontend et backend. Voici les étapes principales réalisées :

Création de Projets SonarQube :

- Nous avons créé deux projets dans SonarQube : « `communicFlow` » pour le frontend et « `communicFlowBack` » pour le backend. Ces projets permettent de suivre la qualité du code de chaque composant séparément, comme le montre la figure 5.3.

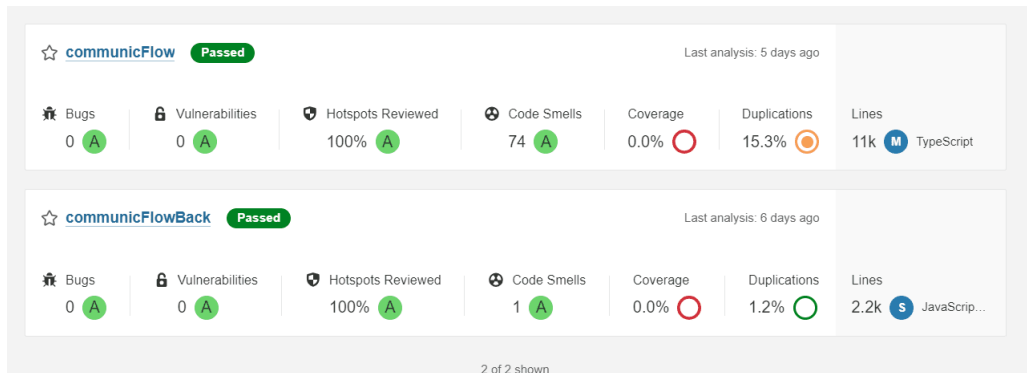


FIGURE 5.3 : Capture des Deux Projets SonarQube Récemment Créés

- Nous avons ajouté le fichier `sonar-project.properties` dans les répertoires des projets frontend et backend pour spécifier les configurations nécessaires à l'analyse SonarQube, comme illustré dans les figures 5.4 et 5.5.

```

sonar-project.properties
You, last week | 1 author (You)
1 sonar.projectKey=communicFlow      You, last month via PR #80 • 20-5-2024
2 sonar.projectName=communicFlow
3 sonar.projectVersion=1.0
4 sonar.language=ts
5 sonar.sources=src
6 sonar.test.inclusions=**/*.test.tsx
7 sonar.typescript.tsconfigPath=tsconfig.sonar.json
8 sonar.scm.disabled=true
9 sonar.sourceEncoding=UTF-8
10 sonar.inclusions=**/*.ts,**/*.tsx,**/*.json,**/*.js,**/*.css
11 sonar.exclusions=**/node_modules/**,**/*.spec.ts,coverage/**,**/*.html,**/*.xml,Dockerfile,**/*.config.js
12 sonar.log.level=DEBUG

```

FIGURE 5.4 : Configuration "sonar-project.properties" pour le Frontend

```

sonar-project.properties
You, 2 weeks ago | 1 author (You)
1 sonar.projectKey=communicFlowBack  You, last month • 21-5-2024
2 sonar.projectName=communicFlowBack
3 sonar.sources=.
4 sonar.sourceEncoding=UTF-8
5 sonar.scm.disabled=true
6 sonar.exclusions=node_modules/**,coverage/**,**/*.html,**/*.xml,Dockerfile

```

FIGURE 5.5 : Configuration "sonar-project.properties" pour le Backend

Configuration des Quality Gates : Nous avons défini des Quality Gates spécifiques pour chacun des projets, établissant ainsi des critères de qualité qui doivent être respectés avant de permettre la progression des builds.

Génération de Tokens d'Analyse : Pour sécuriser les analyses, nous avons généré des tokens d'authentification dans SonarQube. Ces tokens sont utilisés dans les pipelines Jenkins pour authentifier les analyses de code.

5.5.1.4 Nexus Repository

Nexus Repository est un gestionnaire de dépôts qui permet de stocker et de distribuer des composants et des bibliothèques de manière centralisée. Pour notre projet, nous avons utilisé Nexus Repository pour héberger les images Docker de nos applications. Voici les étapes principales réalisées :

Installation et Configuration de Nexus Repository : Nous avons installé et configuré Nexus Repository sur notre serveur pour gérer nos artefacts Docker. Cette configuration permet de stocker de manière sécurisée et efficace les images Docker nécessaires à notre pipeline de déploiement.

Création de Dépôt Docker : Nous avons créé un dépôt Docker nommé «communic-flow» dans Nexus Repository. Ce dépôt de type "hosted" avec le format Docker permet de gérer les images "Docker" de notre projet. Cette étape assure que les images Docker sont stockées de manière centralisée et sont prêtes pour le déploiement, comme illustré dans la figure 5.6.

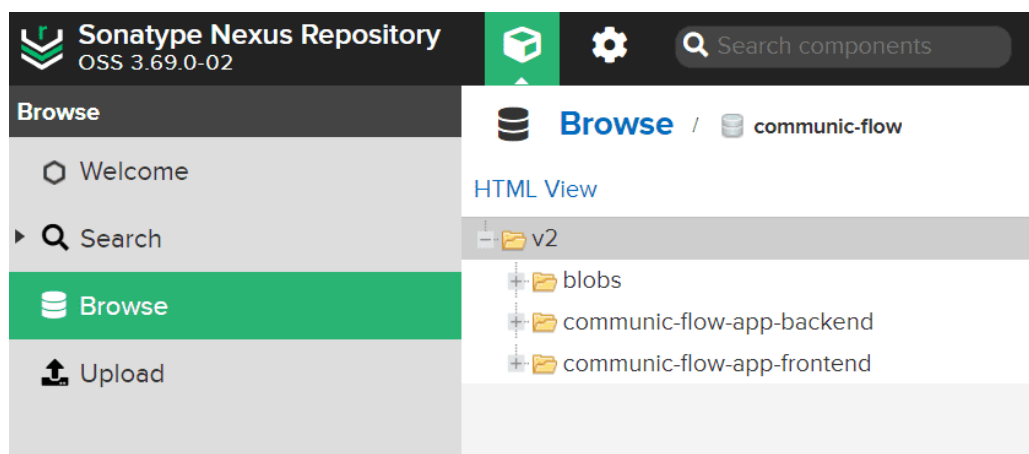


FIGURE 5.6 : Capture du Dépôt Docker « communic-flow » dans Nexus Repository

En intégrant Nexus Repository dans notre infrastructure CI/CD, nous garantissons une gestion efficace et sécurisée de nos artefacts, facilitant ainsi le déploiement et la maintenance de nos applications.

5.5.1.5 Prometheus

Prometheus est un système de surveillance et d'alerte open source utilisé pour collecter et stocker des métriques. Dans notre infrastructure, Prometheus est utilisé pour surveiller les performances et l'état de nos applications. Voici les étapes principales réalisées :

Installation et Configuration de Prometheus : Nous avons installé Prometheus sur notre serveur et configuré les fichiers de configuration nécessaires pour surveiller nos applications. Cette configuration permet de collecter des métriques à partir de différentes sources et de les stocker pour une analyse ultérieure.

Configuration des Cibles de Surveillance : La configuration des cibles de surveillance dans Prometheus a été réalisée à l'aide du plugin Prometheus pour Jenkins. Ce plugin permet de définir automatiquement les endpoints de nos applications frontend et backend, assurant que Prometheus collecte les métriques pertinentes en temps réel. La figure 5.7 illustre cette configuration.

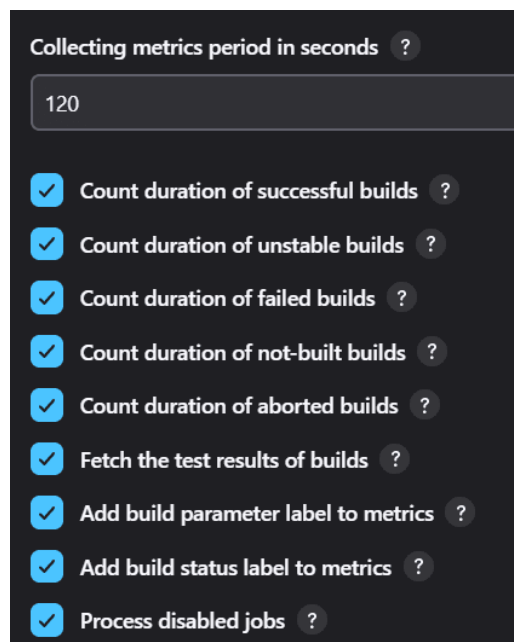


FIGURE 5.7 : Capture de la Configuration des Cibles de Surveillance dans Prometheus

En intégrant Prometheus dans notre infrastructure, nous assurons une surveillance continue et proactive de nos applications, facilitant la détection et la résolution des problèmes de performance et de disponibilité.

5.5.1.6 Grafana

Grafana est un outil de visualisation et de surveillance open source utilisé pour créer des tableaux de bord interactifs à partir de diverses sources de données, y compris Prometheus. Dans notre infrastructure, Grafana est utilisé pour visualiser les métriques collectées par Prometheus. Voici les étapes principales réalisées :

Installation et Configuration de Grafana : Nous avons installé Grafana sur notre serveur et configuré les paramètres de connexion à notre instance Prometheus. Cette configuration permet de créer des tableaux de bord pour visualiser les métriques collectées et analyser les performances de nos applications.

Création des Tableaux de Bord : Nous avons créé plusieurs tableaux de bord dans Grafana pour visualiser les métriques critiques de nos applications frontend et backend. Ces tableaux de bord fournissent des vues détaillées des performances et de l'état des applications, facilitant la surveillance et le diagnostic, comme illustré dans la figure 5.8.

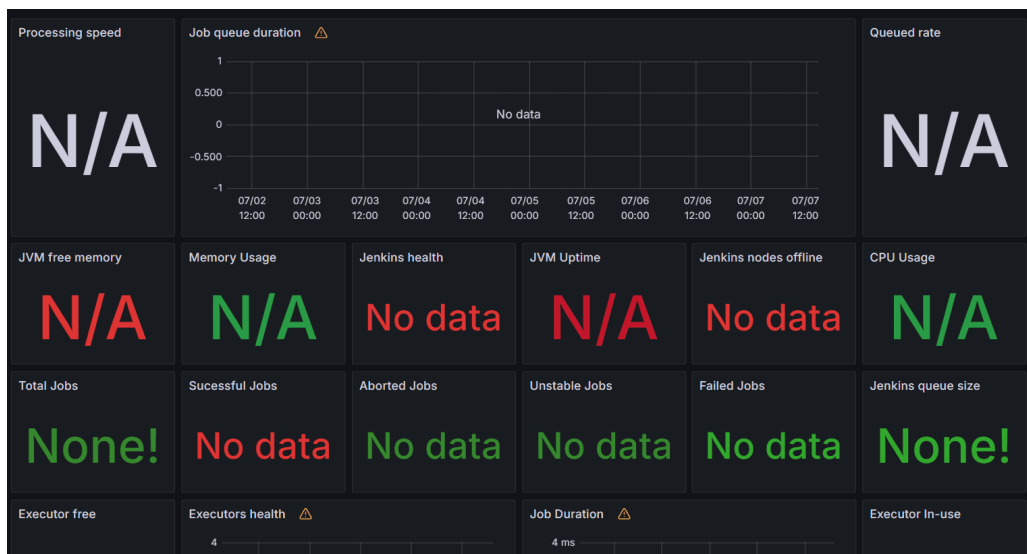


FIGURE 5.8 : Capture des tableaux de bord Grafana pour la surveillance des applications

En utilisant Grafana pour visualiser les métriques collectées par Prometheus, nous avons une vue complète et interactive des performances de nos applications, ce qui améliore notre capacité à surveiller, diagnostiquer et résoudre les problèmes rapidement.

5.5.1.7 Synthèse

La mise en place de l'infrastructure CI/CD avec des outils comme Jenkins, GitHub, SonarQube, Nexus Repository, Prometheus et Grafana a été essentielle pour automatiser et améliorer l'efficacité de notre cycle de développement, garantissant des déploiements rapides et fiables.

5.5.2 Pipeline CI/CD Frontend

Le pipeline CI/CD pour le frontend, nommé « *communicFlow* », joue un rôle crucial dans notre projet en assurant une intégration et un déploiement continu. Ce pipeline, destiné à une application développée en React TS, automatise des tâches essentielles telles que l'installation des dépendances, l'exécution des tests unitaires, l'analyse de la qualité du code, la vérification des gates de qualité, l'archivage de l'espace de travail et le déclenchement du pipeline de déploiement. Cette approche garantit que chaque changement de code est validé, testé et déployé de manière fiable et efficace, comme illustré dans la figure 5.9.

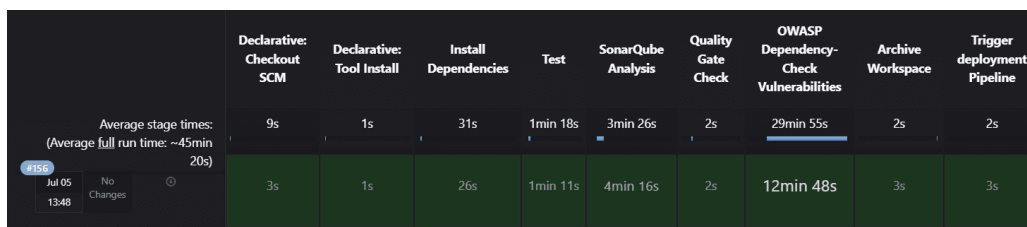


FIGURE 5.9 : Capture d'écran de la pipeline Jenkins pour le frontend.

5.5.2.1 Installation des Dépendances

Pour installer les dépendances nécessaires au projet frontend, nous utilisons Yarn, un gestionnaire de paquets JS/TS. Cette étape assure que toutes les bibliothèques et outils requis sont disponibles avant de procéder aux tests et aux analyses.

5.5.2.2 Tests Unitaires

Pour garantir le bon fonctionnement de notre frontend, nous avons configuré et exécuté des tests unitaires avec Jest. Ces tests assurent que les composants fonctionnent correctement et que les nouvelles modifications n'introduisent pas de régressions.

Script de Test : Nous avons créé un script de test, `test.sh`, qui exécute les tests unitaires en installant préalablement les dépendances nécessaires grâce à Jest. Le contenu détaillé de ce fichier est illustré dans la figure 5.10.

```
#!/bin/bash -ex
#!/usr/bin/env sh
set -x # Enable debugging output

# Clear Jest cache
yarn jest --clearCache
set +x # Disable debugging output

set -x # Enable debugging output
# Install JSDOM environment for Jest
yarn add --dev jest-environment-jsdom
set +x # Disable debugging output

set -x # Enable debugging output
# Run tests using Jest
yarn test
```

FIGURE 5.10 : test.sh

Exemple de Test Unitaire : Nous avons également créé des tests unitaires pour différents composants de notre application React. Un exemple est illustré dans la figure 5.11.

```
import { render, screen } from "@testing-library/react";
import "@testing-library/jest-dom";
import Tooltip from "../Tooltip";

describe("Tooltip component", () => {

  test("renders children", () => {
    render(

      <Tooltip>
        <div data-testid="child">Child content</div>
      </Tooltip>

    );

    expect(screen.getByTestId("child")).toBeInTheDocument();

  });

  test("renders tooltip when tooltip prop is provided", () => {

    render(<Tooltip tooltip="Tooltip content">Hover me</Tooltip>

    );

    expect(screen.getByText("Tooltip content")).toBeInTheDocument();

  });
});
```

FIGURE 5.11 : Tooltip.test.tsx

Ce test vérifie que le composant `ToolTip` fonctionne correctement dans différentes conditions, comme le rendu de base, la gestion des erreurs et les interactions utilisateur. En exécutant ce test dans notre pipeline CI/CD, nous garantissons que les nouvelles modifications n'affectent pas le comportement attendu de ce composant.

5.5.2.3 Analyse de Code avec SonarQube

Pour garantir la qualité du code, nous avons intégré SonarQube dans notre pipeline CI/CD pour le projet frontend « `communicFlow` ». L'analyse de SonarQube a identifié les bugs, les vulnérabilités et les mauvaises pratiques de codage, permettant de maintenir une base de code propre et maintenable.

Résultats de l'Analyse SonarQube : Les résultats des analyses SonarQube sont présentés dans la figure 5.12.

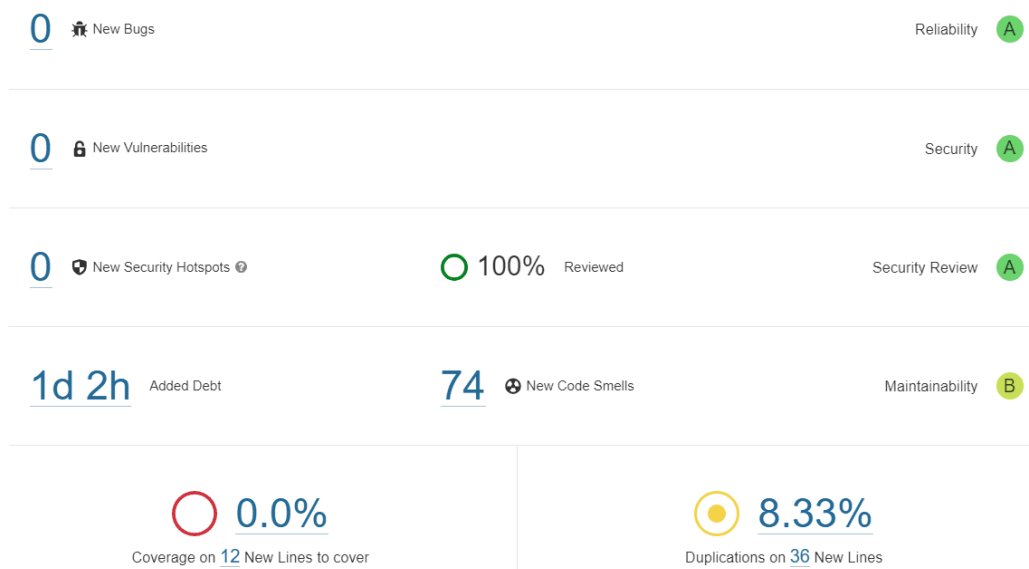


FIGURE 5.12 : Rapport détaillé de SonarQube montrant les bugs, vulnérabilités et code smells

- **Bugs :** Aucune nouvelle anomalie n'a été détectée. Initialement, 40 bugs étaient recensés, et ce chiffre est désormais réduit à 30.
- **Vulnérabilités :** Aucune nouvelle vulnérabilité n'a été détectée depuis les trois dernières analyses. Initialement, 5 vulnérabilités ont été corrigées, assurant une meilleure sécurité du code.
- **Code Smells :** 74 nouveaux code smells ont été détectés. Cela indique une amélioration significative en termes de lisibilité et de maintenabilité du code.
- **Coverage :** Le taux de couverture des tests unitaires n'a pas atteint notre objectif initial de 80%. Actuellement, il est de 0% sur les nouvelles lignes de code à couvrir.

5.5.2.4 Vérification de la Qualité du Build

Cette étape a vérifié si le code satisfaisait aux exigences de qualité définies par les Quality Gates de SonarQube. Si le code ne passait pas les critères de qualité, le pipeline était interrompu, empêchant ainsi le déploiement de code de mauvaise qualité.

Impact de la Vérification de Qualité

- **Détection Précoce des Problèmes** : Les bugs et vulnérabilités sont détectés et corrigés rapidement, avant qu'ils ne puissent affecter la production. Cela réduit le temps de résolution des problèmes et maintient la stabilité du code.
- **Amélioration Continue** : La surveillance continue de la qualité du code encourage les développeurs à adopter de meilleures pratiques de codage. Chaque commit est analysé, et les développeurs reçoivent des feedbacks immédiats sur leur code.
- **Conformité aux Standards** : Les Quality Gates définis garantissent que seules les modifications conformes aux standards de qualité sont déployées. Cela maintient un niveau élevé de qualité du code tout au long du cycle de développement.
- **Réduction des Régressions** : L'analyse automatique et les rapports détaillés de SonarQube aident à prévenir les régressions en identifiant les problèmes potentiels avant le déploiement. Cela permet de maintenir la qualité et la performance du code sur le long terme.

Ces résultats démontrent la réussite de notre démarche d'intégration continue et la stabilité accrue du projet frontend « `communicFlow` », comme le montrent également la figure 5.13.

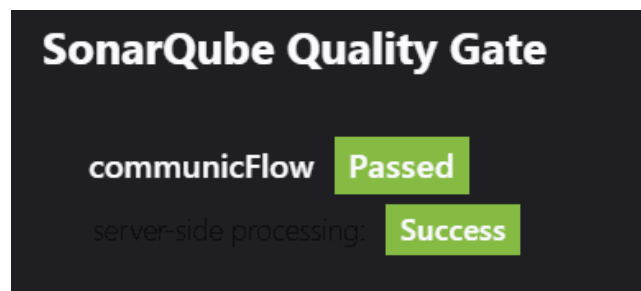


FIGURE 5.13 : Résultats de SonarQube pour le projet frontend « `communicFlow` »

5.5.2.5 Analyse des Vulnérabilités avec OWASP Dependency-Check

Pour assurer la sécurité de notre application frontend, nous avons intégré OWASP Dependency-Check dans notre pipeline CI/CD. Cet outil identifie les composants ayant des vulnérabilités connues.

Rapport de Dépendances : L'analyse des vulnérabilités a été intégrée dans notre pipeline frontend `communicFlow`. Le plugin OWASP Dependency-Check génère un rapport détaillé des vulnérabilités trouvées, comme illustré dans la figure 5.14.

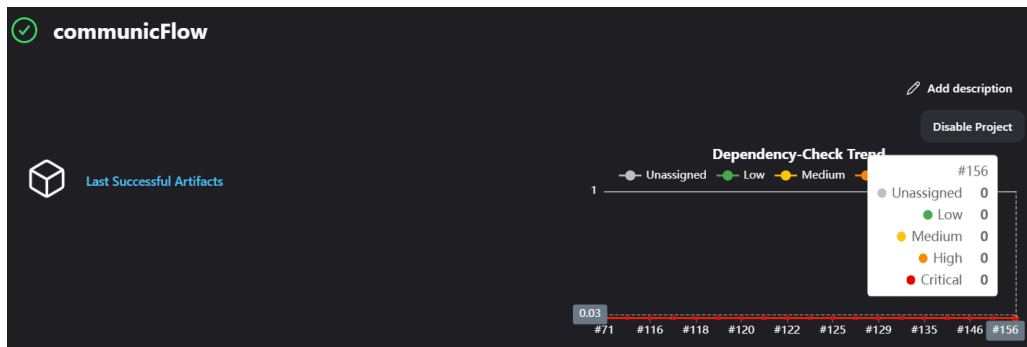


FIGURE 5.14 : Trend des Dépendances OWASP dans Jenkins

Résultats de l'Analyse : Les résultats de l'analyse de Dependency-Check sont essentiels pour identifier et corriger les vulnérabilités dans les dépendances du projet. Voici un résumé des résultats obtenus lors de notre dernière analyse, illustré dans la figure 5.15 :

- **Nombre de vulnérabilités trouvées :** 0 vulnérabilités critiques, 0 vulnérabilités élevées, 0 vulnérabilités moyennes.
- **Composants vulnérables :** Aucune vulnérabilité trouvée.
- **Actions correctives :** Aucune action nécessaire.

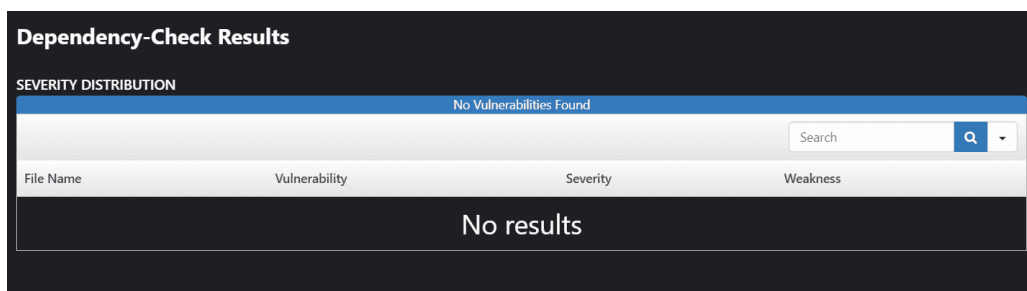


FIGURE 5.15 : Résultats de OWASP Dependency-Check dans Jenkins

Impact de l'Analyse de Vulnérabilités : L'intégration de OWASP Dependency-Check dans notre pipeline CI/CD frontend présente plusieurs avantages :

- **Détection Précoce :** Identification rapide des dépendances vulnérables avant qu'elles ne soient déployées en production.
- **Réduction des Risques :** Correction proactive des vulnérabilités pour minimiser les risques de sécurité.

- **Amélioration de la Confiance** : Maintien d'un niveau élevé de sécurité et de confiance dans notre base de code.

En intégrant OWASP Dependency-Check dans notre pipeline CI/CD frontend, nous avons renforcé la sécurité de notre application, garantissant ainsi une meilleure protection contre les vulnérabilités connues.

5.5.2.6 Archivage de l'Espace de Travail

L'archivage de l'espace de travail est une étape cruciale pour préserver les artefacts générés au cours du processus CI/CD. Cela inclut les fichiers de build, les rapports de tests et autres fichiers nécessaires pour la prochaine étape du pipeline ou pour des audits futurs. Cette étape assure la traçabilité et permet de récupérer facilement les artefacts si nécessaire. Pour automatiser ce processus dans notre pipeline frontend `communicFlow`, nous avons utilisé le plugin "Copy Artifact" de Jenkins.

Résultats de l'Archivage : L'archivage des artefacts a produit plusieurs avantages clés :

- **Traçabilité** : Tous les artefacts des builds sont sauvegardés, permettant de retracer les changements et les résultats de chaque build.
- **Accès aux Artefacts** : Les développeurs peuvent facilement accéder aux artefacts archivés pour analyse ou débogage.
- **Préparation pour le Déploiement** : Les artefacts nécessaires pour le déploiement sont disponibles et prêts à être utilisés dans les étapes suivantes du pipeline de déploiement.

Impact de l'Archivage : L'archivage de l'espace de travail assure que les artefacts sont toujours disponibles pour les étapes ultérieures du pipeline et pour les audits futurs. Cela améliore la robustesse et la fiabilité du processus CI/CD en garantissant que les informations nécessaires sont préservées à chaque étape.

Ces résultats démontrent la réussite de notre démarche d'intégration continue et la stabilité accrue du projet frontend « `communicFlow` », comme le montre également la figure 5.16.

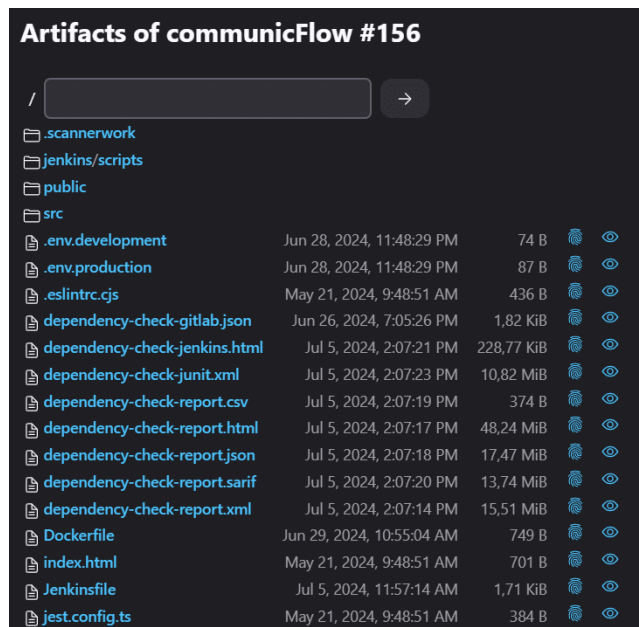


FIGURE 5.16 : Capture d'écran montrant le dernier artefact archivé dans Jenkins

En conclusion, l'archivage de l'espace de travail dans notre pipeline CI/CD frontend assure une gestion efficace des artefacts, facilitant ainsi la traçabilité et la préparation pour les déploiements futurs.

5.5.2.7 Déclenchement du Pipeline de Déploiement

À la fin du pipeline CI/CD frontend, nous déclenchons automatiquement le pipeline de déploiement pour déployer les artefacts générés dans l'environnement approprié.

5.5.2.8 Synthèse

Le pipeline CI/CD pour le frontend a permis une intégration et un déploiement continu de notre application React TS, assurant une qualité de code élevée grâce à des tests unitaires, des analyses de code et des vérifications de vulnérabilités automatisées, comme illustré dans la figure 5.17 qui présente le Jenkinsfile utilisé.

```

pipeline {
  agent any
  tools {
    nodejs "node"
  }
  stages {
    stage('Install Dependencies') {
      steps {
        bat 'yarn'
      }
    }
    stage('Test') {
      steps {
        bat 'sh ./jenkins/scripts/test.sh'
      }
    }
    stage('SonarQube Analysis') {
      environment {
        SCANNER_HOME = tool 'SonarQube'
      }
      steps {
        withSonarQubeEnv(installationName: 'sonar-jenkins') {
          bat "${SCANNER_HOME}/bin/sonar-scanner"
        }
      }
    }
    stage('Quality Gate Check') {
      steps {
        echo 'Quality Gate check....'
        timeout(time: 10, unit: 'MINUTES') {
          waitForQualityGate abortPipeline: true
        }
      }
    }
    stage('OWASP Dependency-Check Vulnerabilities') {
      steps {
        dependencyCheck additionalArguments: ''
          -o './'
          -s './'
          -f 'ALL'
          --prettyPrint'', odcInstallation: 'OWASP Dependency-Check Vulnerabilities'

        dependencyCheckPublisher pattern: 'dependency-check-report.xml'
      }
    }
    stage('Archive Workspace') {
      steps {
        archiveArtifacts artifacts: '**/*', excludes: 'node_modules/**', allowEmptyArchive:
true
      }
    }
    stage('Trigger deployment Pipeline') {
      steps {
        script {
          build job: 'deployment',
            wait: false
        }
      }
    }
  }
}

```

FIGURE 5.17 : Code du Jenkinsfile utilisé pour le pipeline CI/CD frontend.

5.5.3 Pipeline CI/CD Backend

Le pipeline CI/CD pour le backend, nommé « `communicFlow-back` », joue un rôle essentiel dans notre projet en assurant une intégration et un déploiement continu. Ce pipeline, destiné à une application développée en Node.js avec Express, automatise des tâches essentielles telles que l'installation des dépendances, l'exécution des tests unitaires, l'analyse de la qualité du code, la vérification des gates de qualité, l'archivage de l'espace de travail et le déclenchement du pipeline de déploiement. Cette approche garantit que chaque changement de code est validé, testé et déployé de manière fiable et efficace, comme illustré dans la figure 5.18.

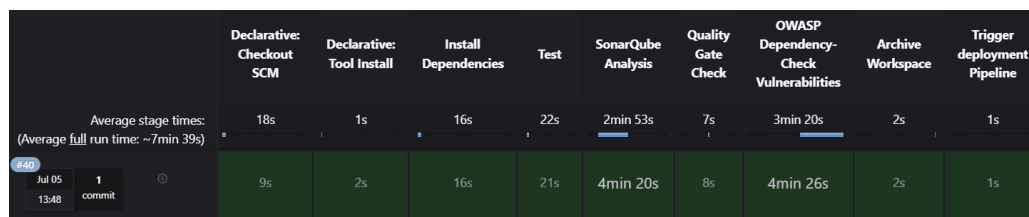


FIGURE 5.18 : Capture d'écran de la pipeline Jenkins pour le backend.

5.5.3.1 Installation des Dépendances

Pour installer les dépendances nécessaires au projet backend, nous utilisons npm. Cette étape assure que toutes les bibliothèques et outils requis sont disponibles avant de procéder aux tests et aux analyses.

5.5.3.2 Tests Unitaires

Pour garantir le bon fonctionnement de notre backend, nous avons configuré et exécuté des tests unitaires avec Jest. Ces tests assurent que les composants fonctionnent correctement et que les nouvelles modifications n'introduisent pas de régressions.

Script de Test : Nous avons créé un script de test, `test.sh`, qui exécute les tests unitaires en installant préalablement les dépendances nécessaires grâce à Jest. Le contenu détaillé de ce fichier est illustré dans la figure 5.19.

```
#!/bin/bash -ex
# Script to clear Jest cache, install necessary dependencies, and run tests.
# -ex ensures the script runs in verbose mode and exits on error.

set -x
# Enable verbose output for debugging purposes

# Clear Jest cache
npx jest --clearCache
# Command to clear the Jest cache before running tests

# Install dependencies
npm install --save-dev jest-environment-jsdom
# Installing jest-environment-jsdom as a development dependency

# Run tests with --passWithNoTests flag
npm run test -- --passWithNoTests -- --detectOpenHandles
# Command to run tests with --passWithNoTests flag and detect open handles

set +x
# Disable verbose output after tests have completed
```

FIGURE 5.19 : Script de test Jest pour le backend.

Exemple de Test Unitaire : Nous avons également créé des tests unitaires pour différents composants de notre application Node.js avec Express. Un exemple est illustré dans la figure 5.20.

```
const request = require("supertest");
const { app, socketServer } = require("./app");
const ioClient = require("socket.io-client");
const { sequelize } = require("./database/db");

jest.setTimeout(20000);

describe("Express App", () => {
  test("GET / responds with 401", async () => {
    const response = await request(app).get("/");
    expect(response.statusCode).toBe(401);
  });

  test("GET /users responds with 404", async () => {
    const response = await request(app).get("/users");
    expect(response.statusCode).toBe(404);
  });

  test("GET /unknownroute responds with 404", async () => {
    const response = await request(app).get("/unknownroute");
    expect(response.statusCode).toBe(404);
  });
});
```

FIGURE 5.20 : Exemple de test unitaire avec Jest pour le backend.

5.5.3.3 Analyse de Code avec SonarQube

Pour garantir la qualité du code, nous avons intégré SonarQube dans notre pipeline CI/CD pour le projet backend « `communicFlow-back` ». L'analyse de SonarQube a identifié les bugs, les vulnérabilités et les mauvaises pratiques de codage, permettant de maintenir une base de code propre et maintenable.

Résultats de l'Analyse SonarQube : Les résultats des analyses SonarQube sont présentés dans la figure 5.21.

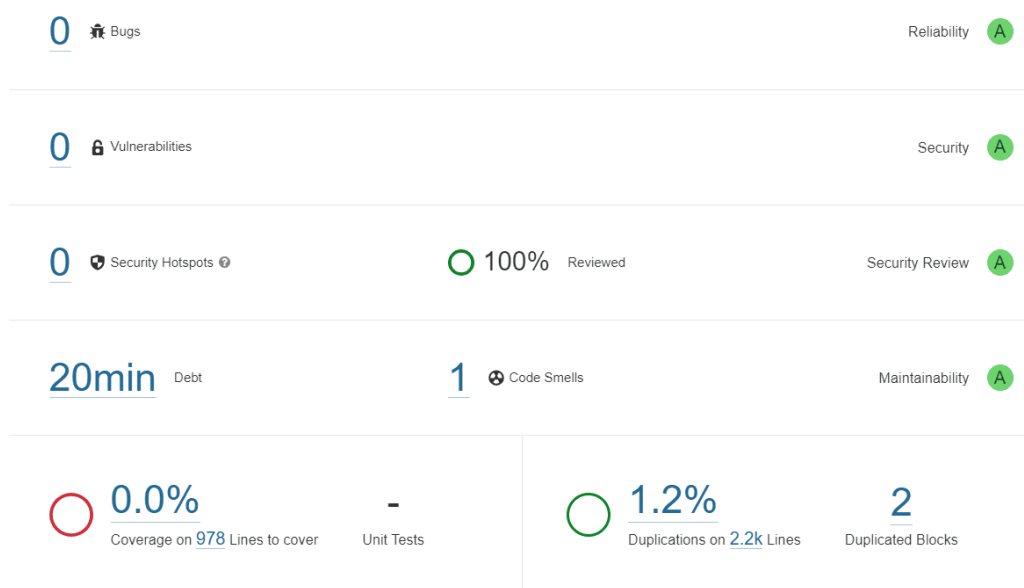


FIGURE 5.21 : Rapport détaillé de SonarQube montrant les bugs, vulnérabilités et code smells

- **Bugs :** Aucune nouvelle anomalie n'a été détectée.
- **Vulnérabilités :** Aucune nouvelle vulnérabilité n'a été détectée.
- **Code Smells :** Des améliorations significatives ont été identifiées en termes de lisibilité et de maintenabilité du code.
- **Coverage :** Le taux de couverture des tests unitaires est en cours d'amélioration.

5.5.3.4 Vérification de la Qualité du Build

Cette étape a vérifié si le code satisfaisait aux exigences de qualité définies par les Quality Gates de SonarQube. Si le code ne passait pas les critères de qualité, le pipeline était interrompu, empêchant ainsi le déploiement de code de mauvaise qualité, comme le montre la figure 5.22.

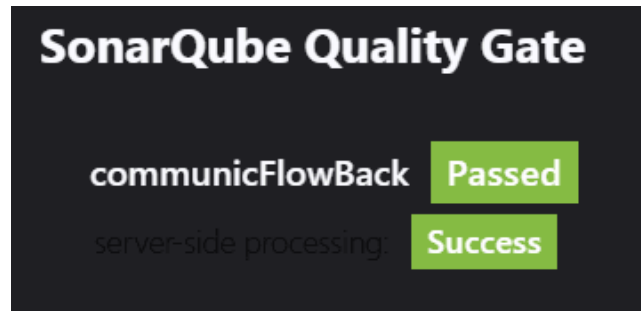


FIGURE 5.22 : Résultats de SonarQube pour le projet backend « communicFlow-back »

5.5.3.5 Vérification des Vulnérabilités avec OWASP Dependency-Check

Pour assurer la sécurité de notre application backend, nous avons intégré OWASP Dependency-Check dans notre pipeline CI/CD. Cet outil identifie les composants ayant des vulnérabilités connues. Le plugin OWASP Dependency-Check génère un rapport détaillé des vulnérabilités trouvées, comme illustré dans la figure 5.23.

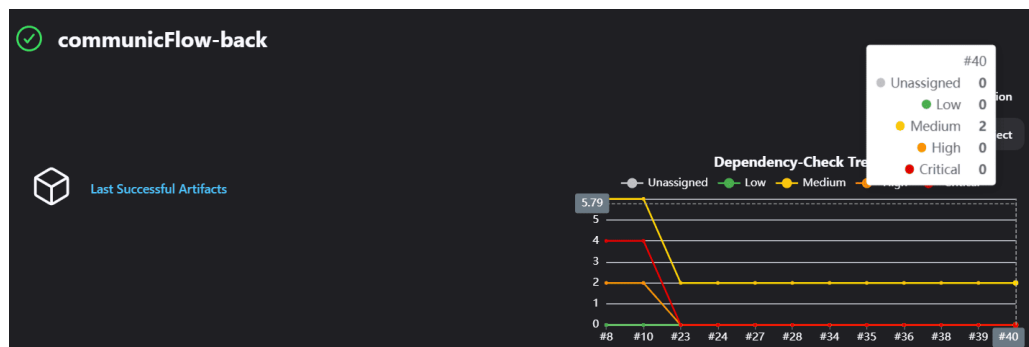


FIGURE 5.23 : Résultats de OWASP Dependency-Check dans Jenkins

Résultats de l'Analyse de Vulnérabilités : Les résultats de l'analyse de Dependency-Check sont essentiels pour identifier et corriger les vulnérabilités dans les dépendances du projet. Voici un résumé des résultats obtenus lors de notre dernière analyse, illustré dans la figure 5.24 :

- **Nombre de vulnérabilités trouvées :** 2 vulnérabilités moyennes.
- **Composants vulnérables :** Liste des bibliothèques et versions ayant des vulnérabilités connues.
- **Actions correctives :** Mise à jour des dépendances vers des versions sécurisées, suppression des dépendances obsolètes ou non sécurisées.

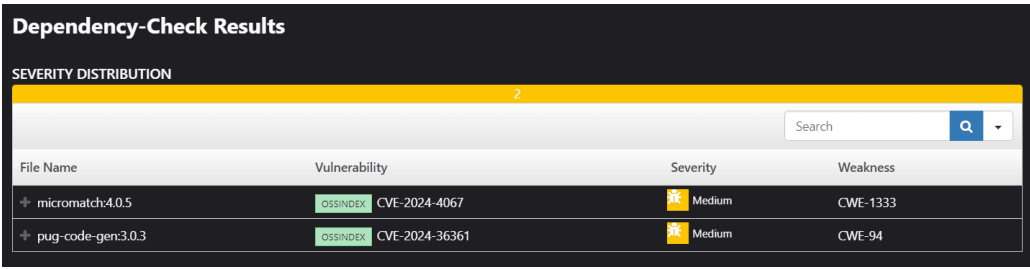


FIGURE 5.24 : Rapport détaillé de OWASP Dependency-Check montrant les vulnérabilités identifiées

Impact de la Vérification des Vulnérabilités : L'intégration de OWASP Dependency-Check dans notre pipeline CI/CD backend présente plusieurs avantages :

- **Détection précoce des vulnérabilités :** Identification rapide des dépendances vulnérables avant qu'elles ne soient déployées en production.
- **Réduction des risques de sécurité :** Correction proactive des vulnérabilités pour minimiser les risques de sécurité.
- **Amélioration de la confiance :** Maintien d'un niveau élevé de sécurité et de confiance dans notre base de code.

5.5.3.6 Archivage de l'Espace de Travail

L'archivage de l'espace de travail est une étape cruciale pour préserver les artefacts générés au cours du processus CI/CD. Cela inclut les fichiers de build, les rapports de tests et autres fichiers nécessaires pour la prochaine étape du pipeline ou pour des audits futurs. Cette étape assure la traçabilité et permet de récupérer facilement les artefacts si nécessaire. Pour automatiser ce processus dans notre pipeline backend `communicFlow-back`, nous avons utilisé le plugin "Copy Artifact" de Jenkins.

Résultats de l'Archivage L'archivage des artefacts a produit plusieurs avantages clés :

- **Traçabilité :** Tous les artefacts des builds sont sauvegardés, permettant de retracer les changements et les résultats de chaque build.
- **Accès aux Artefacts :** Les développeurs peuvent facilement accéder aux artefacts archivés pour analyse ou débogage.
- **Préparation pour le Déploiement :** Les artefacts nécessaires pour le déploiement sont disponibles et prêts à être utilisés dans les étapes suivantes du pipeline de déploiement.

Impact de l'Archivage L'archivage de l'espace de travail assure que les artefacts sont toujours disponibles pour les étapes ultérieures du pipeline et pour les audits futurs. Cela améliore la robustesse et la fiabilité du processus CI/CD en garantissant que les informations nécessaires sont préservées à chaque étape, comme le montre également la figure 5.25.

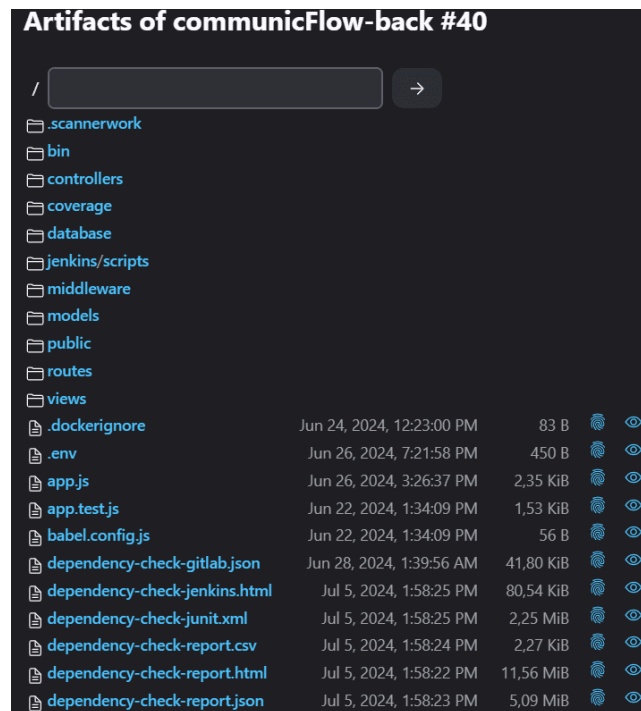


FIGURE 5.25 : Capture d'écran montrant les artefacts archivés dans Jenkins pour le backend

En conclusion, l'archivage de l'espace de travail dans notre pipeline CI/CD backend assure une gestion efficace des artefacts, facilitant ainsi la traçabilité et la préparation pour les déploiements futurs.

5.5.3.7 Déclenchement du Pipeline de Déploiement

À la fin du pipeline CI/CD backend, nous déclenchons automatiquement le pipeline de déploiement pour déployer les artefacts générés dans l'environnement approprié.

5.5.3.8 Synthèse

Le pipeline CI/CD pour le backend, développé en Node.js avec Express, a assuré la validation, le test et le déploiement automatiques de notre code, améliorant la sécurité et la stabilité grâce à des analyses de code et de vulnérabilités rigoureuses, comme illustré dans la figure 5.26.

```

pipeline {
  agent any
  tools {
    nodejs "node"
  }
  stages {
    stage('Install Dependencies') {
      steps {
        bat 'npm install'
      }
    }
    stage('Test') {
      steps {
        bat 'sh ./jenkins/scripts/test.sh'
      }
    }
    stage('SonarQube Analysis') {
      environment {
        SCANNER_HOME = tool 'SonarQube'
      }
      steps {
        withSonarQubeEnv(installationName: 'sonar-jenkins') {
          bat "${SCANNER_HOME}/bin/sonar-scanner"
        }
      }
    }
    stage('Quality Gate Check') {
      steps {
        echo 'Quality Gate check....'
        timeout(time: 10, unit: 'MINUTES') {
          waitForQualityGate abortPipeline: true
        }
      }
    }
    stage('OWASP Dependency-Check Vulnerabilities') {
      steps {
        dependencyCheck additionalArguments: ''
          -o './'
          -s './'
          -f 'ALL'
          --prettyPrint'', odcInstallation: 'OWASP Dependency-Check Vulnerabilities'

        dependencyCheckPublisher pattern: 'dependency-check-report.xml'
      }
    }
    stage('Archive Workspace') {
      steps {
        archiveArtifacts artifacts: '**/*', excludes: 'node_modules/**', allowEmptyArchive: true
      }
    }
    stage('Trigger deployment Pipeline') {
      steps {
        script {
          build job: 'deployment',
            wait: false
        }
      }
    }
  }
}

```

FIGURE 5.26 : Code du Jenkinsfile utilisé pour le pipeline CI/CD backend.

5.5.4 Pipeline de Déploiement

Le pipeline de déploiement est une étape cruciale de notre projet, assurant que toutes les parties de l'application sont correctement déployées et opérationnelles. Cette section décrit en détail chaque étape du pipeline de déploiement, depuis la copie des artefacts jusqu'au déploiement final sur le serveur VPS, comme illustré dans la figure 5.27.

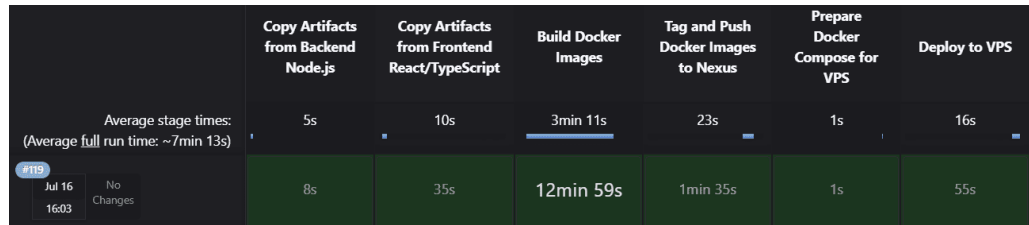


FIGURE 5.27 : Pipeline de déploiement

5.5.4.1 Copie des Artefacts depuis le Backend

Cette étape implique la copie des artefacts générés par le backend Node.js, afin de les préparer pour la construction des images Docker.

5.5.4.2 Copie des Artefacts depuis le Frontend

De manière similaire, les artefacts générés par le frontend React/TypeScript sont copiés et préparés pour la prochaine étape du pipeline.

5.5.4.3 Construction des Images Docker

Dans cette étape, nous avons construit les images Docker pour le frontend et le backend. Les fichiers Docker illustrés dans les figures 5.28, 5.29 et 5.30 ont été utilisés pour cette tâche.

```
# Utiliser Node.js comme image de base pour la construction
FROM node:18.17.1 AS builder
# Définir le répertoire de travail dans le conteneur
WORKDIR /app
# Copier package.json et package-lock.json dans le conteneur
COPY package*.json ./
# Installer les dépendances
RUN npm install
# Copier le reste du code de l'application dans le conteneur
COPY . .
# RUN npm run build
# Exposer le port
EXPOSE 4000 4001
# Exposer le port pour la communication avec d'autres services
# Commande pour démarrer l'application backend
CMD ["node", "app.js"]
```

FIGURE 5.28 : Dockerfile pour le backend

```
# Utiliser Node.js comme image de base pour la construction
FROM node:18.12.1 AS builder
# Définir le répertoire de travail dans le conteneur
WORKDIR /app
# Copier package.json et yarn.lock dans le conteneur
COPY package.json yarn.lock ./
# Installer les dépendances
RUN yarn install
# Copier le reste du code de l'application dans le conteneur
COPY . .
# Construire le code TypeScript
RUN yarn build -- --mode production
# Étape finale pour servir l'application
FROM nginx:alpine
# Copier le code frontend construit dans NGINX
COPY --from=builder /app/dist /usr/share/nginx/html

COPY --from=builder /app/nginx.conf /etc/nginx/nginx.conf
# Exposer le port 80
EXPOSE 80
# Commande pour exécuter NGINX
CMD ["nginx", "-g", "daemon off;"]
```

FIGURE 5.29 : Dockerfile pour le frontend

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include      /etc/nginx/mime.types;
    default_type application/octet-stream;

    sendfile      on;
    keepalive_timeout 65;

    server {
        listen      80;
        server_name localhost;

        location / {
            root      /usr/share/nginx/html;
            try_files $uri $uri/ /index.html;
        }

        error_page   500 502 503 504 /50x.html;
        location = /50x.html {
            root      /usr/share/nginx/html;
        }
    }
}
```

FIGURE 5.30 : Configuration NGINX pour le frontend

5.5.4.4 Tag et Push des Images vers Nexus

Une fois les images Docker construites, elles sont taguées et poussées vers le registre Nexus, comme illustré dans les figures 5.31 et 5.32.

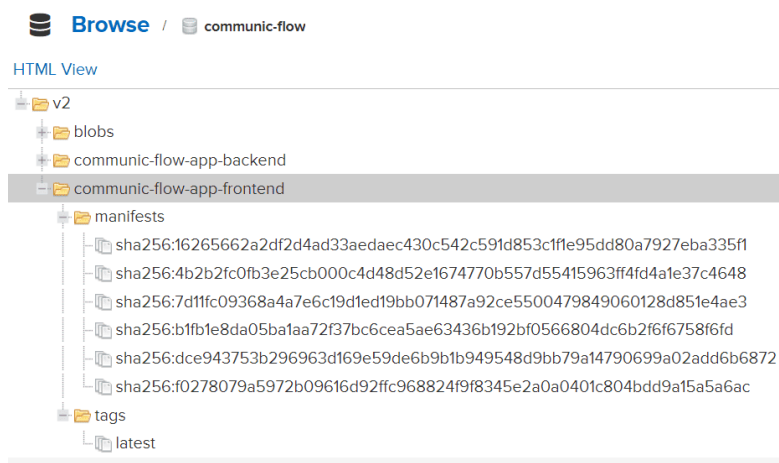


FIGURE 5.31 : Images Docker du frontend dans Nexus

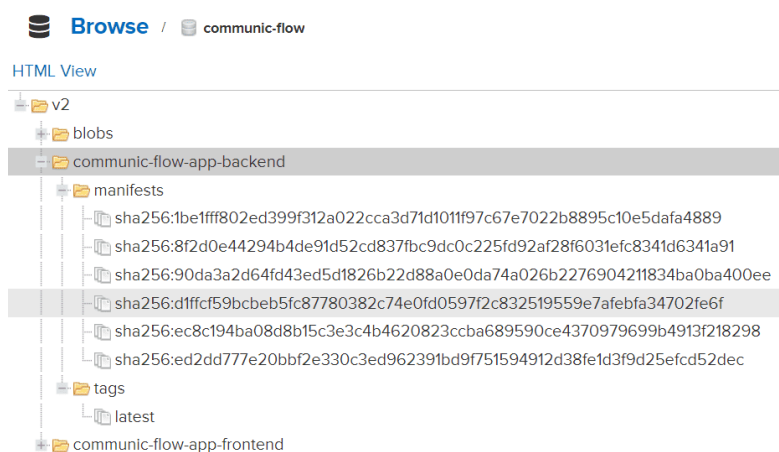


FIGURE 5.32 : Images Docker du backend dans Nexus

5.5.4.5 Préparation du Docker Compose pour le VPS

Pour orchestrer les conteneurs sur le VPS, un fichier Docker Compose est préparé, comme montré dans la figure 5.33.

```

version: "3.8"

services:
  frontend:
    image: 109.176.197.199:8083/communic-flow-app-frontend:latest
    ports:
      - "3001:80"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    image: 109.176.197.199:8083/communic-flow-app-backend:latest
    env_file:
      - .env
    ports:
      - "4000:4000"
      - "4001:4001"
    networks:
      - app-network

networks:
  app-network:
    driver: bridge

```

FIGURE 5.33 : Fichier Docker Compose pour le VPS

5.5.4.6 Déploiement sur le VPS

Enfin, les conteneurs sont déployés sur le VPS, comme illustré dans les figures 5.34 et 5.35.

```

root@srv529365:~# docker images -a
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
109.176.197.199:8083/communic-flow-app-frontend    latest      767642feadb    7 minutes ago  46.6MB
109.176.197.199:8083/communic-flow-app-backend     latest      2b68e3ae5743   13 minutes ago 1.21GB
prom/prometheus   latest      b74abbcc4eac    4 weeks ago   271MB
sonatype/nexus3   latest      a5dc276ca9e6    6 weeks ago   567MB
phpmyadmin         latest      87a2498a12ae    2 months ago   562MB
grafana/grafana    latest      cd2b3782ca63    2 months ago   429MB
postgres           13         32e42e68e624    2 months ago   419MB
mysql              latest      e9387c13ed83    2 months ago   578MB
sonarqube          lts        47031001e158    2 months ago   603MB
root@srv529365:~#

```

FIGURE 5.34 : Images Docker sur le VPS

```

root@srv529365:~# docker ps -a
CONTAINER ID   IMAGE                                     COMMAND                  CREATED        STATUS        PORTS
79231cb09af    109.176.197.199:8083/communic-flow-app-frontend:latest /docker-entrypoint.sh   About a minute Up About a minute 0.0.0.0:3001->80/t
cp, :::8081->80/tcp
8094bdc01299    109.176.197.199:8083/communic-flow-app-backend:latest /docker-entrypoint.sh   2 minutes ago Up 2 minutes      0.0.0.0:4000->4001-
->4000-4001/tcp, :::4000-4001->4000-4001/tcp
b0d3104b25a     prom/prometheus          /bin/prometheus -r...  5 days ago    Up 5 days      0.0.0.0:9090->9090
/tcp, :::9090->9090/tcp
6f24da49302     sonatype/nexus3          /opt/sonatype/nexus... 3 weeks ago    Up 3 weeks      0.0.0.0:8081->8081
tcp, :::8081->8081/tcp, 0.0.0.0:8083->8083/tcp, :::8083->8083/tcp
7650c1b4088     phpmyadmin               /docker-entrypoint.sh   8 weeks ago    Up 3 weeks      0.0.0.0:8880->88/t
cp, :::8880->88/tcp
b0d9e960722     mysql:latest             /docker-entrypoint.sh   8 weeks ago    Up 13 days      0.0.0.0:3306->3306
/tcp, :::3306->3306/tcp, 33060/tcp
7658625f1b69     grafana/grafana:latest   /run.sh                 2 months ago    Up 3 weeks      0.0.0.0:3000->3000
/tcp, :::3000->3000/tcp
a0fca39795c     sonarqube:lts            /opt/sonarqube/dock... 2 months ago    Up 3 weeks      0.0.0.0:9000->9000
/tcp, :::9000->9000/tcp
a37085d1cb77     postgres:13              sonarqube.db            2 months ago    Up 3 weeks      5432/tcp

```

FIGURE 5.35 : Conteneurs Docker en cours d'exécution sur le VPS

5.5.4.7 Synthèse

Le pipeline de déploiement, comme illustré dans les figures 5.36 et 5.37, a permis d'automatiser le processus de mise en production de notre application. Cela inclut la copie des artefacts, la construction des images Docker, le tag et le push vers Nexus, la préparation du fichier Docker Compose et le déploiement sur le VPS. Grâce à ce pipeline, nous avons pu garantir une livraison continue et fiable de notre application.

```

pipeline {
    agent any

    environment {
        DOCKER_REGISTRY_URL = '109.176.197.199:8083' // URL de votre registre Nexus sur le VPS
        DOCKER_IMAGE_PREFIX = 'communic-flow-app' // Préfixe de vos images Docker
        VPS_IP = '109.176.197.199' // Adresse IP de votre VPS
        VPS_SSH = credentials('vps-ssh-credentials')
        VPS_NEXUS = credentials('docker-nexus-credentials')
    }

    stages {
        stage('Copy Artifacts from Backend Node.js') {
            steps {
                copyArtifacts(
                    projectName: 'communicFlow-back',
                    selector: lastSuccessful(),
                    filter: '**/*',
                    target: 'backend'
                )
            }
        }
        stage('Copy Artifacts from Frontend React/TypeScript') {
            steps {
                copyArtifacts(
                    projectName: 'communicFlow',
                    selector: lastSuccessful(),
                    filter: '**/*',
                    target: 'frontend'
                )
            }
        }
        stage('Build Docker Images') {
            steps {
                script {
                    dir('.') {
                        bat 'docker-compose build'
                    }
                }
            }
        }
        stage('Tag and Push Docker Images to Nexus') {
            steps {
                script {
                    withCredentials([usernamePassword(credentialsId: 'docker-nexus-credentials',
passwordVariable: 'DOCKER_PASSWORD', usernameVariable: 'DOCKER_USERNAME')]) {
                        def loginCommand = "echo ${DOCKER_PASSWORD} | docker login -u ${DOCKER_USERNAME}
--password-stdin ${DOCKER_REGISTRY_URL}"
                        powershell returnStdout: true, script: loginCommand
                    }

                    def images = ['frontend', 'backend']
                    images.each { image ->
                        def builtImageName = "deployment-${image}"
                        def fullImageName =
"${DOCKER_REGISTRY_URL}/${DOCKER_IMAGE_PREFIX}-${image}:latest"
                        echo "Tagging image: docker tag ${builtImageName} ${fullImageName}"
                        bat "docker tag ${builtImageName} ${fullImageName}"
                        echo "Pushing image: docker push ${fullImageName}"
                        bat "docker push ${fullImageName}"
                    }
                }
            }
        }
    }
}

```

FIGURE 5.36 : Première partie du Jenkinsfile pour le pipeline de déploiement


```

        stage('Prepare Docker Compose for VPS') {
            steps {
                script {
                    def composeFileContent = """
version: "3.8"

services:
  frontend:
    image: ${DOCKER_REGISTRY_URL}/${DOCKER_IMAGE_PREFIX}-frontend:latest
    ports:
      - "3001:80"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    image: ${DOCKER_REGISTRY_URL}/${DOCKER_IMAGE_PREFIX}-backend:latest
    env_file:
      - .env
    ports:
      - "4000:4000"
      - "4001:4001"
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
"""

                    writeFile file: 'docker-compose-vps.yml', text: composeFileContent.trim()

                }
            }
        }

        stage('Deploy to VPS') {
            steps {
                script {
                    def remote = [:]
                    remote.name = 'vps'
                    remote.host = "${VPS_IP}"
                    remote.user = env.VPS_SSH_USR
                    remote.password = env.VPS_SSH_PSW
                    remote.allowAnyHosts = true

                    // Copy the docker-compose file to the VPS
                    sshPut remote: remote, from: 'docker-compose-vps.yml', into:
'/home/communic_flow/docker-compose.yml'
                    sshPut remote: remote, from: '.env', into: '/home/communic_flow/.env'
                    // Pull and run Docker containers on VPS
                    sshCommand remote: remote, command: """
                        docker login -u ${env.VPS_NEXUS_USR} -p ${env.VPS_NEXUS_PSW}
${DOCKER_REGISTRY_URL}
                        cd /home/communic_flow
                        docker pull 109.176.197.199:8083/communic-flow-app-frontend:latest
                        docker pull 109.176.197.199:8083/communic-flow-app-backend:latest
                        docker-compose up -d
                    """

                }
            }
        }
    }
}

```

FIGURE 5.37 : Deuxième partie du Jenkinsfile pour le pipeline de déploiement

5.5.5 Monitoring

Le monitoring est une composante essentielle de notre projet, nous permettant de surveiller en temps réel l'état de l'application et des infrastructures associées. Nous avons mis en place un tableau de bord Grafana, illustré dans la figure 5.38, pour centraliser les métriques clés et les visualiser de manière intuitive.

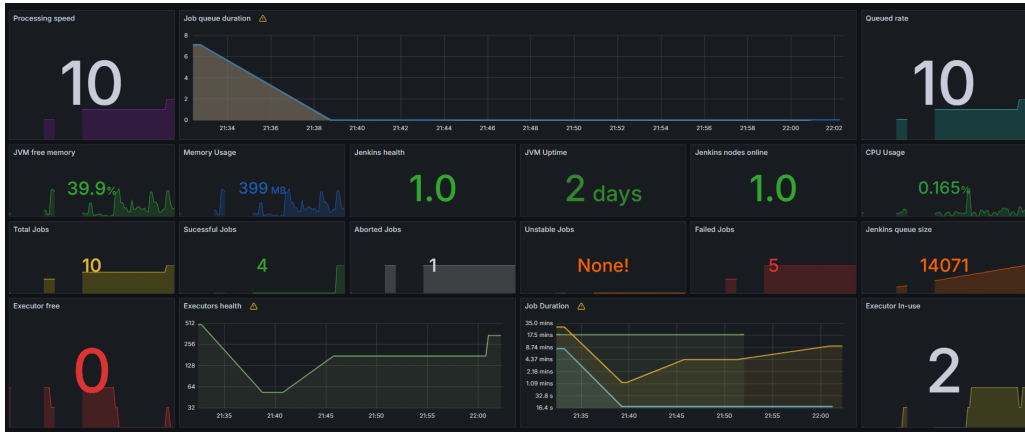


FIGURE 5.38 : Tableau de bord Grafana pour le monitoring des performances et de la santé de Jenkins

Le tableau de bord Grafana présente plusieurs panneaux importants :

- **Processing Speed** : Affiche la vitesse de traitement des tâches Jenkins. Une vitesse de traitement rapide est essentielle pour maintenir des temps de construction courts.
- **Job Queue Duration** : Montre la durée d'attente des tâches dans la file d'attente. Un temps d'attente élevé peut indiquer des goulets d'étranglement dans le traitement des tâches.
- **Queued Rate** : Indique le taux d'augmentation des tâches en file d'attente.
- **JVM Free Memory** : Surveille la mémoire libre de la JVM. Une gestion efficace de la mémoire est cruciale pour la performance de Jenkins.
- **Memory Usage** : Affiche l'utilisation de la mémoire par Jenkins.
- **JVM Uptime** : Indique le temps de fonctionnement de la JVM. Une longue durée de fonctionnement sans redémarrage peut être un signe de stabilité.
- **Jenkins Health** : Fournit une évaluation générale de la santé de Jenkins.
- **Jenkins Nodes Online** : Montre le nombre de nœuds Jenkins en ligne et disponibles pour le traitement des tâches.
- **CPU Usage** : Indique l'utilisation du CPU par Jenkins. Une utilisation élevée peut indiquer une surcharge du serveur.
- **Total Jobs** : Affiche le nombre total de tâches exécutées.

- **Successful Jobs** : Indique le nombre de tâches terminées avec succès.
- **Aborted Jobs** : Montre le nombre de tâches annulées, ce qui peut indiquer des problèmes dans le processus de build.
- **Unstable Jobs** : Affiche le nombre de tâches instables, nécessitant une attention pour améliorer la fiabilité.
- **Failed Jobs** : Indique le nombre de tâches échouées, nécessitant une analyse et des corrections.
- **Jenkins Queue Size** : Montre la taille de la file d'attente Jenkins, permettant d'identifier les périodes de forte demande.
- **Executor Free** : Indique le nombre d'exécuteurs disponibles.
- **Executors Health** : Fournit une vue d'ensemble de la santé des exécuteurs.
- **Job Duration** : Affiche la durée des tâches, permettant d'identifier les tâches qui prennent plus de temps que prévu.
- **Executor In-use** : Indique le nombre d'exécuteurs actuellement utilisés.

Ce tableau de bord nous permet de surveiller en temps réel les performances et la santé de l'infrastructure Jenkins. En centralisant ces informations, nous pouvons rapidement identifier et résoudre les problèmes, optimisant ainsi le processus de développement et de déploiement continu.

5.6 Conclusion

Dans ce chapitre, nous avons exploré les concepts d'intégration continue (CI), de livraison continue (CD) et de déploiement continu. Nous avons également examiné la mise en place d'une infrastructure CI/CD robuste avec des outils comme Jenkins, SonarQube, Nexus Repository, Prometheus et Grafana. Ces outils ont été essentiels pour automatiser et améliorer l'efficacité de notre cycle de développement, garantissant des déploiements rapides et fiables. Le monitoring avec Grafana nous a permis de surveiller en temps réel l'état de notre infrastructure et d'identifier rapidement les problèmes. Grâce à cette approche, nous avons pu garantir une livraison continue et fiable de notre application.

Conclusion générale et perspectives

Dans un contexte où les organisations modernes nécessitent une gestion centralisée et une communication efficace, notre projet a visé à créer une solution innovante pour répondre à ces besoins. La centralisation et l'intégration de plusieurs services essentiels au sein d'une plateforme unique ont démontré une réelle capacité à améliorer la gestion de l'information, la communication, et la collaboration entre différents acteurs institutionnels.

Le Dashboard que nous avons conçu intègre des fonctionnalités avancées qui permettent la création et la gestion d'organisations, l'administration de leurs membres, la gestion des dossiers et des fichiers, ainsi que le partage de rapports. Il inclut également un calendrier pour la gestion des événements, permettant de créer et de suivre des événements importants. De plus, la messagerie instantanée assure une communication en temps réel, tandis que le système de gestion des réclamations facilite la résolution rapide des problèmes en permettant une interaction directe entre l'administrateur et le super administrateur. Ces outils sont spécifiquement conçus pour améliorer l'efficacité, la productivité et la prise de décision au sein des organisations.

La version bêta de l'application a montré des résultats prometteurs, validant non seulement la faisabilité technique de cette intégration mais aussi son adaptabilité aux besoins spécifiques des utilisateurs. L'adoption par des organisations telles que la Chambre Nationale de l'Industrie Pharmaceutique (CNIP) témoigne de la pertinence de notre solution dans un contexte professionnel exigeant.

Pour garantir la qualité continue de notre application, nous avons mis en place un pipeline CI/CD. Ce pipeline permet d'automatiser les tests, les validations, et les déploiements, assurant ainsi que chaque modification apportée au code est rigoureusement testée avant d'être mise en production. Cette approche renforce la fiabilité du système, réduit les risques d'erreurs en production, et permet une livraison plus rapide des nouvelles fonctionnalités.

L'avenir de cette plateforme est orienté vers l'enrichissement des fonctionnalités afin de répondre aux besoins évolutifs des organisations. Les priorités incluent le développement de la gestion des utilisateurs, avec une attention particulière aux privilèges d'accès, à la sécurité des données, et au contrôle de la visibilité des informations. Nous prévoyons également d'ajouter un système de notifications pour améliorer la réactivité des utilisateurs ainsi que de développer une application mobile pour offrir un accès plus flexible à la plateforme. Ces améliorations garantiront non seulement une meilleure sécurité mais aussi une flexibilité accrue, permettant à la plateforme de s'adapter aux évolutions technologiques

et aux exigences croissantes des utilisateurs.

En somme, notre projet a posé des bases solides pour une solution capable de transformer la gestion organisationnelle en répondant aux défis actuels et futurs.

Netographie

- [1] ASQII, *What is ASQII ?* Consulté en juillet 2024. adresse : <https://asqii.tn/en/>.
- [2] GOOGLE, *Google Drive*, Consulté en juillet 2024. adresse : <https://www.google.com/drive/>.
- [3] ZENDESK, *Zendesk*, Consulté en juillet 2024. adresse : <https://www.zendesk.com/>.
- [4] GOOGLE, *Google Calendar*, Consulté en juillet 2024. adresse : <https://www.google.com/calendar/>.
- [5] S. TECHNOLOGIES, *Slack*, Consulté en juillet 2024. adresse : <https://slack.com/>.
- [6] S. SUTHERLAND, *The Scrum Guide*, Consulté en mai 2024. adresse : <https://www.scrum.org/resources/scrum-guide>.
- [7] RUBIN, *Essential Scrum : A Practical Guide to the Most Popular Agile Process*, Consulté en mai 2024. adresse : <https://www.informit.com/store/essential-scrum-a-practical-guide-to-the-most-popular-9780137043293>.
- [8] SCHWABER, *Agile Project Management with Scrum*, Consulté en mai 2024. adresse : <https://www.microsoftpressstore.com/store/agile-project-management-with-scrum-9780735619937>.
- [9] COHN, *User Stories Applied : For Agile Software Development*, Consulté en mai 2024. adresse : <https://www.informit.com/store/user-stories-applied-for-agile-software-development-9780321205681>.
- [10] O. M. GROUP, *UML Specification*, Consulté en juillet 2024. adresse : <https://www.omg.org/spec/UML/>.
- [11] REDUX, *Redux Store*, Consulté en mai 2024. adresse : <https://redux.js.org/introduction/getting-started>.
- [12] REACT, *Component*, Consulté en mai 2024. adresse : <https://legacy.reactjs.org/docs/components-and-props.html>.
- [13] CLOUDFLARE, *Communication*, Consulté en mai 2024. adresse : <https://www.cloudflare.com/fr-fr/learning/ssl/what-is-https/>.
- [14] SOCKET.IO, *Socket io*, Consulté en mai 2024. adresse : <https://socket.io/>.
- [15] SEQUELIZE, *Sequelize*, Consulté en mai 2024. adresse : <https://sequelize.org/>.
- [16] NODEMAILER, *Nodemailer*, Consulté en mai 2024. adresse : <https://nodemailer.com/>.
- [17] CLOUDINARY, *Cloudinary*, Consulté en mai 2024. adresse : <https://cloudinary.com/>.

- [18] V. S. CODE, *Visual Studio Code (VS Code)*, Consulté en mai 2024. adresse : <https://code.visualstudio.com/docs>.
- [19] BILITY, *Postman*, Consulté en mai 2024. adresse : <https://bility.fr/definition-postman/>.
- [20] A. FRIENDS, *XAMPP*, Consulté en juin 2024. adresse : <https://www.apachefriends.org/fr/index.html>.
- [21] L. M. IT, *GitHub*, Consulté en juin 2024. adresse : <https://www.lemagit.fr/definition/GitHub>.
- [22] L. B. DATA, *Jenkins*, Consulté en juin 2024. adresse : <https://www.lebigdata.fr/jenkins-definition-avantages>.
- [23] PROMETHEUS, *Prometheus*, Consulté en mai 2024. adresse : <https://prometheus.io/docs/introduction/overview/>.
- [24] R. HAT, *Grafana*, Consulté en juin 2024. adresse : <https://www.redhat.com/fr/topics/data-services/what-is-grafana>.
- [25] SYLOE, *SonarQube*, Consulté en juin 2024. adresse : <https://www.syloe.com/glossaire/sonarqube/>.
- [26] A. INFORMATIQUE, *OWASP*, Consulté en juin 2024. adresse : <https://actualiteinformatique.fr/cybersecurite/quest-ce-que-owasp>.
- [27] SONATYPE, *Nexus*, Consulté en juin 2024. adresse : <https://www.sonatype.com/products/repository-oss>.
- [28] NGROK, *ngrok*, Consulté en juin 2024. adresse : <https://ngrok.com/docs>.
- [29] OVERLEAF, *Overleaf*, Consulté en juin 2024. adresse : <https://www.overleaf.com/learn>.
- [30] DRAW.IO, *Draw.io*, Consulté en juin 2024. adresse : <https://www.diagrams.net/documents>.
- [31] FIGMA, *Figma*, Consulté en juin 2024. adresse : <https://help.figma.com/hc/en-us>.
- [32] NGINX, *Nginx*, Consulté en juin 2024. adresse : <https://docs.nginx.com/>.
- [33] REACT, *React*, Consulté en juin 2024. adresse : <https://reactjs.org/docs/>.
- [34] TYPESCRIPT, *TypeScript*, Consulté en juin 2024. adresse : <https://www.typescriptlang.org/docs/handbook/intro.html>.
- [35] R. ROUTER, *React Router DOM*, Consulté en juin 2024. adresse : <https://reactrouter.com/en/main/start/overview>.
- [36] AXIOS, *Axios*, Consulté en juin 2024. adresse : <https://axios-http.com/docs/intro>.

- [37] NODE.JS, *Node.js*, Consulté en juin 2024. adresse : <https://nodejs.org/en/docs/>.
- [38] EXPRESS.JS, *Express.js*, Consulté en juin 2024. adresse : <https://expressjs.com/en/starter/installing.html>.
- [39] SEQUELIZE, *Sequelize*, Consulté en juin 2024. adresse : <https://sequelize.org/docs/v6/getting-started/>.
- [40] SQL, *SQL*, Consulté en juin 2024. adresse : <https://sql.sh/>.
- [41] NODEMAILER, *Nodemailer*, Consulté en juin 2024. adresse : <https://nodemailer.com/about/>.
- [42] SOCKET.IO, *Socket.io*, Consulté en juin 2024. adresse : <https://socket.io/docs/v4/>.
- [43] YARN, *Yarn*, Consulté en juin 2024. adresse : <https://classic.yarnpkg.com/en/docs/getting-started>.
- [44] NPM, *npm*, Consulté en juin 2024. adresse : <https://docs.npmjs.com/>.
- [45] VITE, *Vite*, Consulté en juin 2024. adresse : <https://vitejs.dev/guide/>.
- [46] GIT, *Git*, Consulté en juin 2024. adresse : <https://git-scm.com/doc>.
- [47] DOCKER, *Docker*, Consulté en juin 2024. adresse : <https://docs.docker.com/get-started/>.
- [48] JEST, *Jest*, Consulté en juin 2024. adresse : <https://jestjs.io/docs/getting-started>.
- [49] M. FOWLER, *Continuous Integration*, Consulté en mai 2024. adresse : <https://martinfowler.com/articles/continuousIntegration.html>.
- [50] THOUGHTWORKS, *Continuous Delivery*, Consulté en mai 2024. adresse : <https://www.thoughtworks.com/continuous-delivery>.
- [51] AWS, *What is DevOps?* Consulté en mai 2024. adresse : <https://aws.amazon.com/devops/what-is-devops/>.
- [52] VMWARE, *Virtualization Overview*, Consulté en mai 2024. adresse : <https://www.vmware.com/solutions/virtualization.html>.
- [53] DOCKER, *What is Containerization?* Consulté en mai 2024. adresse : <https://www.docker.com/resources/what-container/>.

Résumé

Ce stage s'inscrit dans le cadre de mon projet de fin d'études en vue d'obtenir le diplôme national d'ingénieur en informatique de l'Ecole Supérieure Privée de technologies et d'ingénierie TEKUP. Il porte sur le développement d'un dashboard organisationnel multiniveau. Ce tableau de bord vise à centraliser les données et à améliorer la communication au sein d'organisations institutionnelles, facilitant ainsi la gestion des informations, la collaboration, et la prise de décision grâce à des fonctionnalités avancées telles que le partage de rapports Power BI, la gestion des réclamations, et la messagerie instantanée.

Mots clés : Dashboard organisationnel, centralisation des données, communication institutionnelle, gestion des organisations, Power BI, messagerie instantanée, DevOps, Docker, Jenkins, CI/CD, React.js, Node.js, TypeScript, MySQL, SocketIO etc

Abstract

This internship is part of my final-year project to obtain the national engineering degree from the Private School of Technology and Engineering TEKUP. It focuses on the development of a multi-level organizational dashboard. The dashboard is designed to centralize data and improve communication within institutional organizations, enhancing information management, collaboration, and decision-making through advanced features like Power BI report sharing, claim management, and instant messaging.

Keywords : Organizational dashboard, data centralization, institutional communication, organization management, Power BI, instant messaging, DevOps, Docker, Jenkins, CI/CD, React.js, Node.js, TypeScript, MySQL, SocketIO etc